ID1110 Introduction to Programming (in Python)

Executing Python Code

There are two ways in which we can **run a python code**

1. Interactive mode

- Code written line by line or as blocks
- Interpreter reads and executes commands interactively
- Helps understand execution of the code
- Not suitable for large programs

2. Script mode

- Program is written in a .py file
- Interpreter reads the file and then executes it
- Suitable for writing large programs

Basic Elements of Programming in Python

- Input and Output - reading data and returning/writing results
- Variables and Data Structures - storing and manipulating data
  - Python supports different kinds (types) of data
- Conditional Statements - making choices based on conditions
- Loops - repeated execution of code
- Functions - reusable named blocks of code
- Arithmetic, Logical, Comparison (Relational) and Membership Operators
- Assignment Operators and Bitwise Operators

Let's begin with how Python represents numbers and learn to perform arithmetic operations in Python. Most of the code snippets from this notebook are from `Think Python (Edition 3)` by Allen B. Downey.

Arithmetic Operators and Expressions

- An **arithmetic operator** is a symbol that represents an arithmetic computation
- An **operand** is one of the values on which an operator operates
- An **arithmetic expression** consist of operands combined by arithmetic operations

```
30 + 12
```

```
42
```

View this line also as an **instruction** to execute the corresponding arithmetic computation

```
43 - 1
```

```
42
```

```
6 * 8
```

```
48
```

```
85 / 2
```

```
42.5
```

There are two types of numbers in Python

- **integers**, which represent positive and negative integers
- **floating-point numbers**, which represent integers (positive and negative), rational numbers and real numbers

If you add, subtract, or multiply two integers, the result is an integer. But if you divide two integers, the result is a floating-point number.

Integer division or floor division (rounds down toward the "floor")

```
-85 // 2
```

```
-43
```

Exponentiation

```
84 ** 2
```

```
7056
```

Modulus or Remainder operation

```
17 % 3
```

```
2
```

```
6 + 6 ** 2
```

```
42
```

Order of operations: exponentiation, multiplication and division, addition and subtraction

```
(6 + 6) ** 2
```

```
144
```

```
2 ** 4 ** 3
```

```
18446744073709551616
```

```
2 ** (4 ** 3)
```

```
18446744073709551616
```

```
(2 ** 4) ** 3
```

```
4096
```

```
6 / 4 / 3
```

```
0.5
```

```
(6 / 4)/ 3
```

```
0.5
```

```
2**(2**10)
```

```
179769313486231590772930519078902473361797697894230657273430081157732675805500963132708477322407536021120113879871393357658
```

```
2**(-2**(12))
```

```
0.0
```

The value is too small to distinguish from zero

```
2**(-2**(10))
```

```
5.562684646268003e-309
```

Arithmetic Functions

A **function** is a named reusable block of code that performs a specific task

`round` is a built-in function that takes a floating-point number and rounds it off to the nearest integer

```
round(-41.5)
```

```
-42
```

The `abs` function computes the absolute value of a number

```
But in some programming languages (like Python)
round(41.5) → 42
However: round(40.5) → 40

That's because Python uses "round half to even" (also called banker's rounding):

.5 goes to the nearest even number
```

```
  Cell In[19], line 2
    round(41.5) → 42
                  ^
SyntaxError: invalid character '→' (U+2192)
```

```
abs(-42)
```

When we use a function like above, we say we're **calling** the function

An expression or part of an expression that runs/calls a function is a **function call**

A function call consists of the function name followed by an **argument list** in parentheses

When you call a function, the parentheses are required and if you leave them out, you get an error message

```
abs 42
```

The first line mentions the cell and line in that cell that has an error.

The second line is the code that contains the error, with a caret (`^`) beneath it to indicate where the error was discovered.

The last line indicates that this is a **syntax error**, which means that there is something wrong with the structure of the expression.

In this example, the problem is that a function call requires parentheses.

Let's see what happens if you leave out the parentheses *and* the value.

```
abs
```

A function name all by itself is a legal expression that **has a value**. When it's displayed, the value indicates that `abs` is a function, and it includes some additional information.

Functions may or may not take arguments and may or may not produce a result.

We can define our own functions too! Stay tuned ...

Strings and Operations on Strings

In addition to numbers, Python can also represent sequences of letters, which are called **strings**.

To write a string, we can put a sequence of letters inside straight quotation marks.

```
'Hello'
```

```
'Hello'
```

It is also legal to use double quotation marks.

```
"world"
```

```
'world'
```

Double quotes make it easy to write a string that contains an apostrophe, which is the same symbol as a straight quote.

```
"it's a small "
```

```
"it's a small "
```

```
"Welcome to Introduction"" to Programming!"
```

```
'Welcome to Introduction to Programming!'
```

The `+` operator works with strings; it joins two strings into a single string, which is called **concatenation**.

```
'Well, ' + "it's a small " + 'world.'
```

```
"Well, it's a small world."
```

The `*` operator also works with strings; it makes multiple copies of a string and concatenates them.

```
'Spam, ' * 4
```

```
'Spam, Spam, Spam, Spam, '
```

The other arithmetic operators don't work with strings.

Python provides a function called `len` that computes the length of a string.

```
len('Spam')
```

```
4
```

Notice that `len` counts the letters between the quotes, but not the quotes.

When you create a string, be sure to use straight quotes. The back quote (Option + ` in Mac), also known as a backtick, causes a syntax error.

```
`Hello`
```

```
  Cell In[27], line 1
    `Hello`
    ^
SyntaxError: invalid syntax
```

Smart quotes, also known as curly quotes, are also illegal.

```
‘Hello’
```

```
  Cell In[28], line 1
    ‘Hello’
    ^
SyntaxError: invalid character '‘' (U+2018)
```

Writing first words...

```
print("Welcome to Introduction"'to Programming!')
```

```
Welcome to Introductionto Programming!
```

What happens if you forget parenthesis?

```
print "Welcome to Introduction"" to Programming!"
```

```
  Cell In[30], line 1
    print "Welcome to Introduction"" to Programming!"
    ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
```

What happens if you forget quotes?

```
print(Welcome to Introduction to Programming!)
```

```
  Cell In[31], line 1
    print(Welcome to Introduction to Programming!)
          ^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

Programming errors are called **bugs** and the process of tracking and fixing them is called **debugging**

```
print(5+4)
```

```
9
```

```
print("Once you eliminate the impossible, \nwhatever remains, no matter how improbable,\nmust be the truth")
```

```
Once you eliminate the impossible,
whatever remains, no matter how improbable,
must be the truth
```

```
print("I cannot agree with those who rank modesty among the virtues\n"
"To the logician all things should be seen exactly as they are\n"
"and to underestimate one's self is as much a departure from truth\n"
"as to exaggerate one's own powers")
```

```
I cannot agree with those who rank modesty among the virtues
To the logician all things should be seen exactly as they are
and to underestimate one's self is as much a departure from truth
as to exaggerate one's own powers
```

```
print("This isn't magic—it's logic")
```

```
This isn't magic—it's logic
```

Reading...

```
input("What is your name? ")
```

```
What is your name?  siddu
'siddu'
```

```
name = input("What is your name? ")
print(name)
```

```
What is your name?  s
s
```

Syntax: **input(msg)**

where msg is an optional argument which is a string representing a default message before the input

Returns the user's input as a string

```
input()
```

```
sem = int(input("Which semester are you in? "))
print(sem)
```

Runtime error - occurs when syntax is correct but the interpreter is still not able to run the code

Input validation is important!

Code Repetition

To repeat a set of instructions, we use a **loop**

```
for x in range(0, 6):
    print(x)
    print(x+1)
```

The first line is a header that ends with a colon. The second line is the body, which has to be indented.

The header starts with the keyword `for`, a new variable named `i`, and another keyword, `in`.

- `for` is a loop statement
- `in` is a membership operator
- `i` is called the loop variable

It uses the `range` function to create a sequence of values.

When the `for` statement runs, it assigns the first value from `range` to `i` and then runs the `print` function in the body, which displays `0`.

When it gets to the end of the body, it loops back around to the header, which is why this statement is called a **loop**. The second time through the loop, it assigns the next value from `range` to `i`, and displays it.

This process is repeated for every value from `range`. Then the loop ends.

```
for letter in 'Python':
    print(letter, end=' ')
```

`in` is a membership operator that can also be used with strings to check for substring membership.

Write a program to print the factorial of a given number

Procedure to print the factorial of a given number

Input: a positive integer n

1. If n < 0, declare that the factorial function is not defined for non-positive integers and stop
2. Initialize fact = 1
3. For i from 1 to n

   3.1 fact = fact × i

Output: fact

```
n = int(input("Enter a positive integer: "))

if n < 0:
    print("The factorial function is not defined for non-positive integers")
else:
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
    print("Factorial of", n, "is", fact)
```

Write a program to print the first n Fibonacci numbers

A First Attempt

```
print("Fibonacci Sequence")

n = int(input("Enter the number of terms: "))

a = 0
b = 1

print(a)

for i in range(0,n-1):
    print(b)
    a = b
    b = a + b
```

```
Fibonacci Sequence
Enter the number of terms:  4
0
1
2
4
```

Another Attempt

```
print("Fibonacci Sequence")

n = int(input("Enter the number of terms: "))

a = 0
b = 1

for i in range(n):
    print(a)
    c = a + b
    a = b
    b = c
```

```
Fibonacci Sequence
Enter the number of terms:  1
0
```

Comments

As programs get bigger and complex, readability is affected.

Formal languages are dense, and it is often difficult to look at a piece of code and figure out what it is doing and why.

For this reason, it is a good idea to add notes to programs to explain in natural language what the program is doing.

These notes are called **comments**, and they start with the `#` symbol.

```
# Program to display Fibonacci sequence

print("Fibonacci Sequence")

# Read n from the user
n = int(input("Enter the number of terms: "))

# Initialize the current and next Fibonacci numbers
a = 0       # F_0
b = 1       # F_1

for i in range(n):
    # Print the current Fibonacci number F_{i-1}
    print(a)

    # Compute F_{i+1}
    c = a + b

    # Update current Fibonacci number
    a = b

    # Update next Fibonacci number
    b = c
```

Everything from the `#` to the end of the line is ignored - it has no effect on the execution of the program

A comment may appear on a line by itself or you can also put comments at the end of a line

Comments are most useful when they document non-obvious features of the code

Debugging

Three kinds of errors can occur in a program.

- **Syntax error**: "Syntax" refers to the program structure and the rules about that structure
  - An error message is displayed immediately
- **Semantic error**: "Semantic" refers to the meaning
  - No error messages but the program does not work the intended way
- **Runtime error**: Also called an **exception** - it indicates that something exceptional has happened
  - Examples include ValueError, TypeError, ZeroDivisionError, etc.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Variables - names that refer to values
- Expressions (arithmetic, string, boolean)
  - Arithmetic operators: +, -, *, /, //, %, **
  - Relational operators: ==, !=, <, >, <=, >=
  - Logical operators: and, or, not
- Values and Types
- Statements - one or more lines of code that represent a command or action
  - Assignment - a statement that assigns a value to a variable
  - Conditional - gives the ability to change program's behavior based on certain conditions
  - Loops - for repeated execution of a code block
  - ...

Reference: Think Python (Edition 3) by Allen B. Downey

Variables

A **variable** is a name that refers to a value.

So far we've seen three kinds of values

- `2` is an integer,
- `42.0` is a floating-point number, and
- `'Hello'` is a string.

To create a variable, we can write an **assignment statement**.

```
n = 17
pi = 3.141592653589793
message = 'A'
```

An assignment statement has three parts: the name of the variable on the left, the equals operator, `=`, and an expression on the right.

When you run an assignment statement, there is no output.

Python creates the variable and gives it a value, but the assignment statement has no visible effect.

After creating a variable, you can use it as an expression.

And we can display the value of `message` like this (in interactive mode)

```
message
```

```
'A'
```

Or like this (in interactive and script modes)

```
print(message)
```

```
A
```

You can also use a variable as part of an expression with arithmetic operators.

And you can use a variable when you call a function.

```
n + 25 * pi, round(pi), len(message)
```

```
(95.53981633974483, 3, 1)
```

It is legal to make more than one assignment to the same variable. A new assignment makes an existing variable refer to a new value (and stop referring to the old value).

A common kind of assignment is an **update**, where the new value of the variable depends on the old.

```
n = n + 1
```

This statement means "get the current value of `x`, add one, and assign the result back to `x`."

If you try to update a variable that doesn't exist, you get an error, because Python evaluates the expression on the right before it assigns a value to the variable on the left.

```
z = z + 1
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 z = z + 1

NameError: name 'z' is not defined
```

Before you can update a variable, you have to **initialize** it, usually with a simple assignment:

```
z = 0
z = z + 1
z
```

```
1
```

Increasing the value of a variable is called an **increment**; decreasing the value is called a **decrement**. Because these operations are so common, Python provides **augmented assignment operators** that update a variable more concisely. For example, the `+=` operator increments a variable by the given amount.

```
z += 2
z
```

```
3
```

There are augmented assignment operators for the other arithmetic operators, including `-=` and `*=`.

Variable names

- Can be as long as you like but don't keep them too long
- Can contain both letters and numbers
- Can't begin with a number
- Can contain uppercase letters, but it is conventional to use only lower case letters
- Only punctuation that can appear is the underscore character, `_`

If you give a variable an illegal name, you get a syntax error

```
#SyntaxError

million! = 1000000
#contains punctuation

76trombones = 'big parade'
#starts with a number

class = 'Self-Defence Against Fresh Fruit'
#class is a keyword


# Good idea to add comments to programs
# notes to explain in natural language what the program is doing
# Text included in a program that provides information about the program
# but has no effect on its execution
```

```
  Cell In[11], line 6
    76trombones = 'big parade'
     ^
SyntaxError: invalid decimal literal
```

Keywords

A **keyword** is a predefined word that has a special meaning to the interpreter

Variable names cannot be keywords to avoid ambiguity for the interpreter

Here's a complete list of Python's keywords:

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

You don't have to memorize this list. In most development environments, keywords are displayed in a different color; if you try to use one as a variable name, you'll know.

Expressions

An expression can be a **single value**, like an integer, floating-point number, or string.

It can also be a **collection of values** and operators following a structure.

And it can include **variable names** and **function calls**.

```
19 + n + abs(-5.1) * 2
```

```
46.2
```

```
'Introduction ' + 'to ' + 'Programming'
```

```
'Introduction to Programming'
```

Evaluating an expression is performing the operations in the expression in order to compute a value

Values and Types

Every expression has a **value**. For example, the expression `6 * 7` has the value `42`.

Every value has a **type** -- or we sometimes say it "belongs to" a type.

Python keeps track of names dynamically

- Values are assigned as they come
- Values have types, names inherit their type from the value
- Names inherit their type from the values they currently hold
    - Not a good practice to use the same name for different types of values

Variables, values and types Variables (names) have no intrinsic types Values have types A variable inherits the type of the value it currently holds The type of value a variable holds can vary over time But not a good idea to use the same name for different types of values in the same piece of code Reduces readability, maintainability The type() function returns the type of a variable that is currently assigned a value

The function del() unassigns a value from a name

Python provides a function called `type` that tells you the type of any value.

```
type(2)    #The type of an integer is `int`.

type(42.0) #The type of a floating-point number is `float`.

type('Hello, World!') #The type of a string is `str`.

type('126')
```

```
# A kind of value is called a **type**.
```

```
str
```

The del() function removes that binding: If you call del(x), Python unassigns the value from the name x. After this, x no longer refers to anything, and trying to use it will raise a NameError.

It doesn't delete the value itself: The integer 10 still exists in memory if other variables reference it. What's removed is the association between the name x and that value.

```
print(type(2))

print(type(42.0))

print(type('Hello, World!'))

print(type('126'))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'str'>
```

```
type(2),type(.5),type('126')
```

```
(int, float, str)
```

So far we have seen 3 kinds of types.

integer: A type that represents numbers with no fractional or decimal part.

floating-point: A type that represents integers and numbers with decimal parts.

string: A type that represents sequences of characters.

All these are built-in types but a string is a collection data type.

And the type of an arithmetic exprerssion is ...

```
type(19 + n + abs(-5.1) * 2)
```

```
float
```

```
type(19 + n + abs(-5) * 2)
```

```
int
```

The types `int`, `float`, and `str` can be used as functions.

For example, `int` can take a floating-point number and convert it to an integer (always rounding down). And `float` can convert an integer to a floating-point value.

```
int(42.9), float(42)
```

```
(42, 42.0)
```

If you use an operator with a type it doesn't support, that's a runtime error.

```
#TypeError: values in the expression (operands) have the wrong type

'126' / 3
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[25], line 3
      1 #TypeError: values in the expression (operands) have the wrong type
----> 3 '126' / 3

TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

If you have a string that contains digits, you can use `int` or `float` to convert it to an integer.

```
int('126') / 3, float('12')
```

```
(42.0, 12.0)
```

```
int('2.5')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 int('2.5')

ValueError: invalid literal for int() with base 10: '2.5'
```

`int`, `float`, and `str` are not Python keywords. They are variables that represent types, and they can be used as functions. So it is *legal* to have a variable or function with one of those names, but it is strongly discouraged.

- Reduced code readability - makes it difficult to understand if a particular usage of int refers to the built-in type or a custom variable
- Potential for errors - if we wish to use the built-in functionality of int() in a part of the code where int also is a custom variable

When you write a large integer, you might be tempted to use commas between groups of digits, as in `1,000,000`. This is a legal expression in Python, but the result is not an integer.

```
1,000,000
```

```
(1, 0, 0)
```

Python interprets `1,000,000` as a comma-separated sequence of integers. We'll learn more about this kind of sequence later.

You can use underscores to make large numbers easier to read.

```
1_000_000
```

```
1000000
```

When you evaluate an expression, the result is displayed in interactive mode.

```
n + 1
```

```
18
```

But if you evaluate more than one expression, only the value of the last one is displayed in interactive mode.

```
n + 2
n + 3
```

To display more than one value, you can use the `print` function.

```
print(n+2)
print(n+3)
```

It also works with floating-point numbers and strings.

```
print('The value of pi is approximately')
print(pi)
```

```
The value of pi is approximately
3.141592653589793
```

You can also use a sequence of expressions separated by commas.

```
print('The value of pi is approximately', pi)
```

```
The value of pi is approximately 3.141592653589793
```

Notice that the `print` function puts a space between the values.

Boolean Expressions

A **boolean expression** is an expression that is either true or false.

`True` and `False` are special values that belong to the type `bool`; they are not strings:

```
type(True), type(False)
```

```
(bool, bool)
```

Relational operators

```
x = 5 #assignment operator -- not a relational operator
y = 7

x == y
#a realational operator
#compares two values and produces `True` if they are equal and `False` other

5 == 5
5 == 7

#other relational operators
x != y               # x is not equal to y
x > y                # x is greater than y
x < y                # x is less than to y
x >= y               # x is greater than or equal to y
x <= y               # x is less than or equal to y
```

```
True
```

```
5 != 6
```

```
True
```

Logical operators

To combine two boolean expressions into another boolean expression, we can use **logical operators**.

```
x > 0 and x < 10
# `True` if both of the conditions are true

x % 2 == 0 or x % 3 == 0
# `True` if *either or both* of the conditions is true

not x > y
#the `not` operator negates a boolean expression
```

```
x = 6
x % 2 == 0 or x % 3 == 0
```

```
True
```

Statements

A **statement** is a unit of code (one or more lines) representing a command/action that has an effect, but no value.

For example, an **assignment statement** creates a variable and gives it a value, but the statement itself has no value.

```
n = 17
```

```
type(n = 17) #runtime error
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[27], line 1
----> 1 type(n = 17) #runtime error

TypeError: type() takes 1 or 3 arguments
```

Computing the value of an expression is called **evaluation**. Running a statement is called **execution**.

Conditional Statements

Give the ability to check conditions and change the behavior of the program accordingly.

The `if` statement

```
if x > 0:
    print('x is positive')
```

```
        print('Cool')
```

```
x is positive
Cool
```

`if` is a Python keyword.

structure of `if` statement: a header followed by an indented sequence of statements called a **block**.

The boolean expression after `if` is called the **condition**. If it is true, the statements in the indented block run. If not, they don't.

There is no limit to the number of statements that can appear in the block, but there has to be at least one.

Occasionally, it is useful to have a block that does nothing. In that case, you can use the `pass` statement, which does nothing.

```
if x < 0:
    pass
```

The `else` clause

An `if` statement can have a second part, called an `else` clause. The syntax looks like this:

```
x=17
if x % 2 == 0:
    print('x is even')

else:
    print('x is odd')
```

If the condition is true, the first indented statement(s) runs; otherwise, the second indented statement(s) runs.

Since the condition must be true or false, exactly one of the alternatives will run. The alternatives are called **branches**.

Chained conditionals

Sometimes there are more than two possibilities and we need more than two branches. One way to express a computation like that is a **chained conditional**, which includes an `elif` clause.

```
if x < y:
    print('x is less than y')
elif x > y:
    print('x is greater than y')
else:
    print('x and y are equal')
```

`elif` is an abbreviation of "else if". There is no limit on the number of `elif` clauses. If there is an `else` clause, it has to be at the end, but there doesn't have to be one.

Each condition is checked in order. If the first is false, the next is checked, and so on. If one of them is true, the corresponding branch runs and the `if` statement ends. Even if more than one condition is true, only the first true branch runs.

Nested Conditionals

One conditional can also be nested within another.

```
x = 5
y = 5
if x == y:
    print('x and y are equal')
else:
    if x < y:
        print('x is less than y')
    else:
        print('x is greater than y')
```

The outer `if` statement contains two branches. The first branch contains a simple statement. The second branch contains another `if` statement, which has two branches of its own. Those two branches are both simple statements, although they could have been conditional statements as well.

Although the indentation of the statements makes the structure apparent, **nested conditionals** can be difficult to read.

```
if 0 < x:
    if x < 10:
        print('x is a positive single-digit number.')
```

The `print` statement runs only if we make it past both conditionals, so we get the same effect with the `and` operator.

```
if 0 < x and x < 10:
    print('x is a positive single-digit number.')
```

Logical operators often provide a way to simplify nested conditional statements.

For this kind of condition, Python provides a more concise option.

```
if 0 < x < 10:
    print('x is a positive single-digit number.')
```

```
x is a positive single-digit number.
```

Ternary Conditional Statement

```
n = float(input("Enter a number: "))
m = n if n >= 0 else -n
print(m)
```

Exercise: Read more about the usage

Match-Case Statement

```
n  = 17

match n%10:
    case 1:
        print("One")
    case 2 | 3:
        print("Two or Three")
    case 0:
        print("Zero")
    case _:
        print("Neither 1 nor 2 nor 3")
```

The match-case works similar to if-elif:

..Check first case

..If it matches ⟶ execute it

..Stop checking further cases

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Code Repetition
  - **Iteration** is the process of repeatedly executing a block of code
  - Loops - `for` and `while`
    - `break` and `continue` statements
  - Comparing `for` and `while`
    - `for` - used to iterate over any iterable object processing each item one by one
    - `while` - used when the number of iterations is unknown or infinite
      - useful for input validation along with `try`, `except`
- Data Structures are a way of organizing data so that it can be accessed/processed efficiently
  - **List** data structure - one of 4 built-in data types used to store collections of data
  - The other 3 being **tuple**, **set**, **dictionary**, all with different properties and usage

Reference: Think Python (Edition 3) by Allen B. Downey

Control flow

- A Python program is a sequence of statements
- Normal execution is sequential, top to bottom
- To perform interesting computations we need to control the flow

Conditionals -- take different paths based on the values computed so far

- Basic statement is if

For Loop

To **iterate**, we can use the `for` loop and the `range` function

```
for x in range(3):  #header ends with a colon
    print(x)        #body - indented
```

```
0
1
2
```

- The variable `x` defined in the `for` loop is called the **loop variable**
- The `range` function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number
  - range(i, j) generates the sequence $i, i+1, \ldots, j-1$
  - range(n) generates the sequence $0, 1, \ldots, n-1$ (implicitly starts with 0)
  - Optional third argument is the step size: range(i, j, k) is $i, i+k, \ldots, i+mk$ for the largest m such that $i+mk < j$ and $i+(m+1)k >= j$ (so that no term in the sequence exceeds j). Depending on whether it is increasing or decreasing, the last value will be less than j or greater than j.
- `in` is a membership operator
- The `for` statement executes the loop body for each value in `range`
  - When the `for` statement runs, it assigns the first value from `range` to `x` and then runs the body
  - When it gets to the end of the body, it loops back to the header
  - The second time through the loop, it assigns the next value from `range` to `x` and runs the body
  - This process is repeated for every value in `range`

- Then the loop ends

- To customize the sequence generated by `range`, we can manipulate the **arguments**

In a function call, the information enclosed in the parenthesis is called the **argument list**

Some functions can take **additional arguments** that are optional

```
for x in range(2, 11, 3):
  print(x)
```

```
2
5
8
```

If you call a function and provide too many or too few arguments, that results in a `TypeError`

```
range()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 range()

TypeError: range expected at least 1 argument, got 0
```

`TypeError` is raised when an operation or function is applied to an object of an incompatible data type

```
range(1,10,2,1)
```

Some functions can take any number of arguments, like `print`

```
for x in range(3):
    print(x, end=' ')
```

This version of `print` uses the keyword argument `end` so the `print` function puts a space after each number rather than a newline

While Loop

We use `for` loop to iterate when the number of iterations is known or finite

When the number of iterations is unknown and/or depends on a condition, we use `while` loop

With the while loop we can execute a set of statements as long as a condition is true

```
i = 1
while i < 7:
  print(i)
  i = i + 1
  if i % 4 == 0:
    i = 4
```

Note: remember to increment/update loop variable(s), otherwise the loop may continue forever

Break and Continue Statements

With the **break** statement we can stop the loop abruptly

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i = i + 1
```

```
for i in range(1, 6):
    print(i)
```

```
    if i == 3:
        break
```

With the **continue** statement we can stop the current iteration of the loop and proceed to the next

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

```
for i in range(1, 7):
    if i == 3:
        continue
    print(i)
```

For vs While

- `for` is typically used when the number of iterations is known in advance
- `while` is used when the number of iterations is unknown and depends on a dynamic condition
- `for` loop does not require an indexing variable to set beforehand
- `while` loop usually requires relevant variables like indexing variables to be ready before start of the loop

A `for` loop is in general used for iterating over an iterable object.

Here's an example that uses a `for` loop to display the letters in a string.

```
for letter in 'Python':
    print(letter, end=' ')
```

We can use often `while` to achieve the same effect but the code using `for` is more elegant

```
word = "Python"
i = 0
while i < len(word):
    print(word[i], end=' ')
    i += 1
```

Characters of a string can be accessed using indices. The first character has index [0], the second has index [1] and so on.

```
str='Python'
str[0]
```

If an index has a negative value, it counts backward from the end of the string.

```
str[-1]
```

Lists

- A **list** is an ordered collection of data
- Enables storing/accessing multiple items using a single variable
- Items in a list do not need to be of the same type
- Items are changeable and duplicate values are allowed

There are several ways to create a new list; the simplest is to enclose the elements in square brackets (`[` and `]`).

```
numbers = [42, 123]
```

```
numbers
```

```
[42, 123]
```

```
type(numbers)
```

```
list
```

```
cheeses = ['Cheddar', 'Feta', 'Mozzarella']
```

The elements of a list don't have to be the same type.

```
t = ['spam', 2.0, 5, [10, 20]]
```

A list within another list is **nested**.

A list that contains no elements is called an **empty list**

```
empty = []
```

The `len` function returns the length of a list. The length of an empty list is `0`.

```
len(cheeses)
```

```
3
```

List items are indexed, the first item has index [0], the second item has index [1] and so on.

```
cheeses[0]
```

```
'Cheddar'
```

Changing a list

```
numbers
```

```
[42, 123]
```

```
numbers[1] = 17
numbers
```

```
[42, 17]
```

```
numbers = [1,2,3]
```

```
numbers
```

```
[1, 2, 3]
```

If you try to read or write an element that does not exist, you get an `IndexError`

`IndexError` is raised when you try to access a sequence (such as a list or string) using an index that is outside the valid range of indices for that sequence

```
numbers[10]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[21], line 1
----> 1 numbers[10]

IndexError: list index out of range
```

Any integer expression can be used as an index.

```
i = 1
numbers[i] = 'Python'
numbers
```

```
[1, 'Python', 3]
```

We can add an element to the end of the list using `append`

```
numbers.append("k")
numbers
```

```
[1, 'Python', 3, 'k']
```

```
numbers[-2] = -1
```

```
numbers
```

```
[1, 'Python', -1, 'k']
```

If an index has a negative value, it counts backward from the end of the list.

The `in` operator is used to check whether a given element appears anywhere in the list.

```
'Feta' in cheeses
```

```
True
```

```
'Mozzarella' not in cheeses
```

```
False
```

Although a list can contain another list, the nested list still counts as a single element

```
t = ['spam', 2.0, 5, [10, 20]]
len(t[3])
```

```
2
```

And `10` is not considered to be an element of `t` because it is an element of a nested list, not `t`.

Searching Through a List

Search

Input: A list `L` of `n` integers and a target number `k`

Output: An index of `k` in `L`, if it exists

```
nums = []

n = int(input("Enter number of numbers in the list: "))

for i in range(n):
    value = int(input(f"Enter the {i+1}th number: "))
    nums.append(value)

# Formatted string
# To specify a string as an f-string, simply add an `f` in front of the string
# Allows you to format selected parts of a string
# Allows expressions inside `{}`

print(nums)

target = int(input("Enter target element: "))

index = 0
found = False

for value in nums:
    if value == target:
        print(f"Element found at index {index}")
        found = True
        break
    index = index + 1

if found == False:
    print("Element not found")
```

This procedure of sequentially scanning each element of the list until a match is found or the entire list has been searched is called **linear search**

How **efficient** is this procedure? Is it fast?

If the input list is sorted, **can you search faster**?

Nested Loops

A **nested loop** is a loop placed inside the body of another loop

The inner loop is executed for every iteration of the outer loop

```
#program to print all primes between 100 and 500 (inclusive)

primes = []
for num in range(100,501):
    is_prime = True
    for i in range(2,num):
        if num % i == 0:
            is_prime = False
            break
    if is_prime == True:
        primes.append(num)
print(primes)
```

Do you need to run the inner loop for each value in `{2,...,num-1}`? Is it correct to break from the loop if `p * p > num`?

```
#program to print the first n primes

n = int(input("Enter a positive integer: "))

primes = []
num = 2

while len(primes) < n:
    is_prime = True
    for p in primes:
        if num % p == 0:
            is_prime = False
            break
    if is_prime:
        primes.append(num)
    num = num + 1

print(primes)
```

Can the condition inside the inner loop be modified so that the program is more efficient?

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Code Reuse Part 1
  - Standard libraries
  - Third-party libraries

Libraries are collections of code implementing different groups of functions relevant to a given theme. For example, the `math` library has mathematical functions like sqrt , log , sin , cos, etc.

Python Libraries

The Python interpreter has a rich collection of functions and types built into it that are always available for use in programs (pre-installed **libraries**).

Modules and Packages

- A Python module is a single file of Python code (with a .py extension) that contains functionality (like functions, variables) which you can use in other Python programs
- A Python package is a collection of Python modules organized in a folder
  - Allows you to structure code into multiple modules and submodules in a hierarchical way
- A Python library is a collection of Python packages

Datatypes

- Each data is called an **object** and every object has a **type** (also called **class**)
  - Eg: `int` is a built-in type that is used to represent integer data
- An object is an **instance** of a type.
  - Eg: `15` is an instance of `int`

Built-in types: Numeric types (int, float, complex), Boolean type (bool), Sequence types (lists, tuples, range), ...

Built-in constants: False, True (bool type, assignments are illegal (raise a SyntaxError)), ...

Internally, these belong to a **module** named `builtins`

```
type(15)
```

```
int
```

```
print(type(True))
```

```
<class 'bool'>
```

```
print(type(7))
print(type(int))
```

```
<class 'int'>
<class 'type'>
```

```
s = [1,2,3]
print(type(s))
```

```
<class 'list'>
```

```
print(type(list))
```

```
<class 'type'>
```

Functions

- The **module** named `builtins` also has a large collection of built-in functions

- **Built-in functions:** abs(), round(), range(), len(), float(), int(), str(), bool(), print(), input(), ...

Python has other **modules** as part of its standard library. These modules come pre-installed with Python.

Some widely-used **standard modules** are

- math - provides mathematical functions used for advanced arithmetic operations
- random - provides functionality of various random operations
- tkinter - standard GUI library to create GUI elements
- datetime - allows for manipulation and reading of date and time values
- calendar - allows operations and manipulations related to calendars
- json - to encode and decode JSON (a format widely used on the web to exchange information) data
- os - to interact with the operating system, offers functionalities like interacting with file system, launching applications
- sys - provides functions and variables that interact with the Python runtime environment, offers functionalities such as input/output stream access, memory info access

Using Standard Modules

- To use a standard module, you must first import it into your program using the `import` statement
- An import statement has an effect -- it reads a module so we can use the variables and functions it contains -- but it has no visible effect

```
import math
```

`import` statement - to make code from modules available to your program

The `math` module provides a variable called `pi` that contains the value of the mathematical constant π.

```
math.pi
```

```
3.141592653589793
```

Double-click (or enter) to edit

The `math` module also contains functions

```
math.sqrt(25)
```

```
5.0
```

```
math.factorial(5)
```

```
120
```

```
math.pow(5, 2)
```

```
25.0
```

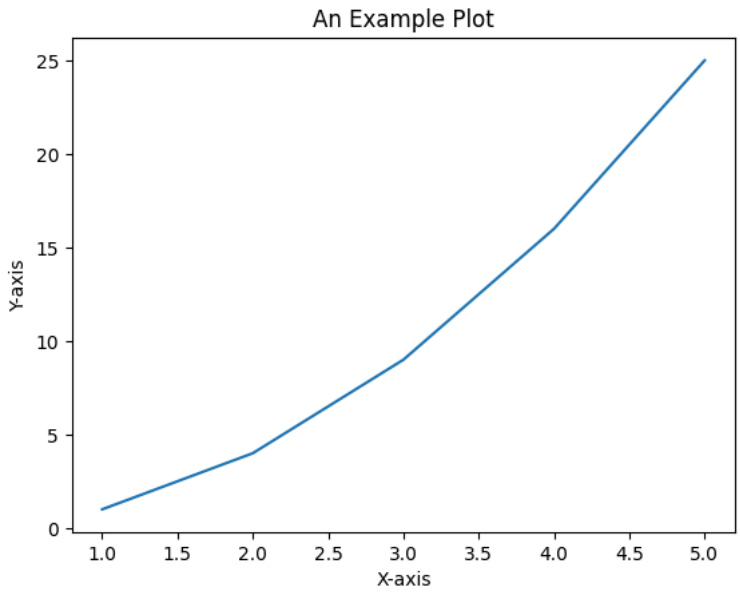
To use a variable or a function in a module, you have to use the **dot operator** (`.`) between the name of the module and the name of the variable/function.

Using Third-party Libraries

- Python also has a huge ecosystem of **third-party libraries**
  - To use them, we need to install and then import them
- Eg: Matplotlib is a plotting library for Python

```
%pip install matplotlib

# matplotlib is a third-party module not included in Python's standard library

# can be installed directly from a Jupyter cell
```

```
import matplotlib.pyplot as plt
# statement used to import the pyplot module from the matplotlib library with the alias plt

x = [1, 2, 3, 4, 5]      # x is a list of numbers
y = [1, 4, 9, 16, 25]    # y is a list of numbers

plt.plot(x, y)           # creates a plot from the given data
# syntax: plt.plot(x, y)
# x is a sequence of values to be plotted along the x-axis
# y is a sequence of values to be plotted along the y-axis

plt.xlabel('X-axis')
plt.ylabel('Y-axis')
# add labels to x-axis and y-axis to make the plot more understandable

plt.title('An Example Plot')   # adds a title to the plot to provide context

plt.show()                # displays the created plot
```



Some popular **third-party libraries** are

matplotlib – basic plotting and charts

numpy – numerical computing, arrays, linear algebra

pandas – data analysis and manipulation

scipy – scientific and technical computing

sympy – symbolic mathematics

scikit-learn – machine learning algorithms

qiskit – build and simulate quantum circuits

django – full-stack web framework

flask – lightweight web framework

Python also lets programmers define functions and types

- **User-defined functions** are custom blocks of code created for specific, reusable tasks
- **User-defined types (classes)** are for creating complex data structures

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Code Reuse Part 2
  - Defining new functions
  - Function calls
  - Arguments, parameters and local variables

Reference: Think Python (Edition 3) by Allen B. Downey

Defining a New Function

- Suppose you want to write a function `distance` that finds the distance between two points represented by the coordinates (x_0, y_0) and (x_1, y_1).
- How should a `distance` function look like -- what are the inputs and what is the output?

Mathematical Abstraction: Think of function `distance` as its mathematical analogue $dist$ -- a map from a domain to a codomain, i.e., $dist : \mathbb{R}^4 \rightarrow \mathbb{R}$

- The input (element of the domain) consists of 4 numbers -- together representing 2 points.
- By the Pythagorean theorem, the distance between (x_0, y_0) and (x_1, y_1) is $\sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2}$.
- The output (element of the codomain) is the distance.

```
def dist(x0, y0, x1, y1):
    return ((x1-x0)**2 + (y1-y0)**2)**0.5
```

A **function definition** specifies the name of a new function and the sequence of statements that run when the function is called.

Calling a Function

Now that we've defined a function, we can **call** it (in the main program or in another function) the same way we call built-in functions.

```
dist(1,1,4,5)
```

Mathematical Abstraction: Think of function compositions

A function must be defined before it is used (just like any other name)

Arguments, Parameters and Return Values

Argument

- A value provided to a function when the function is called.
- You can also use a variable as an argument.

Parameter

- A variable listed inside the parentheses in the function definition.
- A name used inside a function to refer to the value passed as an argument.

Return value

- The result of a function.
- If a function call is used as an expression, the return value is the value of the expression.
- A function may or may not return a value

When the function is called,

- New variables (specifed as parameters) are created and assigned the same objects as the arguments.
- The function runs, i.e., the statements in the function body is executed. The control is transferred to the code that called the function.
- When the function execution reaches a `return` statement, the function's execution stops executing and the result is sent back to the code that called the function.

```
d = dist(1,1,4,5)
```

```
print(d)
```

Revisiting Function Definitions

```
import math
def dist(x0, y0, x1, y1):
    dx = (x1-x0)
    dy = (y1-y0)
    dsq = dx**2 + dy**2
    return math.sqrt(dsq)
```

- `def` is a keyword that indicates that this is a function definition.
  - The first line of the function definition is called the **header** -- the rest is called the **body**.
  - The header has to end with a colon and the body has to be indented. By convention, indentation is always four spaces.
  - The body of a function can contain any number of statements of any kind.
- The name of the function is `dist`. Anything that's a legal variable name is also a legal function name.
- Variable names in parentheses of the function definition are called **parameters**.
  - In this example, this information indicates that the function takes 4 arguments.
  - An empty parentheses indicates that the function doesn't take any arguments.
- The `return` statement is used to exit the function and send data back to the code that called it.
  - Statements after the `return` statement will not be executed.
- Local variables
  - A variable defined inside a function that can only be accessed inside the function.
  - When you define a variable inside a function, it is **local**, which means that it only exists inside the function.
  - When `dist` runs, it creates local variables `dx`, `dy`, `dsq` which are destroyed when the function execution ends.
  - If we try to display these variables, we get a `NameError`.

```
# NameError

print(dsq)
```

Outside of the function, `dsq` is not defined.

Parameters are also local. For example, outside `dist`, there is no such thing as `x0`, `x1`, `y0` or `y1`.

```
print(x0)
```

Return Types

Functions may or may not send data back to the code that called them

```
import math
def dist(x1, x0, y0, y1):
    dx = (x1-x0)
    dy = (y1-y0)
    dsq = dx**2 + dy**2
    print(f"Distance between {(x0,y0)} and {(x1,y1)} is {math.sqrt(dsq)}")
```

If a function doesn't have a `return` statement, it returns `None` by default. Such a function is called a void function.

Boolean functions - those that return boolean values (`True`, `False`)

```
def is_prime(n):
    if n <= 1:
        return False

    for i in range(2,n):
        if n % i == 0:
            return False

    return True
```

Boolean function calls can be used in conditionals.

```
# Print all primes between 1 and n

def print_primes(n):
    for k in range(2, n + 1):
        if is_prime(k) == True:
            print(k)
```

Named Arguments and Default Arguments

When calling a function, it is a good idea to include the names of the parameters in the arguments list.

```
dist(1.5,2,4,7)
```

```
dist(x1=4,x0=1,y0=1,y1=5)
```

These are called as **named arguments** because they include the parameter names. They are also called as **keyword arguments**.

- Identifies the arguments by the parameter names.
- This allows you to skip arguments or place them out of order.

If parameter names are not included in the function call, then the arguments have to be in the same order as parameters defined in the function. These are called as **positional arguments**.

A **default argument** is an argument that assumes a default value if a value is not provided in the function call for that argument

```
import math
def dist(x1, y1, x0=0, y0=0):
    dsq = (x1-x0)**2 + (y1-y0)**2
    return math.sqrt(dsq)
```

```
dist(y1=5,x1=4)
```

```
dist(y1=5)
```

```
dist(y1=5,x1=4,x0=2,y0=2)
```

```
dist(y1=5,x1=4,y0=2)
```

```
dist(y1=5,x1=4,x0=2)
```

```
dist(x0=2,y0=2,y1=5,x1=4)
```

A non-default argument cannot follow a default argument

```
dist(y0=2,y1=5,x1=4)
```

```
dist(y0=2,y1=5,x1=4,1)
```

```
import math
def dist(x0=0, y0=0, x1, y1):
    dsq = (x1-x0)**2 + (y1-y0)**2
    return math.sqrt(dsq)
```

Debugging

When a runtime error occurs in a function, the error message includes a **traceback** that shows the name of the function that was running, the name of the function that called it, and so on.

```
def is_prime(n):
    if n <= 1:
        return False

    for i in range(2,n):
        if n % i == 0:
            return False
    return True
```

```
def print_primes(n):
    for k in range(2, n + 1):
        if is_prime(k):
            print(k)
```

```
print_primes(10)
```

Why define functions?

- Lets you to name a group of statements - makes the program easier to read and debug.
- Once you write a function, you can reuse it - helps eliminate repetitive code.
- To make a change in the function's behaviour, you only have to make it in the definition.

The design of a function has two parts:

- The **interface** is how the function is used, including its name, the parameters it takes and what the function is supposed to do.
- The **implementation** is how the function does what it's supposed to do.

The following two functions have the same interface (they take the same parameters and accomplish the same thing) but they have different implementations.

```
def is_prime(n):
    if n == 1:
        return False
    for i in range(2,n):
        if n % i == 0:
            return False
    return True
```

```
def is_prime(n):
    if n == 1:
        return False
    elif n == 2:
        return True
    elif n % 2 == 0:
        return False
    else:
        for i in range(3,n,2):
            if n % i == 0:
                return False
    return True
```

A **docstring** is a string at the beginning of a function that explains the interface ("doc" is short for "documentation")

By convention, docstrings are triple-quoted strings, also known as **multiline strings** because the triple quotes allow the string to span more than one line.

```
def is_prime(n):

    """Checks if the number n is prime or not
    n: positive integer
    """

    # implementation details
    if n <= 1:
        return False
    elif n == 2:
        return True
    elif n % 2 == 0:
        return False
    else:
        for i in range(3,n,2):
```

```
                if n % i == 0:
                    return False
        return True
```

A docstring should

- Explain concisely what the function does, without getting into the details of how it works.
- Explain what effect each parameter has on the behavior of the function.
- Indicate what type each parameter should be, if it is not obvious.

Writing this kind of documentation is an important part of interface design. A well-designed interface should be simple to explain.

An interface is like a contract between a function and a caller. The caller agrees to provide certain arguments and the function agrees to do accomplish some task.

For example, `is_prime` requires one argument that has to be a positive integer.

Such requirements are called **preconditions** because they are supposed to be true before the function starts executing.

The conditions at the end of the function are **postconditions**. Postconditions include the effect of the function (like printing if the number is prime or not).

Preconditions are the responsibility of the caller. If the caller violates a precondition and the function doesn't work correctly, the bug is in the caller, not the function.

If the preconditions are satisfied and the postconditions are not, the bug is in the function.

Typical Structure of a Python Program

First function definitions, then statements of the "main" program

```
def function1(...):
    stmt1
    stmt2
    ...


def function2(...):
    stmt1
    stmt2
    ...
```

```
    return

# Main program
Statement 1
Statement 2   #call function1 or function2 ...
...
Statement n
```

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Recursion and recursive functions

Reference: Think Python (Edition 3) by Allen B. Downey

Factorial

Consider the following function that returns the factorial of a natural number.

```
def factorial(n):
    fact = 1
    for i in range(2, n + 1):
        fact = fact * i
    return fact
```

```
print(factorial(7))
```

Why is this code correct?

Let $\mathbb{N}$ denote the set $\{0, 1, 2, \ldots\}$ of all natural numbers.

The factorial function is defined on $\mathbb{N}$ as:

- $0!$ is 1 and
- for each $n \in \mathbb{N} \setminus \{0\}$, $n! = 1 \cdot 2 \cdot \ldots \cdot n$.

Here's another way to defined the factorial function:

- $0! = 1$ and
- for each $n \in \mathbb{N} \setminus \{0\}$, $n! = n(n-1)!$.

Recursion

Recursion is a way of defining a mathematical object.

A **recursive definition** (or inductive definition) of a function $f : D \to C$ consists of two parts.

- (Base Case) The first part defines the value of f for few elements in D explicitly.
- (Recursive Case) The second part defines the value of f for other elements in D in terms of some of the already determined values of f.

A **recursive definition of a function** may be thought of **as** giving **a procedure** for computing the value of the function at a given element in the domain.

- Such a procedure is said to be a **recursive procedure**.

Recursion also refers to a technique where a procedure, in order to accomplish a task, calls itself with some part of the task.

In programming terminology, a **recursive function** is one that calls itself.

```
def factorial(n):
    if n == 0: #base case
        return 1
    else:
        return factorial(n-1) * n #recursive case (recursive call)
```

```
print(factorial(3))
```

The print function begins execution and calls `factorial(3)`.

```
factorial(3)
|-- factorial(2)
```

```
    |-- factorial(1)
        |-- factorial(0) = 1
```

The above representation of recursive calls is called a **recursion tree**.

The execution of `factorial` begins with `n=3` and since `3` is not `0`, the second branch is taken and `factorial` is called with argument `2`...

> The execution of `factorial` begins with `n=2` and since `2` is not `0`, the second branch is taken and `factorial` is called with argument `1`...
>
> > The execution of `factorial` begins with `n=1` and since `1` is not `0`, the second branch is taken and `factorial` is called with argument `0`...
> >
> > > The execution of `factorial` begins with `n=0` and since `0` is `0`, the first branch is taken and `1` is returned without making any more recursive calls.
> >
> > The `factorial` that got `n=1` as argument receives the value `1`, multiplies with `n` which is `1` and returns the result `1`.
>
> The `factorial` that got `n=2` as argument receives the value `1`, multiplies with `n` which is `2` and returns the result `2`.

The `factorial` that got `n=3` as argument receives the value `2`, multiplies with `n` which is `3` and returns the result `6`. This is the return value of the function call that started the whole process.

The `print` function now has its argument ready and prints `6`. The execution of `print` is completed.

Pretending to be a Python interpreter and executing the recursive calls, we have the following

`factorial(3)`

`= factorial(2) * 3`

`= factorial(1) * 2 * 3`

`= factorial(0) * 1 * 2 * 3`

`= 1 * 1 * 2 * 3`

Fibonacci Numbers

After `factorial`, the most common example of a recursive function is `fibonacci`, which has the following definition:

$$\begin{aligned}\text{fibonacci}(0) &= 0\\ \text{fibonacci}(1) &= 1\\ \text{fibonacci}(n) &= \text{fibonacci}(n-1) + \text{fibonacci}(n-2)\end{aligned}$$

Translated into Python, we have the following

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

```
fibonacci(5)
```

```
fibonacci(5)
|-- fibonacci(4)
|   |-- fibonacci(3)
|   |   |-- fibonacci(2)
|   |   |   |-- fibonacci(1) = 1
|   |   |   |-- fibonacci(0) = 0
|   |   |-- fibonacci(1) = 1
|   |-- fibonacci(2)
|       |-- fibonacci(1) = 1
```

```
|       |-- fibonacci(0) = 0
|-- fibonacci(3)
    |-- fibonacci(2)
    |   |-- fibonacci(1) = 1
    |   |-- fibonacci(0) = 0
    |-- fibonacci(1) = 1
```

As an aside, this way of computing Fibonacci numbers is very inefficient and there is a way to improve it.

Infinite recursion

If a recursion never reaches a base case, it goes on making recursive calls forever, and the program never terminates. This is known as **infinite recursion**, and it is generally not a good idea. Here's a minimal function with an infinite recursion.

```
def recurse():
    recurse()
```

To keep track of function calls, Python uses a **call stack**

- A call stack is a stack of frames
- A frame is a function call's workspace
- Each frame contains local variables, arguments passed, return address
- For every function call,
  - a new frame is created and placed on the stack
  - the function is executed
- When the function returns, the frame is taken off the stack and control goes back to the next frame on the stack

In Python, there is a limit to the number of frames that can be on the stack at a time. If a program exceeds the limit, it causes a runtime error.

```
# RecursionError

recurse()
```

If you encounter an infinite recursion, review the function to confirm that there is a base case that does not make a recursive call. And if there is a base case, check whether it is guaranteed to be reached.

Checking Types

What happens if we call `factorial` and give it `1.5` as an argument?

```
def factorial(n):
    if n == 0: #base case
        return 1
    else:
        return factorial(n-1) * n #recursive case (recursive call)
```

```
#RecursionError

factorial(1.5)
```

This is an infinite recursion. However, the function has a base case when `n == 0`. But if `n` is not an integer, we can *miss* the base case and recurse forever.

In this example, the initial value of `n` is `1.5`. In the first recursive call, the value of `n` is `0.5`. In the next, it is `-0.5`. From there, it gets smaller and smaller but it will never be `0`.

To avoid infinite recursion in such cases we can use the built-in function `isinstance` to check the type of the argument. Here's how we check whether a value is an integer.

```
isinstance(3, int)
```

```
isinstance(1.5, int)
```

Factorial with Error Checks

Now here's a version of `factorial` with error-checking.

```
def factorial(n):
    if isinstance(n, int) == False:
        print('The factorial function is defined only on non-negative integers.')
        return None
    elif n < 0:
        print('The factorial function is not defined for negative integers.')
        return None
    elif n == 0:
        return 1
    else:
        return factorial(n-1) * n
```

First it checks whether `n` is an integer. If not, it displays an error message and returns `None`.

```
factorial('hello')
```

```
The factorial function is defined only on non-negative integers.
```

```
factorial(1.5)
```

Then it checks whether `n` is negative. If so, it displays an error message and returns `None.`

```
factorial(-2)
```

If we get past both checks, we know that `n` is a non-negative integer, so we can be confident the recursion will terminate.

```
factorial(+5)
```

Checking the parameters of a function to make sure they have the correct types and values is called **input validation**.

```
factorial(15000)
```

Debugging

Breaking a large program into smaller functions creates natural checkpoints for debugging. If a function is not working, there are three possibilities to consider:

- There is something wrong with the arguments the function is getting -- that is, a precondition is violated.
- There is something wrong with the function -- that is, a postcondition is violated.
- The caller is doing something wrong with the return value.

To rule out the first possibility, you can add a `print` statement at the beginning of the function that displays the values of the parameters (and maybe their types). Or you can write code that checks the preconditions explicitly.

If the parameters look good, you can add a `print` statement before each `return` statement and display the return value. If possible, call the function with arguments that make it easy check the result.

If the function seems to be working, look at the function call to make sure the return value is being used correctly -- or used at all!

Adding `print` statements at the beginning and end of a function can help make the flow of execution more visible. For example, here is a version of `factorial` with print statements:

```
def factorial(n):
    space = ' ' * (4 * n)
    print(space, 'factorial', n)
    if n == 0:
        print(space, 'returning 1')
        return 1
    else:
        fact = factorial(n-1)
        result = n * fact
```

```
        print(space, 'returning', result)
        return result
```

`space` is a string of space characters that controls the indentation of the output. Here is the result of `factorial(3)`:

```
factorial(5)
```

```
                    factorial 5
                factorial 4
            factorial 3
        factorial 2
    factorial 1
factorial 0
returning 1
    returning 1
        returning 2
            returning 6
                returning 24
                    returning 120
120
```

If you are confused about the flow of execution, this kind of output can be helpful.

Exercise

The Ackermann function $A(m, n)$ is defined as

- $A(m, n) = n + 1$ if $m = 0$
- $A(m, n) = A(m - 1, 1)$ if $m > 0$ and $n = 0$
- $A(m, n) = A(m - 1, A(m, n - 1))$ if $m > 0$ and $n > 0$

Write a function named `ackermann` that evaluates the Ackermann function.

What happens if you call `ackermann(5, 5)`? If you get a `RecursionError`, try to figure out the reason. Hint: you may want to add a print statement to the beginning of the function to display the values of the parameters and then run the examples again.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Binary Search
- Exponentiation
- Tower of Hanoi Puzzle

Searching Through a List

Search

Input: A list L of n integers and a target number k

Output: An index of k in L, if it exists

Linear Search

```
n = int(input("Enter number of numbers in the list: "))
nums = [int(input(f"Enter the {i+1}th number: ")) for i in range(n)]

target = int(input("Enter target element: "))

index = 0
found = False

for value in nums:
    if value == target:
        print(f"Element found at index {index}")
        found = True
        break
    index = index + 1

if found == False:
    print("Element not found")
```

```
Enter number of numbers in the list:  3
Enter the 1th number:  1
Enter the 2th number:  2
Enter the 3th number:  3
Enter target element:  3
Element found at index 2
```

Note: In case input is a list of floating-point numbers, then `nums[mid] == target` may not return the correct answer.

```
if 0.1 + 0.2 == 0.3:
    print("equal")
else:
    print(0.1 + 0.2)
```

```
0.30000000000000004
```

Fix: Change the condition to `abs(value - target) < 1e-9`. This lets us treat `value` and `target` as equal if their difference is negligibly small. That is, if the two numbers differ by less than one-billionth, they are close enough to be considered equal.

Searching Through a Sorted List

Sorted Search

Input: A sorted list L of n integers and a target number k

Output: An index of k in L, if it exists

Binary Search (Iterative)

```
nums = []

n = int(input("Enter number of numbers in the sorted list: "))

for i in range(n):
    value = int(input(f"Enter the {i+1}th number: "))
    nums.append(value)

target = int(input("Enter target element: "))

left = 0
right = len(nums) - 1
found = False

while left <= right:
    mid = (left + right) // 2

    if nums[mid] == target:
        print(f"Element found at index {mid}")
        found = True
        break
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1

if found == False:
    print("Element not found")
```

Binary Search (Recursive)

```
def binary_search(nums, target, left, right):
    # Base case: search space is empty
    if left > right:
        return -1

    mid = (left + right) // 2

    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        return binary_search(nums, target, mid + 1, right)
    else:
        return binary_search(nums, target, left, mid - 1)
```

```
nums = []

n = int(input("Enter number of numbers in the sorted list: "))
for i in range(n):
    nums.append(int(input(f"Enter the {i+1}th number: ")))

target = int(input("Enter target element: "))

index = binary_search(nums, target, 0, len(nums) - 1)

if index != -1:
    print(f"Element found at index {index}")
else:
    print("Element not found")
```

Exponentiation by a Positive Integer

For $a \in \mathbb{R}$ and $n \in \mathbb{N}$, define

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ a \cdot a^{n-1} & \text{if } n > 0 \end{cases}$$

Repeated Product

```
def power1(a, n):
    if n == 0:
        return 1
    return a * power1(a, n-1)
```

```
power1(5,6)
```

Repeated Squaring

For $a \in \mathbb{R}$ and $n \in \mathbb{N}$, we have

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{\lfloor \frac{n}{2} \rfloor})^2 & \text{if } n \text{ is even} \\ a \cdot (a^{\lfloor \frac{n}{2} \rfloor})^2 & \text{if } n \text{ is odd} \end{cases}$$

```
def power2(a, n):
    if n == 0:
        return 1
    temp = power2(a, n // 2)
    return temp * temp if n % 2 == 0 else a * temp * temp
```

```
power2(5,6)
```

How do we handle negative powers?

```
def power3(a, n):
    if n < 0:
        return 1 / power3(a, -n)
    if n == 0:
        return 1
    temp = power3(a, n // 2)
    return temp * temp if n % 2 == 0 else a * temp * temp
```

Compare this with the following iterative version.

```
def power3_iter(a, n):
    if n < 0:
        a = 1 / a
        n = -n

    result = 1
    base = a

    while n > 0:
        if n % 2 == 1:
            result = result * base
        base = base * base
        n = n // 2

    return result
```

```
power3_iter(5,-6)
```

Towers of Hanoi

The puzzle begins with n disks stacked on one rod (source) in order of decreasing sizes. The objective is to move the entire stack to one of the other rods (destination) such that

- Only one disk is moved at a time.
- Each move consists of taking the disk at the top from one of the stacks and placing it on the top of another stack or on an empty rod.
- No disk can be placed on top of a disk that is smaller than it.

Image Source: Amazon

Consider the following approach to solving the problem of moving n disks from source to destination.

- At some point, we have to move the largest disk (disk n) from source.
- When this move is made, the source cannot have any other disk.
- Therefore, we have to solve the subproblem of moving $n-1$ disks from source before making this move.
- At some point, we also have move disk n to destination.
- When this move is made, the destination cannot have any other disk.
- Therefore, after disk n is moved to destination, we have to solve the subproblem of moving $n-1$ disks to destination.

Therefore, for $n > 1$, let us break the problem of moving n disks from source to destination into three steps.

- (Recursively) Move $n-1$ disks from source to auxiliary (using destination as temp).
- Move disk n from source to destination.
- (Recursively) Move $n-1$ disks from auxiliary to destination (using source as temp).

Correctness: By induction on n. This strategy takes $2^n - 1$ moves in total and we can show that this is the optimal strategy.

```
def towers(n, source, destination, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return
    towers(n - 1, source, auxiliary, destination)
    print(f"Move disk {n} from {source} to {destination}")
    towers(n - 1, auxiliary, destination, source)
```

```
towers(3,source='S',destination='D',auxiliary='A')
```

```
Move disk 1 from S to D
Move disk 2 from S to A
Move disk 1 from D to A
Move disk 3 from S to D
Move disk 1 from A to S
Move disk 2 from A to D
Move disk 1 from S to D
```

```
towers(3, S, D, A)
|
|-- towers(2, S, A, D)
|   |
|   |-- towers(1, S, D, A)
|   |      print: Move disk 1 from S to D
|   |
|   |   print: Move disk 2 from S to A
|   |
|   |-- towers(1, D, A, S)
|          print: Move disk 1 from D to A
|
|   print: Move disk 3 from S to D
|
|-- towers(2, A, D, S)
    |
    |-- towers(1, A, S, D)
    |      print: Move disk 1 from A to S
    |
    |   print: Move disk 2 from A to D
    |
    |-- towers(1, S, D, A)
           print: Move disk 1 from S to D
```

Recursion for arbitrary n

```
towers(n, S, D, A)
|
|-- towers(n-1, S, A, D)
|   |
|   |-- towers(n-2, S, D, A)
|   |   |
|   |   |-- ...
|   |   |
|   |   print: Move disk (n-1) from S to A
|   |   |
|   |-- towers(n-2, D, A, S)
|   |
|   print: Move disk n from S to D
|   |
|-- towers(n-1, A, D, S)
    |
    |-- towers(n-2, A, S, D)
    |   |
```

```
|   |-- ...
|   |
|   print: Move disk (n-1) from A to D
|
|-- towers(n-2, S, D, A)
```

Think about how the iterative analogue would look like.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Code Reuse Part 3
  - Defining new types
  - Data attributes and methods

References:

- https://docs.python.org/3/reference/datamodel.html
- Think Python (Edition 3) by Allen B. Downey

A Primer to Types

Python supports different kinds of data. Each data is called an **object** and every object has a **type**.

An object is an **instance** of a type. Eg: 1234 is an instance of `int`.

```
i = 1234
f = 3.14159
s = "Hello"
b = True
l = [1, 5, 7, 11, 13]
print(type(i),type(f),type(s),type(b),type(l))
```

```
<class 'int'> <class 'float'> <class 'str'> <class 'bool'> <class 'list'>
```

Let us define a datatype to represent a point in the 2D plane. Call the new type `Point`.

- How should a `Point` object (instance of `Point`) look like?
- A point in the plane is typically associated with two numbers representing its x and y coordinates.
- A `Point` object should ideally contain variables that can store the coordinates.

What are all the operations that we would want on `Point` objects?

A new type can be defined using the `class` keyword -- think of decsribing the datatype using a template.

A class definition specifies data and procedures associated with an object of this class.

- data attributes (or simply attributes) -- data that make up an object of this class
- procedural attributes (methods) -- specify how to interact with an object of this class

```
import math
class Point:
    """Represents a point in 2-D space."""

    # A Point object p has attributes p.x and p.y that are initialized to a and b resp.
    # self refers to the object being created
    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b

    # Special method that specifies how an object is shown as a string
    # Automatically invoked by the print function
    # self refers to the object that is being printed
    def __str__(self):
        return f'({self.x}, {self.y})'

    # Method to calculate the distance of a point from origin
    # self refers to the object on which this method is called
    def dist_to_origin(self):
        dx = self.x
        dy = self.y
        return round(math.sqrt(dx*dx + dy*dy),2)
    # the first parameter of a method is conventionally named self.
    # This parameter is a reference to the object of the class on which the method is called
```

Methods are functions defined within a class and can be called on objects of that class. Methods **act** on `Point` object(s).

- Methods define the behaviors and actions that an object of a class can perform.
- A method can take arguments, including a reference (address) to the object on which it is called.
  - This allows the method to access and modify the object's attributes.
- In the definition of `dist_to_origin()`, `self` is a reference to the object on which the method is being invoked.

Constructor

- `__init__` is a special **method** (called **constructor**) that is invoked automatically when you create a new object of the class.
- `__init__` takes the coordinates as parameters and assigns them to **attributes** `x` and `y` of the object **(instance variables)** being created.

We can customize how an object is shown when printed.

- `__str__` is a special method that specifies how an object is shown as a string.
- This method is automatically invoked by the `print` function.

The name `self` for the current object (first parameter in a method) is only a convention - we can use any other name.

Once the new type (class) is defined, independent instances (objects) of this class can be created.

- Each object has its own internal state -- the values of its local variables (attributes).
- All objects of a class share the same functions (methods) to query/update their state.
- Think of methods as functions being invoked on an object of this class.

Instantiation is the process of creating an object of a class.

```
z1 = Point()
z = Point(1,7)
print(z)
print(f"Distance between (0,0) and {z} is {z.dist_to_origin()}")
# When we call a method on an object, reference to this object gets automatically passed to the method
```

```
(1, 7)
Distance between (0,0) and (1, 7) is 7.07
```

A docstring is a string written using triple quotes (`' ' '` or `" " "`) that appears as the first statement inside a class.

Docstrings explain what the class is meant to represent. They are stored as metadata, accessible at runtime and used by help and documentation tools.

```
help(Point)
```

```
Help on class Point in module __main__:

class Point(builtins.object)
 |  Point(a=0, b=0)
 |
 |  Represents a point in 2-D space.
 |
 |  Methods defined here:
 |
 |  __init__(self, a=0, b=0)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  __str__(self)
 |      Return str(self).
 |
 |  dist_to_origin(self)
 |      # Method to calculate the distance of a point from origin
 |      # self refers to the object on which this method is called
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors defined here:
 |
 |  __dict__
 |      dictionary for instance variables
 |
 |  __weakref__
 |      list of weak references to the object
```

Identity, Type and Value of an Object

As mentioned earlier, objects are Python's abstraction for data. All data in a Python program is represented by objects or by relations between objects. Even code is represented by objects.

Every object has an **identity**, a **type** and a **value**.

An object's identity never changes once it has been created; you may think of it as the object's address in memory.

- The `id` function returns an integer representing its identity.
- The `is` operator compares the identity of two objects.

An object's type determines the operations that the object supports and also defines the possible values for objects of that type.

- The `type` function returns an object's type (which is an object itself).
- Like its identity, an object's type is also unchangeable.

The value of some objects can change.

- Objects whose value can change are said to be **mutable**
- Objects whose value is unchangeable once they are created are called **immutable**

An object's mutability is determined by its type; for instance, numbers, strings and tuples are immutable, while dictionaries and lists are mutable.

By default, instances of user-defined classes are mutable. However, it is possible to define custom classes so that the objects are immutable.

Here's a function that changes a `Point` object. The `id` function returns the *identity* of an object. This is an integer which is guaranteed to be unique and constant for this object during its lifetime. Two objects with non-overlapping lifetimes may have the same `id` value.

```
def reset(p):
    print("Id of Point p Before Reset:",id(p))
    p.x = 0
    p.y = 0
    print("Id of Point p After Reset:",id(p))
    return p
```

Note that we can pass an object as an argument in a function call.

```
w = Point(1,5)
print("Id of Point w Before Function Call:",id(w))
reset(w)
print("Id of Point w After Function Call:",id(w))
print(f'{w.x}, {w.y}')
```

```
Id of Point w Before Function Call: 1832786722016
Id of Point p Before Reset: 1832786722016
Id of Point p After Reset: 1832786722016
Id of Point w After Function Call: 1832786722016
0, 0
```

In contrast, consider the following snippet.

```
def increment(n):
    print("Id of n Before Increment:",id(n))
    n = n + 1
    print("Id of n After Increment:",id(n))
    return n
```

```
x = 17
print("Id of x Before Function Call:",id(x))
x = increment(x)
print("Id of x After Function Call:",id(x))
```

```
Id of x Before Function Call: 140709571593640
Id of n Before Increment: 140709571593640
Id of n After Increment: 140709571593672
Id of x After Function Call: 140709571593672
```

Integers are immutable. If we update `x` from 17 to 18, `x` points to the one value of 18 instead of the one value of 17.

```
x = 17
print("Id of x Before Function Call:",id(x))
increment(x)
print("Id of x After Function Call:",id(x))
```

```
Id of x Before Function Call: 4384597304
Id of n Before Increment: 4384597304
Id of n After Increment: 4384597336
Id of x After Function Call: 4384597304
```

Note that if two variables point to the same object, their `id` values are the same.

```
z = 10
print(id(z))
z = z + 1
print(id(z))
```

```
4384597080
4384597112
```

Types affect almost all aspects of object behavior. Even the importance of object identity is affected in some sense: for immutable types, operations that compute new values may actually return a reference to any existing object with the same type and value, while for mutable objects this is not allowed.

```
z1 = 10
z2 = 10
print(id(z1),id(z2))
```

```
4384597080 4384597080
```

`z1` and `z2` may or may not refer to the same object with the value 10, depending on the implementation. This is because `int` is an immutable type, so the reference to 10 can be reused. This behaviour depends on the implementation used, so should not be relied upon, but is something to be aware of when making use of object identity tests.

Unlike integers, lists are mutable.

```
zl1 = []
zl2 = []
print(id(zl1),id(zl2))
```

```
2524292582336 2524292582400
```

`zl1` and `zl1` are guaranteed to refer to two different, unique, newly created empty lists. Note that `zl1 = zl2 = []` assigns the same object to both `zl1` and `zl1`.

```
z1 = [10]
print(id(z1))
z1[0] = 100
print(id(z1))
```

```
2524292581952
2524292581952
```

```
Start coding or generate with AI.
```

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture: List - a built-in mutable collection data type

- A list is a sequence of values indexed by position.
- Values are ordered but not necessarily distinct.

References:

- https://docs.python.org/3/reference/datamodel.html
- Think Python (Edition 3) by Allen B. Downey

List

Sequences in Python represent finite ordered sets indexed by non-negative numbers. Sequences are distinguished according to their mutability.

A list is a mutable sequence of values indexed by position.

- The values in a list are called its **elements/members**
- Values are not necessarily distinct -- they are not necessarily of same type

The items of a list are arbitrary objects. Lists are formed by placing a comma-separated list of expressions in square brackets.

Creating a List

There are several ways to create a list (i.e., an instance of list type); the simplest is to enclose the elements separated by comma in square brackets.

Creating an empty list (a list that contains no elements)

```
empty = []
empty
```

```
[]
```

Creating a list containing integers

```
numbers = [45, 123, -1, 0, 4, 65]
```

```
print(type(numbers))
```

```
<class 'list'>
```

A list within another list is **nested**

```
l = [2.0, 5, [10, 20]]
```

Other ways to create a list

```
l1 = list()
l2 = list([1,2,3])
```

Acessing and Changing a List

Use index/position - to access an element of a list

```
numbers[1]
```

```
123
```

Indexing -- when the length of a sequence is n, the index set contains the numbers $0, \ldots, n-1$. Item i of sequence `s` is selected by `s[i]`. Some sequences, including lists, interpret negative subscripts by adding the sequence length.

- The expression in brackets is an **index** and it indicates which element in the sequence to select.
- An index is an offset from the beginning/end of the sequence.
- The offset of the first element is 0 from the beginning and -1 from the end.
- Valid indices for a list containing n elements
  - $0, \ldots, n-1$ (used to access elements in order)
  - $-1, \ldots, -n$ (used to access elements in reverse order)
- Index can be a variable or an expression (of type integer)
- The value of the index has to be an integer -- otherwise a `TypeError` is raised.
- If you try to read or write an element that does not exist, an `IndexError` is raised.

Accessing elements of a nested list -- use multiple subscripting

```
l = [2.0, 5, [10, 20]]
l[2][1]
```

```
20
```

Lists are **mutable** -- the elements of a list can be changed.

Changing a list element -- when the bracket operator appears on the left side of an assignment, it identifies the element of the list that will be assigned.

```
numbers = [45, 123, -1, 0, 4, 65]
numbers[1] = 17
numbers
```

```
[45, 17, -1, 0, 4, 65]
```

```
l[2][1] = -1
l
```

```
[2.0, 5, [10, -1]]
```

Slicing a List

Sequences support slicing -- a technique to extract a segment/portion of a sequence (called **slice**) by specifying a range of indices

The operator `[n:m]` returns the part of the list from the `n`th element to the `m`th element, including the former but excluding the latter.

```
numbers
```

```
[45, 17, -1, 0, 4, 65]
```

```
numbers[1:4]
```

```
[17, -1, 0]
```

If the first index is greater than or equal to the second, the result is an **empty list**.

```
numbers[1:1]
```

```
[]
```

If you omit the first index, the slice starts at the beginning.

```
numbers[:2]
```

```
[45, 17]
```

If you omit the second index, the slice goes to the end.

```
numbers[1:]
```

```
[17, -1, 0, 4, 65]
```

If you omit both, the slice is a copy of the whole list.

```
numbers[:]
```

```
[45, 17, -1, 0, 4, 65]
```

Operations on Lists

The `+` operator creates a new list from 2 lists by concatenation.

```
numbers = [1,2,3,4,5,6,7]
```

```
numbers + [4,5,6]
```

```
[1, 2, 3, 4, 5, 6, 7, 4, 5, 6]
```

```
numbers
```

```
[1, 2, 3, 4, 5, 6, 7]
```

The original list is unaffected unless there is a reassignment

```
numbers = numbers + [4,5,6]
```

```
numbers
```

```
[1, 2, 3, 4, 5, 6, 7, 4, 5, 6]
```

Adding an element to the end of the list

```
numbers = numbers + [-10]
```

```
numbers
```

```
[1, 2, 3, 4, 5, 6, 7, 4, 5, 6, -10]
```

Check types before attempting to concat

```
numbers + 9
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 numbers + 9

TypeError: can only concatenate list (not "int") to list
```

```
9 + numbers
```

```
[0] + numbers
```

The `*` operator repeats a list a given number of times.

```
[1] * 4
```

```
[1, 1, 1, 1]
```

Membership operator - to check whether a given element appears in the list

```
13 in numbers
```

```
False
```

```
13 not in numbers
```

```
True
```

List Methods

Python provides methods that operate on lists. For example, `append` adds a new element to the end of a list.

```
numbers
```

```
numbers.append(-10)
numbers
```

```
numbers.append(2+5)
numbers
```

`extend` takes a list as an argument and appends all of the elements of this list to the list on which it is called.

```
numbers.extend([100, 13])
numbers
```

```
[45, 17, -1, 0, 4, 65, -10, 100, 13]
```

Recall the difference between append and concat

```
numbers
```

```
numbers.append(7)
numbers
```

```
[45, 17, -1, 0, 4, 65, -10, 100, 13, 7]
```

```
numbers + [8]
numbers
```

```
[45, 17, -1, 0, 4, 65, -10, 100, 13, 7]
```

```
numbers = numbers + [8]
numbers
```

```
[45, 17, -1, 0, 4, 65, -10, 100, 13, 7, 8]
```

To insert an item at a given position, we can use the `insert` method.

```
numbers.insert(2,-105)
numbers
```

```
[1, 2, -105, -105, -105, 3, 4, 5, 6, 7, 4, 5, 6, -10]
```

If you know the index of the element you want to remove from the list, you can use `pop`.

```
numbers.pop(1)
```

```
17
```

The return value is the element that was removed. And we can confirm that the list has been modified.

```
numbers
```

```
[45, -1, 0, 4, 65, -10, 100, 13, 7, 8]
```

If you know the element you want to remove, you can use `remove` to delete the first occurrence.

```
numbers.remove(100)
```

The return value from `remove` is `None`. But we can confirm that the list has been modified.

```
numbers
```

```
[45, -1, 0, 4, 65, -10, 13, 7, 8]
```

If the element you ask to delete is not in the list, that's a `ValueError`.

```
numbers.remove(10)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[30], line 1
----> 1 numbers.remove(10)

ValueError: list.remove(x): x not in list
```

```
numbers.remove(5)
numbers
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[31], line 1
----> 1 numbers.remove(5)
      2 numbers

ValueError: list.remove(x): x not in list
```

To delete a segment of a list, you can reassign the corresponding slice to `[]`.

```
numbers[2:4] = []
numbers
```

```
[45, -1, 65, -10, 13, 7, 8]
```

To remove all items from the list, use the `clear` method.

```
numbers.clear()
```

To reverse a list, you can use the `reverse` method

```
numbers = [1,2,3,4,5,6]
numbers.reverse()
numbers
```

```
[6, 5, 4, 3, 2, 1]
```

To sort a list using `sort` method, the values must be of a uniform comparable type

```
numbers.sort()
numbers
```

```
[1, 2, 3, 4, 5, 6]
```

```
numbers.sort(reverse=True)
numbers
```

```
[6, 5, 4, 3, 2, 1]
```

To copy a list,

```
nums = numbers.copy()
nums
```

```
[6, 5, 4, 3, 2, 1]
```

Built-in Functions Useful for Lists

The function `sorted` sorts the elements of a list but the original list is unchanged.

```
scramble = ['c', 'a', 'b']
sorted(scramble)
```

```
['a', 'b', 'c']
```

```
scramble
```

```
['c', 'a', 'b']
```

```
s = sorted(scramble)
s
```

The function `reversed` reverse the elements of a list but the original list is unchanged.

```
scramble = ['c', 'a', 'b']
reversed(scramble)
scramble
```

```
['c', 'a', 'b']
```

The built-in function `len` returns the number of items of a sequence and can be used to return the length of a list.

```
len(numbers)
```

```
6
```

The function `sum` adds up the elements.

```
sum(numbers)
```

And `min` and `max` find the smallest and largest elements.

```
min(numbers)
```

```
max(numbers)
```

To copy a list we can use the `copy` method or one of the following.

```
numbers = [1,2,3,4,5]
```

```
new1 = numbers[:]
new1
```

```
[1, 2, 3, 4, 5]
```

Another way to copy a list is to use the `list` function.

```
new2 = list(numbers)
new2
```

```
[1, 2, 3, 4, 5]
```

In general `list` works provided its argument is a sequence

```
l = list(range(10))
l
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
new3 = numbers
new3
```

```
[1, 2, 3, 4, 5]
```

```
id(numbers), id(new1), id(new2), id(new3)
```

```
(1765529864000, 1765529843072, 1765529894272, 1765529864000)
```

An object with more than one reference has more than one name, so we say the object is **aliased**. If the aliased object is mutable, changes made with one name affect the other.

Although this behavior can be useful, it is error-prone. In general, it is safer to avoid aliasing when you are working with mutable objects.

Equality and Is

`x == y` checks if `x` and `y` have the same value

```
new1 = numbers[:]
new2 = list(numbers)
new3 = numbers
new4 = numbers + [10]
```

To check whether two variables refer to the same object, you can use the `is` operator.

```
new1 is numbers, new2 is numbers, new3 is numbers, new4 is numbers
```

```
(False, False, True, False)
```

Looping Through a List

You can use a `for` statement to loop through the elements of a list.

```
    for num in numbers:
        print(num)
```

A `for` loop over an empty list never runs the indented statements.

```
    for x in []:
        print('This is never printed.')
```

When looping through a list, the position index and corresponding value can be retrieved at the same time using the `enumerate` function.

```
    for x in enumerate(['tic', 'tac', 'toe']):
        print(x)
```

```
    (0, 'tic')
    (1, 'tac')
    (2, 'toe')
```

The `zip` function can be used to loop over two or more lists at the same time.

```
    questions = ['name', 'property1', 'property2']
    answers = ['python', 'simplicity', 'readable']
    for q in zip(questions, answers):
        print(q)
        print(type(q))
```

```
    ('name', 'python')
    <class 'tuple'>
    ('property1', 'simplicity')
    <class 'tuple'>
    ('property2', 'readable')
    <class 'tuple'>
```

Combining `enumerate` and `zip` allows you to iterate over multiple lists simultaneously while also accessing the index of the current items.

List Arguments

When you pass a list to a function, the function gets a reference to the list. If the function modifies the list, the caller sees the change.

For example, `pop_first` uses the list method `pop` to remove the first element from a list.

```
def pop_first(lst):
    return lst.pop(0)
```

```
numbers = [1,2,3]
pop_first(numbers)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[50], line 2
      1 numbers = [1,2,3]
----> 2 pop_first(numbers)

NameError: name 'pop_first' is not defined
```

The return value is the first element, which has been removed from the list -- as we can see by displaying the modified list.

```
numbers
```

```
[1, 2, 3]
```

In this example, the parameter `lst` and the variable `numbers` are aliases for the same object.

Passing a reference to an object as an argument to a function creates a form of **aliasing**. If the function modifies the object, those changes persist after the function is done.

```
def change_list_v1(l):
    l = l + l[1:] #creates a new list object
    return(l)

list1 = [1,3,5,7]
list2 = change_list_v1(list1)
print(list1)
print(list2)
```

```
[1, 3, 5, 7]
[1, 3, 5, 7, 3, 5, 7]
```

```
def change_list_v2(l):
    l.append(10) #modifies the same list
    return(l)

list1 = [1,3,5,7]
list2 = change_list_v2(list1)
print(list1)
print(list2)
```

```
[1, 3, 5, 7, 10]
[1, 3, 5, 7, 10]
```

```
def change_list_v3(lst):
    lst = lst + [1]
    lst.append(2)

l = [5,6]
change_list_v3(l)
print(l)
```

Since lists are mutable objects in Python, passing them into a function means the function can directly alter the original list

```
[5, 6]
```

```
def change_list_v4(lst):
    lst += [1]
    lst.append(2)

l = [5,6]
change_list_v4(l)
print(l)
```

```
[5, 6, 1, 2]
```

```
+=  ---->modifies list
+   ----->creates new list
```

The behavior of the augmented assignment operator `+=` depends on whether the object's type is mutable or immutable.

List Comprehension

A list comprehension is a language construct inspired by the set-builder notation in mathematics. List comprehensions provide a concise way to create lists. For example, suppose we wish to create a list of squares in a range.

```
squares = []
for x in range(10):
    squares.append(x**2)
squares
```

A more concise and readable way to accomplish the same is as follows.

```
squares = [x**2 for x in range(10)]
squares
```

A **list comprehension** consists of brackets containing an expression followed by a `for` clause, then zero or more `for` or `if` clauses. The result will be a new list resulting from evaluating the expression in the context of the `for` and `if` clauses which follow it.

Nested list comprehensions: The initial expression in a list comprehension can be any arbitrary expression, including another list comprehension.

```
matrix = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
]
matrix
```

The following list comprehension will transpose rows and columns.

```
[[row[i] for row in matrix] for i in range(4)]
```

List comprehensions are concise and easy to read, at least for simple expressions. And they are usually faster than the equivalent for loops, sometimes much faster. However, list comprehensions may be harder to debug.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture: Dictionary - a built-in mutable collection data type

- Dictionary is a collection of (key, value) pairs indexed by key.
- Keys are distinct but values are not necessarily distinct.

Reference: Think Python (Edition 3) by Allen B. Downey

From Lists to Dictionaries

- A list is a sequence of values indexed by position.
- A list can be thought of as a function $f : \{0, 1, \ldots, n-1\} \to \{v_0, \ldots, v_{n-1}\}$ for some $n \in \mathbb{N}$ and values $v_0, \ldots, v_{n-1}$.
- A list maps positions to values.
- Generalize this to a function $f : \{k_0, k_1, \ldots, k_{n-1}\} \to \{v_0, \ldots, v_{n-1}\}$ for some $n \in \mathbb{N}$, values $v_0, \ldots, v_{n-1}$ and keys $k_0, \ldots, k_{n-1}$.
- Instead of positions, index an element by a *key*.
- A dictionary maps keys to values.
- A dictionary is a collection of (key, value) pairs (also called items) indexed by key.
    - Each value is called an element or member of the dictionary.
- A dictionary can be thought of as a lookup table.

Create an empty dictionary using `{}`

```
empty = {}
empty
```

```
{}
```

```
print(type(empty))
```

```
<class 'dict'>
```

Alternatively, use the constructor

```
empty = dict()
empty
```

```
{}
```

Start with an empty dictionary and add elements one by one

```
numbers = {}
numbers['zero'] = 0
numbers['one'] = 1
numbers['two'] = 2
numbers
```

```
{'zero': 0, 'one': 1, 'two': 2}
```

Create a dictionary by adding multiple elements

```
dnew = {"A":"1", "B":"10"}
dnew
```

```
{'A': '1', 'B': '10'}
```

Build a dictionary directly from a sequence of (key, value) pairs using the constructor

```
d1 = dict([('abc', 4139), ('x', 4127), ('z', 4098)])
d1
```

```
{'abc': 4139, 'x': 4127, 'z': 4098}
```

When the keys are simple strings, it is sometimes easier to specify pairs using named arguments

```
d2 = dict(abc=4139, x=4127, z=4098)
d2
```

```
{'abc': 4139, 'x': 4127, 'z': 4098}
```

Dict comprehensions can be used to create dictionaries from key and value expressions. A dict comprehension, in contrast to list comprehensions, needs two expressions separated with a colon followed by the usual `for` and `if` clauses. When the comprehension is run, the resulting key and value elements are inserted in the new dictionary in the order they are produced.

```
d3 = {x: x**2 for x in range(1,5)}
d3
```

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

Access an element using key

```
bdays = {"Harry":"03-05-2001", "Hermione":"17-10-1999"}
bdays["Hermione"]
```

```
'17-10-1999'
```

Accessing a non-existent key results in `KeyError`

```
bdays["Ron"]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[3], line 1
----> 1 bdays["Ron"]

KeyError: 'Ron'
```

The main operations on a dictionary are storing a value with some key and extracting the value given the key. If you store using a key that is already in use, the old value associated with that key is forgotten.

Assignment to non-existent key creates a new (key, value) pair

```
bdays["Ron"] = "13-08-2000"
bdays
```

```
{'Harry': '03-05-2001', 'Hermione': '17-10-1999', 'Ron': '13-08-2000'}
```

Update a value using its key

```
bdays["Ron"] = "13-09-2000"
bdays
```

```
{'Harry': '03-05-2001', 'Hermione': '17-10-1999', 'Ron': '13-09-2000'}
```

Copying a dictionary

```
bdays_copy = dict(bdays)
bdays_copy
```

```
{'Harry': '03-05-2001', 'Hermione': '17-10-1999', 'Ron': '13-09-2000'}
```

Number of elements in a dictionary

```
len(bdays)
```

```
3
```

The `in` operator works on dictionaries -- it tells you whether something appears as a key in the dictionary.

```
numbers, 'one' in numbers
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[8], line 1
----> 1 numbers, 'one' in numbers

NameError: name 'numbers' is not defined
```

The `in` operator does not check whether something appears as a value.

```
1 in numbers
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[9], line 1
----> 1 1 in numbers

NameError: name 'numbers' is not defined
```

To see whether something appears as a value in a dictionary, you can use the method `values`, which returns a sequence of values, and then use the `in` operator.

```
1 in numbers.values()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 1 in numbers.values()

NameError: name 'numbers' is not defined
```

`keys` and `values` methods generate sequences corresponding to the keys and values of a dictionary, respectively

```
bdays.keys()
```

```
dict_keys(['Harry', 'Hermione', 'Ron'])
```

```
bdays.values()
```

```
dict_values(['03-05-2001', '17-10-1999', '13-09-2000'])
```

Now we can iterate through all entries in a dictionary

```
for name in bdays.keys():
  print(name)
```

```
Harry
Hermione
Ron
```

```
namelist = list(bdays.keys())
namelist
```

```
['Harry', 'Hermione', 'Ron']
```

```
list(bdays.values())
```

```
['03-05-2001', '17-10-1999', '13-09-2000']
```

Listing Order of Keys

- In theory, this order is arbitrary and you should not make any assumptions
- In practice, in Python, keys are listed in the order added
  - Replacing an existing key does not change the order
  - Removing a key and re-inserting it will add it to the end instead of keeping its old place

When looping through dictionaries, the key and corresponding value can be retrieved at the same time using the `items` method

```
knights = {'Richard': 'Lionheart', 'Robin': 'Hero'}
for k in knights.items():
    print(k)
```

```
('Richard', 'Lionheart')
('Robin', 'Hero')
```

We can used `sorted` function while looping -- just as in lists

```
d = {}
d['a'] = 7
d['c'] = 9
d['b'] = 8
for name in sorted(d.keys()):
  print(d[name])
```

```
7
8
9
```

Delete a key

```
print(d)
str = d.pop('a')
print(d)
print(str)
```

```
{'a': 7, 'c': 9, 'b': 8}
{'c': 9, 'b': 8}
7
```

Removes all items from a dictionary, leaving it empty

```
d.clear()
d
```

```
{}
```

Deleting the entire dictionary object or a (key, value) pair

```
del d
```

The `del` statement can in general be used to delete references to objects. This can include variables, list items, dictionary keys, or entire objects.

```
d
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 d

NameError: name 'd' is not defined
```

Dictionaries are mutable.

```
def modify_dict(d):
    d["x"] = 999
```

```
data = {"a": 1, "b": 2}
modify_dict(data)
print("Final dictionary:", data)
```

```
Final dictionary: {'a': 1, 'b': 2, 'x': 999}
```

List and Dictionaries: Implementation Details

List (represents a finite set of objects indexed by position)

- Initially a contiguous block of memory is allocated to the list
- Flexible size
- When storage runs out, the allocation is doubled
- **Random access** -- accessing the value at `l[i]` does not depend on `i`
- Slices vs sublists
- Inserts and deletes may be expensive
- Lookup time (searching for a value) may depend on the length of the list

Dictionary (represents a finite set of objects indexed by nearly arbitrary values)

- Items stored using **hash table** data structure that guarantees that average lookup time is a constant independent of the number of items
- Items are stored in a table T of size t which is proportional to the number of items
- Keys are mapped to location in T by a **hash function** $h : keys \rightarrow \{0, 1, \ldots, t-1\}$
- For a key k, its hash value $h(k)$ determines the location of the corresponding item in T
- **Collisions** may occur and there are different ways to handle collision like chaining and probing
- Given a key k, lookup requires computing $h(k)$ (takes roughly the same time for any k) and searching in T at the corresponding location(s)
- Delete involves a lookup followed by additional bookkeeping
- The hash table size may grow (by doubling) or shrink (by halving) if efficiency (measured by average lookup/insert/delete time) drops and in that case items

may have to be rehashed

- With a hash table of size t, it is necessary that a key's hash value remains the same across inserts, deletes and lookups into this table
    - In this context, we say that the keys have to be hashable
    - Otherwise, checking the hash value of a key might make us look in the wrong location and thus never find the associated value

If a key is immutable, then its hash value is always the same

- Immutable types like integers, floats and strings are hashable

If a key is mutable, then its hash value could change and the hash table may not work

- Mutable types like lists and dictionaries are not hashable

However, there is no requirement that all keys (or values) have a uniform type.

```
bdays[0] = 7
bdays
```

```
{'Harry': '03-05-2001', 'Hermione': '17-10-1999', 'Ron': '13-09-2000', 0: 7}
```

A list cannot be used as a key

```
bdays[[1]] = 77
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[25], line 1
----> 1 bdays[[1]] = 77

TypeError: unhashable type: 'list'
```

The value at a key can be a list

```
bdays[1] = [77,88]
bdays
```

```
{'Harry': '03-05-2001',
 'Hermione': '17-10-1999',
 'Ron': '13-09-2000',
 0: 7,
 1: [77, 88]}
```

We can use multiple subscripting to access inner components

```
bdays[1][1]
```

```
88
```

Likewise, a value can be a nested dictionary

```
brothers = {}
brothers["Ron"] = {"first":"Fred", "second":"George"}
brothers
```

```
{'Ron': {'first': 'Fred', 'second': 'George'}}
```

```
brothers["Ron"]["second"]
```

```
'George'
```

Computing Fibonacci Numbers

If you execute the `fibonacci` function from our earlier lecture, you would notice that the bigger the argument you provide, the longer the function takes to execute.

```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n-1) + fibonacci(n-2)
```

```
%time fibonacci(40)
```

```
CPU times: user 11.3 s, sys: 75.4 ms, total: 11.4 s
Wall time: 11.5 s
102334155
```

Observe that we make redundant recursive calls. Let us avoid this by keeping track of values that have already been computed by storing them in a list or dictionary.

```
knownFib = [0, 1]

def fibonaccilist(n):
    if n < len(knownFib):
        return knownFib[n]
    knownFib.append(fibonaccilist(n-1) + fibonaccilist(n-2))
    return knownFib[n]
```

```
%time fibonaccilist(40)
```

```
CPU times: user 24 µs, sys: 0 ns, total: 24 µs
Wall time: 26.9 µs
102334155
```

```
knownFib = {0:0, 1:1}

def fibonaccidict(n):
    if n in knownFib:
        return knownFib[n]
    knownFib[n] = fibonaccidict(n-1) + fibonaccidict(n-2)
    return knownFib[n]
```

```
%time fibonaccidict(40)
```

```
CPU times: user 23 µs, sys: 1 µs, total: 24 µs
Wall time: 26.2 µs
102334155
```

Memoization is an optimization technique used to speed up programs. It works by storing the results of expensive calls to functions so that these results can be returned quickly should the same inputs occur again.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture: Built-In Immutable Collection Data Types in Python

1. String is a sequence of characters indexed by position.

   Characters are ordered, not necessarily distinct and cannot be changed.

2. Tuple is a sequence of values indexed by position.

   Members are ordered, not necessarily distinct and cannot be changed.

Reference: Think Python (Edition 3) by Allen B. Downey

Strings

String - specialized, built-in data type for representing text.

- A string is a sequence of values that represent characters.
- A character can be a letter (in almost any alphabet), a digit, a punctuation mark, or white space.

```
s = 'python' #single quotes
print(type(s))
```

```
<class 'str'>
```

```
s1 = "hello" #double quotes
```

```
#triple quotes
s2 = """this is a
multi-line string
in python"""

print(s2)
```

```
this is a
multi-line string
in python
```

Empty string

```
t = ''
```

```
t1 = str()
```

Using constructor,

```
r1 = str('hello')
```

There is no separate single character type; a single character is a string of length 1

```
c = 'k'
```

Escape characters

```
st1 = "hi's"
st2 = 'hi\'s'
st3 = """I said "Hello" """
st4 = "The path is C:\\Users\\Admin\\Documents"
st5 = "Name:\tSem:\tDept:"
st6 = "XXX\t02\tCSE\nK"
```

```
print(st4)
print(st5)
print(st6)
```

Triple quotes let you write a string across multiple lines without using newline characters (`\n`).

We can use the built-in function `len` to get the length of a string.

```
s = 'python'
n = len(s)
n
```

Indexing and Slicing

Index - to access a character of a string. Works like indices in lists.

```
fruit = 'mango'
letter = fruit[1]
letter
```

The value of the index has to be an integer -- otherwise you get a `TypeError`.

```
fruit[1.5]
```

Accessing invalid index,

```
fruit[10]
```

```
fruit[-1]
```

String slices (substrings) are like list slices

```
fruit = 'mango'
fruit[1:3]
```

Concatenation operator `+` is used to concatenate two strings

```
fruit1 = 'mango'
fruit2 = 'apple'
fruit1 + fruit2
```

And the `*` operator duplicates a string a given number of times

```
fruit1 * 4
```

The `in` operator is used to check whether a character appears in a string.

```
word = 'apple'
'ae' in word
```

```
False
```

Strings are Immutable

It is tempting to use the `[]` operator on the left side of an assignment, with the intention of changing a character in a string

```
greeting = 'Hello, world!'
```

```
greeting[0] = 'J'
```

The result is a `TypeError`. In the error message, the "object" is the string and the "item" is the character we tried to assign.

The reason for this error is that strings are immutable - you can't change an existing string. The best you can do is create a new string that is a variation of the original.

```
newgreeting = 'J' + greeting[1:]
newgreeting
```

This example concatenates a new first letter onto a slice of `greeting`. It has no effect on the original string.

```
greeting
```

String Comparison

The relational operators work on strings. To see if two strings are equal, we can use the `==` operator.

```
word = 'mango'

if word == 'mango':
    print("here's a mango.")
```

Other relational operations are useful for putting words in alphabetical order

```
def compare_word(word):
    if word < 'mango':
        print(word, 'comes before mango.')
    elif word > 'banana':
        print(word, 'comes after mango.')
    else:
        print('here\'s a mango.')
```

```
compare_word('apple')
```

In Python, all the uppercase letters come before all the lowercase letters.

```
compare_word('Pineapple')
```

If necessary, we can convert strings to a standard format, such as all lowercase, before performing the comparison.

String Methods and Functions

Strings provide methods that perform a variety of useful operations. For example, the method `upper` takes a string and returns a new string with all uppercase letters.

```
word = 'mango'
new_word = word.upper()
new_word
```

Similarly, we can use the `lower` method.

```
word = 'Mango'
new_word = word.lower()
new_word
```

To break a string into words, you can use the `split` method

```
s = 'To be, or not to be'
t = s.split()
t
```

```
['To', 'be,', 'or', 'not', 'to', 'be']
```

An optional argument called a **delimiter** specifies which characters to use as word boundaries. The following example uses a hyphen as a delimiter.

```
s = 'know-it-all'
t = s.split('-')
```

```
tuple(t)
```

```
('know', 'it', 'all')
```

After using `split` to make a list of words, we can use `for` to loop through them.

```
s = 'To be, or not to be'

for word in s.split():
    print(word)
```

```
To
be,
or
not
to
be
```

If you have a list of strings, you can concatenate them into a single string using the `join` method -- you have to invoke it on the delimiter and pass the list as an argument.

```
delimiter = ' '
t = ['a', 'rose', 'is', 'a', 'rose', 'is', 'a', 'rose']
s = delimiter.join(t)
s
```

```
'a rose is a rose is a rose'
```

In this case the delimiter is a space character, so `join` puts a space between words. To join strings without spaces, you can use the empty string, `''`, as a delimiter.

To convert from a string to a list of characters,

```
s = 'spam'
t = list(s)
t
```

```
['s', 'p', 'a', 'm']
```

`sorted` works with any kind of sequence, not just lists.

```
sorted('letters')
```

```
['e', 'e', 'l', 'r', 's', 't', 't']
```

The result is a list. To convert the list to a string, we can use `join`.

```
''.join(sorted('letters'))
```

```
'eelrstt'
```

The `reversed` function also works with strings.

```
s = "letters"
reversed(s)
```

```
<reversed at 0x19bda636230>
```

```
s = "letters"
rs = "".join(reversed(s))
print(rs)
```

```
srettel
```

From Strings to Tuples

- String is a collection for representing text. Ordered and unchangeable. Allows duplicate members.
- Tuple is a collection which is ordered and unchangeable. Allows duplicate members.

To create a tuple, you can write a comma-separated list of values.

```
t1 = 's', 'h', 'e', 'r', 'l', 'o', 'k' #comma-separated sequence of values
t2 = ('h', 'o', 'l', 'm', 'e', 's') #can enclose comma-separated sequence in parentheses
t3 = 'z', #to create a tuple with a single element, include a comma -- to avoid confusing with strings
t4 = tuple() #to create an empty tuple
t5 = tuple('sherlock holmes')
t6 = 1, 2, 3, 456
```

```
print(type(t5))
print(t3)
```

```
<class 'tuple'>
('z',)
```

```
print(type(t6))
print(t6)
```

```
<class 'tuple'>
(1, 2, 3, 456)
```

What does the following code create?

```
t6 = ('p')
type(t6)
print(t6)
```

```
p
```

```
t7 = ('p',)
type(t7)
print(t7)
```

```
('p',)
```

If the argument to the constructor is a sequence (string, list or tuple), the result is a tuple with the elements of the sequence.

```
t = tuple('sirius')
t
```

```
('s', 'i', 'r', 'i', 'u', 's')
```

The bracket operator indexes an element of a tuple.

```
t = tuple('sherlock holmes')
t[0]
```

```
's'
```

And the slice operator selects a range of elements.

```
t[1:3]
```

```
('h', 'e')
```

```
t1 = (1, 2)
print(id(t1))   # memory address of t1

t1 += (3,)
print(t1)       # (1, 2, 3)
print(id(t1))   # different memory address → new tuple created
```

The `+` operator concatenates tuples.

```
tuple('sher') + ('l', 'o')
```

```
('s', 'h', 'e', 'r', 'l', 'o')
```

And the `*` operator duplicates a tuple a given number of times.

```
tuple('sheer') * 2
```

```
('s', 'h', 'e', 'e', 'r', 's', 'h', 'e', 'e', 'r')
```

The `sorted` function works with tuples -- but the result is a list and not a tuple.

```
sorted(t)
```

```
[' ', 'c', 'e', 'e', 'h', 'h', 'k', 'l', 'l', 'm', 'o', 'o', 'r', 's', 's']
```

The `reversed` function also works with tuples.

```
reversed(t)
```

```
<reversed at 0x19bd8e938b0>
```

The result is a `reversed` object, which we can convert to a list or tuple.

```
tuple(reversed(t))
```

```
('s', 'e', 'm', 'l', 'o', 'h', ' ', 'k', 'c', 'o', 'l', 'r', 'e', 'h', 's')
```

Based on the examples so far, it might seem like tuples are the same as lists.

Tuples are Immutable

If you try to modify a tuple with the bracket operator, you get a `TypeError`.

```
t[0] = 'L'
```

And tuples don't have any of the methods that modify lists, like `append` and `remove`.

As tuples are immutable, they can be used as keys in a dictionary. For example, the following dictionary contains two tuples as keys that map to integers.

```
d = {}
d[1, 2] = 3
d[3, 4] = 7
```

We can look up a tuple in a dictionary like this:

```
d[1, 2]
print(d)
```

```
{(1, 2): 3, (3, 4): 7}
```

Or if we have a variable that refers to a tuple, we can use it as a key.

```
t = (3, 4)
d[t]
```

```
7
```

Tuples can also appear as values in a dictionary.

```
t = tuple('abc')
d = {'key': t}
d
```

```
{'key': ('a', 'b', 'c')}
```

Tuple Assignment

You can put a tuple of variables on the left side of an assignment, and a tuple of values on the right.

```
a, b = 1, 2
```

The values are assigned to the variables from left to right -- in this example, `a` gets the value `1` and `b` gets the value `2`.

```
a, b
```

```
(1, 2)
```

Another example

```
point = (10, 20)
x, y = point
print("x-coord:", x, "y-coord:", y)
```

```
x-coord: 10 y-coord: 20
```

More generally, if the left side of an assignment is a tuple, the right side can be any kind of sequence -- string, list or tuple.

```
email = 'monty@python.org'
username, domain = email.split('@')
```

The return value from this split is a list with two elements -- the first element is assigned to `username`, the second to `domain`.

```
username, domain
```

```
('monty', 'python.org')
```

The number of variables on the left and the number of values on the right have to be the same -- otherwise you get a `ValueError`.

```
a, b = 1, 2, 3
```

Tuple assignment is useful if you want to swap the values of two variables. With conventional assignments, you have to use a temporary variable.

```
temp = a
a = b
b = temp
```

That works, but with tuple assignment we can do the same thing without a temporary variable.

```
a, b = b, a
```

This works because all of the expressions on the right side are evaluated before any of the assignments.

Tuples as Return Values

Strictly speaking, a function can only return one value, but if the value is a tuple, the effect is the same as returning multiple values.

The built-in function `divmod` takes two arguments and returns a tuple of two values, the quotient and remainder.

```
divmod(7, 3)
```

```
(2, 1)
```

We can use tuple assignment to store the elements of the tuple in two variables.

```
quotient, remainder = divmod(7, 3)
quotient, remainder
```

```
(2, 1)
```

Here is an example of a function that returns a tuple.

```
def min_max(t):
    return min(t), max(t)
```

`max` and `min` are built-in functions that find the largest and smallest elements of a sequence. `min_max` computes both and returns a tuple of two values.

```
min_max([2, 4, 1, 3])
```

```
(1, 4)
```

We can assign the results to variables like this

```
low, high = min_max([2, 4, 1, 3])
low, high
```

```
(1, 4)
```

We can also use tuple assignment in a `for` statement.

```
d = {'one': 1, 'two': 2}

for item in d.items():
    key, value = item
    print(key, '->', value)
```

```
one -> 1
two -> 2
```

Each time through the loop, `item` is assigned a tuple that contains a key and the corresponding value.

```
for key, value in d.items():
    print(key, '->', value)
```

```
one -> 1
two -> 2
```

Each time through the loop, a key and the corresponding value are assigned directly to `key` and `value`.

Revisiting Linear Search

```
nums = []

n = int(input("Enter number of numbers in the list: "))

for i in range(n):
    value = int(input(f"Enter the {i+1}th number: "))
    nums.append(value)

print(nums)

target = int(input("Enter target element: "))

for index, value in enumerate(nums):
    if value == target:
        print(f"Element {target} found at index {index}")
        break
else:
    print("Element not found")
```

```
Enter number of numbers in the list:  3
Enter the 1th number:  1
Enter the 2th number:  2
Enter the 3th number:  3
[1, 2, 3]
Enter target element:  2
Element 2 found at index 1
```

The argument to `enumerate` must be an iterable object such as a list, tuple, string, dictionary, etc.

- An iterable object is any object you can loop over (one element at a time), i.e., an object that supports iteration.

`enumerate` returns an iterator that produces (count, item) tuples for each element of the iterable.

- An iterator is an object that produces values one at a time when requested.
- The count starts from 0 by default and the items are the values obtained while iterating over the iterable.

The `map` function (syntax: `map(function, iterable)`) returns an iterator

- Every element of the iterable is passed as an argument to the function specified yeilding the results

These results can be converted into a list, if needed

```
nums = list(map(int, input("Enter numbers separated by space: ").split()))
print(nums)
```

```
target = int(input("Enter target element: "))

positions = [i for i, v in enumerate(nums) if v == target]

if len(positions) > 0:
    print(f"Element {target} found at indices:", positions)
else:
    print("Element not found")
```

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Set is a mutable unordered unindexed collection of distinct values.
- Counter is a mutable unordered collection of values stored as dictionary keys with their counts stored as dictionary values.

References:

- https://docs.python.org/3/library/collections.html#collections.Counter
- https://docs.python.org/3/tutorial/datastructures.html#sets
- Think Python (Edition 3) by Allen B. Downey

Sets

Python provides a class called `set` that represents an unordered collection of distinct elements (not necessarily of the same type).

A set object is an unordered collection of distinct hashable objects

To create an empty set, we can use the `set` constructor.

```
s1 = set()
s1
```

```
set()
```

```
print(type(s1))
```

```
<class 'set'>
```

We can pass any kind of sequence to `set` constructor

```
s2 = set('acd')
s2
```

```
{'a', 'c', 'd'}
```

```
s3 = set([1,2,3])
s3
```

```
{1, 2, 3}
```

If you create a set with a sequence that contains duplicates, the result contains only unique elements.

```
s4 = set('banana')
s4
```

```
{'a', 'b', 'n'}
```

```
s5 = set([1,2,3,2])
s5
```

```
{1, 2, 3}
```

We can create a non-empty set by listing its elements (in arbitrary order) within curly braces.

```
s6 = {1,2,3}
s6
```

```
{1, 2, 3}
```

A **set comprehension** returns a new set. Syntax: `{` expression `for` member `in` iterable `if` condition `}`

To build the new set, obtain the elements from the elements of an iterable. The syntax includes an optional conditional at the end, which you can use to filter existing collections or generate elements conditionally.

```
s7 = {x for x in 'logicnotmagic' if x not in 'actg'}
s7
```

```
{'i', 'l', 'm', 'n', 'o'}
```

```
s8 = {x**2 for x in range(50) if x % 2 != 0}
print(s8)
```

```
{1, 9, 529, 1681, 25, 289, 2209, 169, 49, 441, 1849, 961, 1089, 841, 1225, 81, 1369, 729, 225, 2401, 361, 2025, 625, 1521, 1
```

Computing size of a set

```
len(s7)
```

```
5
```

Iterating over the elements of a set

```
fruits = {"apple", "banana", "mango"}

for x in fruits:
  print(x)
```

```
apple
mango
banana
```

Set elements cannot be changed but a set can be changed!

We can use the `add` method to add elements.

```
fruits.add('lemon')
fruits.add('pomogranate')
fruits
```

```
{'apple', 'banana', 'lemon', 'mango', 'pomogranate'}
```

```
fruits.add([1,2,3])
```

An element can only appear once in a `set`. If you add an element that's already there, it has no effect.

```
fruits.add('mango')
fruits
```

```
{'apple', 'banana', 'lemon', 'mango', 'pomogranate'}
```

We can use the `remove` method to remove elements.

```
fruits.remove('banana')
fruits
```

```
{'apple', 'lemon', 'mango', 'pomogranate'}
```

The `clear` method empties the set

```
fruits.clear()
```

The `del` statement will delete the set completely

```
del fruits
```

Set Operations

Methods that perform set operations

- `union` method returns a new set that contains all items from both sets
  - can use the `|` operator instead
- `intersection` method returns a new set that contains all the items that are present in both sets
  - can use the `&` operator instead
- `difference` method returns a new set that contains all the items from the first set that are not present in the other set
  - can use the `-` operator instead
- `symmetric_difference` method returns a new set that contains only the elements that are not present in both sets
  - can use the `^` operator instead

```
S1 = {1,2,3,4,5,6,7}
S2 = {5,10,15,20,25,30}
S1.union(S2), S1.intersection(S2), S1.difference(S2), S2.difference(S1), S1.symmetric_difference(S2)
```

```
({1, 2, 3, 4, 5, 6, 7, 10, 15, 20, 25, 30},
 {5},
 {1, 2, 3, 4, 6, 7},
 {10, 15, 20, 25, 30},
 {1, 2, 3, 4, 6, 7, 10, 15, 20, 25, 30})
```

```
S1, S2
```

```
({1, 2, 3, 4, 5, 6, 7}, {5, 10, 15, 20, 25, 30})
```

```
S3 = {2,4,6,8,10,12,14}
S4 = {5,10,15,20,25,30}
U = S3 | S4
I = S3 & S4
D = S3 - S4
S = S3 ^ S4
S3, S4, U, I, D, S
```

```
({2, 4, 6, 8, 10, 12, 14},
 {5, 10, 15, 20, 25, 30},
 {2, 4, 5, 6, 8, 10, 12, 14, 15, 20, 25, 30},
 {10},
 {2, 4, 6, 8, 12, 14},
 {2, 4, 5, 6, 8, 12, 14, 15, 20, 25, 30})
```

Set operations extend to more than two sets

```
S5 = {2,4,6,8,10,12,14}
S6 = {5,10,15,20,25,30}
S7 = {3,6,9,12,15,18}
U1 = S5 | S6 | S7
U2 = S5.union(S6,S7)
U1, U2
```

```
({2, 3, 4, 5, 6, 8, 9, 10, 12, 14, 15, 18, 20, 25, 30},
 {2, 3, 4, 5, 6, 8, 9, 10, 12, 14, 15, 18, 20, 25, 30})
```

```
I1 = S5 & S6 & S7
I1
```

```
set()
```

```
D1 = S5 - S6 - S7
D1
```

```
{2, 4, 8, 14}
```

```
S1 = S5 ^ S6 ^ S7
S1
```

```
{2, 3, 4, 5, 8, 9, 14, 18, 20, 25, 30}
```

Following methods perform set operations that change the original set instead of creating a new one

- `union_update`
- `intersection_update`
- `difference_update`

- `symmetric_difference_update`

To add items from another set into the current set, use the `update` method -- changes the original set

```
fruits = {"apple", "banana", "pomogranate"}
fruits1 = {"pineapple", "mango", "papaya"}

fruits.update(fruits1)

print(fruits,fruits1)
```

```
{'apple', 'pomogranate', 'mango', 'banana', 'pineapple', 'papaya'} {'mango', 'pineapple', 'papaya'}
```

The object in the `update` method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.)

```
thisset = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]

thisset.update(mylist)

print(thisset)
```

```
{'kiwi', 'orange', 'cherry', 'apple', 'banana'}
```

Relationship Between Sets

The relational operators work with sets. For example, `<=` checks whether one set is a subset of another, including the possibility that they are equal.

```
S = {1,2,3}
T = {1}
```

```
S <= T
```

```
False
```

```
S >= T
```

```
True
```

```
S == T
```

```
False
```

```
S < T
```

```
False
```

```
S > T
```

```
True
```

From Sets to Counters

A `Counter` is used to represent a multiset. The `Counter` class is defined in a module called `collections`. This module implements specialized container datatypes providing alternatives to Python's general purpose built-in containers (dict, list, set and tuple).

Sets and Counters are implemented as hash tables in Python.

```
from collections import Counter
```

```
counter = Counter('banana')
counter
```

```
Counter({'a': 3, 'n': 2, 'b': 1})
```

Alternatively, import the entire module

```
import collections

c = collections.Counter([1,2,2,3])
c
```

```
Counter({2: 2, 1: 1, 3: 1})
```

```
from collections import Counter

t = (1, 1, 1, 2, 2, 3)
counter = Counter(t)
counter
```

```
Counter({1: 3, 2: 2, 3: 1})
```

A `Counter` object is a collection where elements are stored as dictionary keys and their counts are stored as dictionary values. Counts are allowed to be any integer value including zero or negative counts.

Unlike dictionaries, `Counter` objects don't raise an exception if you access an element that doesn't appear. Instead, they return `0`.

```
counter['d']
```

```
0
```

Setting a count to zero does not remove an element from a counter.

```
counter
```

```
Counter({1: 3, 2: 2, 3: 1})
```

```
counter[1] = 0
counter
```

```
Counter({2: 2, 3: 1, 1: 0})
```

```
counter
```

```
Counter({2: 2, 3: 1, 1: 0})
```

```
counter[4] = 5
counter
```

```
Counter({4: 5, 2: 2, 3: 1, 1: 0})
```

```
counter[5] = -1
counter
```

```
Counter({4: 5, 2: 2, 3: 1, 1: 0, 5: -1})
```

```
counter[[1,2,3]] = -1
counter
```

Counter objects support additional methods beyond those available for dictionaries. They also provide methods and operators to perform set-like operations, including addition, subtraction, union and intersection.

The `elements` method returns an iterator over elements repeating each as many times as its count. Elements are returned in the order encountered. If an element's count is less than one, it is ignored.

```
counter
```

```
Counter({4: 5, 2: 2, 3: 1, 1: 0, 5: -1})
```

```
sorted(counter.elements())
```

```
[2, 2, 3, 4, 4, 4, 4, 4]
```

```
counter
```

```
Counter({4: 5, 2: 2, 3: 1, 1: 0, 5: -1})
```

The `most_common` method returns a list of value-frequency pairs, sorted from most common to least. Elements with equal counts are ordered in the order first encountered.

```
counter.most_common()
```

```
[(4, 5), (2, 2), (3, 1), (1, 0), (5, -1)]
```

```
counter.most_common(2)
```

```
[(4, 5), (2, 2)]
```

The `total` method computes the sum of the counts

```
counter.total()
```

```
7
```

The `+` operator combines two `Counter` objects and creates a new `Counter` that contains the keys from both and the sums of the counts

```
counter2 = Counter('bans')
counter + counter2
```

```
Counter({4: 5, 2: 2, 3: 1, 'b': 1, 'a': 1, 'n': 1, 's': 1})
```

The `items` method is used to access the (element, count) pairs

```
counter.items()
```

```
dict_items([(1, 0), (2, 2), (3, 1), (4, 5), (5, -1)])
```

```
counter.clear()
counter
```

```
Counter()
```

```
c = collections.Counter([1,2,2,3])
c
```

```
Counter({2: 2, 1: 1, 3: 1})
```

```
c[5] = -1
c
```

```
Counter({2: 2, 1: 1, 3: 1, 5: -1})
```

```
s = set(c)
s
```

```
{1, 2, 3, 5}
```

Here are some more common usage of counters.

```
c.clear()                       # reset all counts
list(c)                         # list unique elements
set(c)                          # convert to a set
dict(c)                         # convert to a regular dictionary

list_of_pairs = (1,1), (2,2), (3,3)
Counter(dict(list_of_pairs))    # convert from a list of (elem, cnt) pairs
```

```
Counter({3: 3, 2: 2, 1: 1})
```

```
list_of_pairs = (1,1), (1,3), (3,3)
Counter(list_of_pairs)
```

```
Counter({(1, 1): 1, (1, 3): 1, (3, 3): 1})
```

```
list_of_pairs = (1,1), (1,3), (3,3)
Counter(dict(list_of_pairs))
```

```
Counter({1: 3, 3: 3})
```

```
c = collections.Counter([1,2,2,3])
n = 4
c.most_common()[:-n-1:-1]       # n least common elements
```

```
[(3, 1), (1, 1), (2, 2)]
```

```
c[5] = 6
c[6] = 0
c[7] = -1
```

```
c
```

```
Counter({5: 6, 2: 2, 1: 1, 3: 1, 6: 0, 7: -1})
```

```
+c     # remove zero and negative counts
c
```

```
Counter({5: 6, 2: 2, 1: 1, 3: 1, 6: 0, 7: -1})
```

```
+c
```

```
Counter({5: 6, 2: 2, 1: 1, 3: 1})
```

Counters also support rich comparison operators for equality, subset, and superset relationships: `==`, `!=`, `<`, `<=`, `>`, `>=`. All of these tests treat missing elements as having zero counts.

Detect Duplicates in a List

```
nums = [4,7,2,4,8,7,4,9,2]

c = Counter(nums)

duplicates = [n for n,count in c.items() if count > 1]

print(duplicates)
```

```
[4, 7, 2]
```

```
unique = [n for n,count in c.items() if count == 1]
print(unique)
```

```
[8, 9]
```

Anagrams Detector

Let us define a function that takes two words and checks whether they are anagrams -- that is, whether the letters from one can be rearranged to spell the other.

```
def is_anagram(word1, word2):
    return Counter(word1) == Counter(word2)
```

If two words are anagrams, they contain the same letters with the same counts, so their `Counter` objects are equivalent.

```
is_anagram('silent', 'listen')
```

```
True
```

```
is_anagram('silent', 'violent')
```

```
False
```

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Scope and Global Variables
- Packing and Unpacking Operators

Reference:

- https://www.cmi.ac.in/~madhavan/courses/python2025/
- Think Python (Edition 3) by Allen B. Downey

Example 1

Consider the following function definition. Note that the value of `x` is not yet known. However, at this point, Python only parses the function definition i.e, it reads it and checks if the definition is valid syntactically.

```
def f():
    y = x + 7
    print(y)
    return
```

Let us now call the function.

```
f()
```

The error is raised when the function executes. However, if `x` is defined before the function call, no error is raised.

```
x = 7
f()
```

```
14
```

The **scope** of a variable refers to the portion of the program where its value is available. If we refer to a value that is not defined in a function, it is looked up in the **global** context.

Example 2

As soon as we assign a variable a value inside a function, all occurrences of the variable are treated as local to the function

- This decision is **static** based on the program text. In the code below, we cannot be sure that the assignment `z = 10` will execute, but Python still treats `z` to be local to `f`
- Though the check is based on the static program text, the error is flagged only when the function executes. The definition of `f` does not trigger an error though the problem is evident in the text of the function.

```
def g():
    y = z + 5 # raises error though there is a global z
    print(y)
    if y > 1000:
        z = 10  #this makes z local
    return
```

```
z = 7
g()
```

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[4], line 2
      1 z = 7
----> 2 g()

Cell In[3], line 2, in g()
      1 def g():
----> 2     y = z + 5 # raises error though there is a global z
      3     print(y)
      4     if y > 1000:

UnboundLocalError: cannot access local variable 'z' where it is not
associated with a value
```

This static check applies even if it is impossible for the local assignment to be executed

```
def check():
    m = n + 2    # raises error even though there is global n
    return
    if False:
        n = 7     #n is local
```

```
n = 8
check()
```

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[6], line 2
      1 n = 8
----> 2 check()

Cell In[5], line 2, in check()
      1 def check():
----> 2     m = n + 2    # raises error even though there is global n
      3     return
      4     if False:

UnboundLocalError: cannot access local variable 'n' where it is not
associated with a value
```

Example 3

Here is another example of using global values within a function without redefining the variable

```
def display_count():
    print(count)
    return
```

If we call `display_count` without a global definition for `count` we get an error

```
display_count()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[8], line 1
----> 1 display_count()

Cell In[7], line 2, in display_count()
      1 def display_count():
----> 2     print(count)
      3     return

NameError: name 'count' is not defined
```

If `count` is available in the global context, the function works as expected

```
count = 7
display_count()
```

```
7
```

If we try to update `count` inside the function, all occurrences become local (see the below snippet)

- The occurrence on the right hand side of the assignment generates an error because its value is now undefined
- Once again, this error is only triggered at run-time when the function executes

```
def increment_local(k):
    count = count + k # count is local, assignment raises an err even though
    return
```

```
increment_local(2)
```

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[11], line 1
----> 1 increment_local(2)

Cell In[10], line 2, in increment_local(k)
      1 def increment_local(k):
----> 2     count = count + k # count is local, assignment raises an err
even though there is global count
      3     return

UnboundLocalError: cannot access local variable 'count' where it is not
associated with a value
```

```
count
```

```
7
```

Reassigning a variable within a function also disconnects it from the external variable with the same name

```
def reset_local(k):
    count = k # count is local
    print(count)
    return
```

```
reset_local(77)
```

```
77
```

```
count
```

```
7
```

We can declare a variable to be `global` to override Python's default scope rules

- `global` tells Python to treat the variable inside the function as one from the global context

```
def increment_global(k):
    global count
    count = count + k
    return
```

```
increment_global(8)
display_count()
```

```
15
```

Example 4

Here's an example involving lists.

```
def concat_local():
    l1 = l1 + l2 #l1 is local and this assign raises an error, l2 is taken f
    return
```

```
l1 = [1,2,3]
l2 = [4,5,6]
concat_local()
```

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[24], line 3
      1 l1 = [1,2,3]
      2 l2 = [4,5,6]
----> 3 concat_local()

Cell In[23], line 2, in concat_local()
      1 def concat_local():
----> 2     l1 = l1 + l2 #l1 is local and this assign raises an error, l2 is
taken from global context
      3     return

UnboundLocalError: cannot access local variable 'l1' where it is not
associated with a value
```

```
def concat_global():
    global l1
    l1 = l1 + l2
    return
```

```
l1 = [1,2,3]
l2 = [4,5,6]
concat_global()
```

```
l1, l2
```

```
([1, 2, 3, 4, 5, 6], [4, 5, 6])
```

Example 5

We can define a value inside a function and "export" it outside by declaring it `global`

```
def concat_global():
    global l3
    l3 = [1,2,3]
    l3 = l3 + l4
    return
```

```
l4 = [4,5,6]
concat_global()
```

```
l3
```

```
[1, 2, 3, 4, 5, 6]
```

The following would work with *dynamic* scoping -- based on execution of program

- `init` defines `b` and then calls `change`, so with dynamic scoping, `b` is known to `change`

However, Python uses *static* scoping -- based on text of program -- so this code generates an error

- Most languages use static scoping because dynamic scoping makes it hard to reason about correctness

```
def change():
    a = b + 5
    print(a)

def initialize():
    b = 7
    change()
```

```
initialize()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[32], line 1
----> 1 initialize()

Cell In[31], line 7, in initialize()
      5 def initialize():
      6     b = 7
----> 7     change()

Cell In[31], line 2, in change()
      1 def change():
----> 2     a = b + 5
      3     print(a)

NameError: name 'b' is not defined
```

Example 6

Functions can take a variable number of arguments. A parameter name that begins with the `*` operator **packs** arguments into a tuple.

```
def mean(*vals):
    return(sum(vals)/len(vals))
```

```
mean(1,2,3)
```

```
2.0
```

```
mean(1,2,3,4,5)
```

```
3.0
```

```
def mean1(vals):
    return(sum(vals)/len(vals))
```

```
mean1((1,2,3))
```

```
2.0
```

If you have a sequence of values and you want to pass them to a function as multiple arguments, you can use the `*` operator to **unpack** the tuple (or other sequence).

```
t = (7,3)
divmod(t)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[38], line 2
      1 t = (7,3)
----> 2 divmod(t)

TypeError: divmod expected 2 arguments, got 1
```

```
t = (7,3)
divmod(*t)
```

```
(2, 1)
```

Packing and unpacking can be useful if you want to adapt the behaviour of an existing function.

- pack: Collect multiple arguments into a tuple.
- unpack: Treat a tuple (or other sequence) as multiple arguments.

The `*` operator doesn't pack keyword arguments. To pack keyword arguments, we can use the `**` operator. This operator can also be used in an argument list to unpack a dictionary.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Assertions

Assertions

The `assert` statement is a debugging aid that

- Evaluates a boolean expression and checks if the result is `True` or `False`.
- If the result is `True`, it does nothing.
- If the result is `False`, it immediately raises an `AssertionError` exception which halts the program's execution.
- This mechanism helps developers find bugs early in the development and testing phases.

A common use case is ensuring that a function's input is valid (precondition)

```
def factorial(n):
    assert isinstance(n, int)
    assert n >= 0, "n has to be a non-negative integer" #With error message
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
    return fact
```

```
factorial(-1)
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[2], line 1
----> 1 factorial(-1)

Cell In[1], line 3, in factorial(n)
      1 def factorial(n):
      2     assert isinstance(n, int)
----> 3     assert n >= 0, "n has to be a non-negative integer" #With error message
      4     fact = 1
      5     for i in range(1, n + 1):

AssertionError: n has to be a non-negative integer
```

Assertions can be disabled when Python is run in optimized mode (eg: `python -O script.py`).

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Runtime Errors (Exceptions)

Reference: https://docs.python.org/3/tutorial/errors.html

Exceptions

A syntactically correct statement or expression may cause an error when an attempt is made to execute it. Such errors detected during execution are called **exceptions** and are not unconditionally fatal. It is possible to write programs that handle selected exceptions.

Consider the following program to compute the factorial of a given number.

```
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
    return fact
```

```
user_input = input("Enter the integer whose factorial you want to compute: ")
num = int(user_input)
if num < 0:
    print("Invalid Input - Please enter a non-negative integer!")
else:
    fact = factorial(num)
    print(f"Factorial of {num} is {fact}")
```

Question: What happens if the user enters a non-integer as input?

`ValueError` is raised when an operation or function receives an argument that has an inappropriate value

```
user_input = input("Enter the integer whose factorial you want to compute: ")
#str.isdigit() returns True if the string str is non-empty, all chars are digits & False otherwise
if not user_input.isdigit():
    print("Invalid Input - Please enter a non-negative integer!")
else:
    num = int(user_input)
    if num < 0:
        print("Invalid Input - Please enter a non-negative integer!")
    else:
        fact = factorial(num)
        print(f"Factorial of {num} is {fact}")
```

```
Invalid Input - Please enter a non-negative integer!
```

Question: What happens if the user enters +5 as input? Or 5 followed by an unintentional space?

Let us handle exceptions using `try` and `except` constructs instead of making the program's code unnecessarily complex by adding additional checks.

```
try:
    stmt11
    stmt12
    ...

except ErrorName:
    stmt21
    stmt22
    ...

stmt3
```

```
user_input = input("Enter the number whose factorial you want to compute: ")
try:
    num = int(user_input)
    if num < 0:
        print("Input cannot be a negative integer!")
    else:
        fact = factorial(num)
        print(f"Factorial of {num} is {fact}")
except ValueError:
    print("Input has to be a non-negative integer!")
```

```
Input has to be a non-negative integer!
```

Question: Can we repeatedly prompt the user for input until the provided data meets the specified criteria?

`while` loop along with `try`-`except` can be useful for input validation

```
while True:
    user_input = input("Enter the number whose factorial you want to compute: ")
    try:
        num = int(user_input)
        if num < 0:
            print("Input cannot be a negative integer!")
            continue
        else:
            fact = factorial(num)
            print(f"Factorial of {num} is {fact}")
            break
    except ValueError:
        print("Input has to be a non-negative integer!")
        continue
    except Exception as e:
        print("Unexpected error:", e)
```

```
Unexpected error: invalid literal for int() with base 10: '9.6'
Factorial of 9 is 362880
```

When you run the above code, you'll be prompted to enter a non-negative integer until you enter one.

This ensures that by the time the execution leaves the while loop, the num variable will contain a valid value.

```
try:
    stmt11
    stmt12
    ...

except NameOfError1:
    stmt21
    stmt22
    ...

except NameOfError2:
    stmt31
    stmt32
    ...

except Exception:
    stmt41
    stmt42
    ...

stmt5
```

- First, the try clause (the statement(s) between the try and except keyword(s)) is executed.
- If no exception occurs, the except clause is skipped and execution of the try statement is finished.
- If an exception occurs during execution of the try clause, the rest of the clause is skipped. Then, if its type matches the exception named after the except keyword, the except clause is executed, and then execution continues after the try/except block.
- If an exception occurs which does not match the exception named in the except clause, it is passed on to outer try statements; if no handler is found, it is an unhandled exception and execution stops with an error message.

- A try statement may have more than one except clause, to specify handlers for different exceptions. At most one handler will be executed.
- Handlers only handle exceptions that occur in the corresponding try clause, not in other handlers of the same try statement. An except clause may name multiple exceptions as a parenthesized tuple.

Here's another example.

```
try:
  print(z)
except NameError:
  print("Variable z not defined")
except (TypeError,ValueError):
  print("Type Error or Value Error")
except ZeroDivisionError:
  print("Attempt to Divide by 0")
except Exception as e:
  print("Unexpected Error",e)
else:
  print("No Error")
finally:
  print("Execution of try-except blocks complete!")
```

Summarizing properties of `try`-`except`,

- The `try` and `except` blocks let you handle runtime errors (exceptions)
- If the `try` block raises an error, the except (exception) block will be executed
- Without `try`-`except` blocks, the program may raise an error and crash
- You can define as many `except` blocks as you want
- The `else` block is executed when there is no error
- The `finally` block is executed regardless of the result of the try and except blocks

The `raise` statement allows the programmer to force a specified exception to occur. This is useful for signaling error conditions in code which stops the normal flow of the program and transfers control to an exception handler.

```
def factorial(n):
    if n < 0:
        raise ValueError("Factorial not defined for negative numbers")

    fact = 1
    for i in range(1, n+1):
        fact = fact * i
    return fact
```

Note: Python's built-in operations automatically raise exceptions for various errors without requiring manual intervention.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Reading and writing files

Reference: Think Python (Edition 3) by Allen B. Downey

Files

The programs that we have seen so far are **ephemeral** in the sense that they run for a short time, produce output and when they terminate, the data in memory is lost.

- Variables, lists, dictionaries, tuples, sets, etc. only offer temporary data storage and the data is lost when the program terminates.

We can write programs that are **persistent** by storing at least some of the data in long-term storage. A simple way for programs to maintain their data is by reading and writing **files** and a more versatile alternative is to store data in a **database**.

Filenames and Paths

- Files are organized into **directories**, also called "folders".
- Every running program has a **current working directory**, which is the default directory for most operations.
- For example, when you open a file, Python looks for it in the current working directory.

The `os` (operating system) module provides functions for working with files and directories. It provides a function called `getcwd` that gets the name of the current working directory.

```
import os

os.getcwd()
```

```
'/Users/krithika/Documents/Courses/Python26'
```

- A string like `/Users/krithika/Documents/Courses/Python26` that identifies a file or directory is called a **path**.

- A simple filename like `words.txt` is also considered a path, but it is a **relative path** because it specifies a file name relative to the current directory.
- In this example, the current directory is `/Users/krithika/Documents/Courses/Python26`, so `words.txt` is equivalent to the complete path `/Users/krithika/Documents/Courses/Python26/words.txt`.
- A path that begins with `/` does not depend on the current directory -- it is called an **absolute path**.

To find the absolute path to a file, you can use `abspath`.

```
os.path.abspath('words.txt')
```

```
'/Users/krithika/Documents/Courses/Python26/words.txt'
```

The `os` module provides other functions for working with filenames and paths.

- `listdir` returns a list of the contents of the given directory, including files and other directories.
- Here's an example that lists the contents of a directory named `Lab`.

```
os.listdir('Lab')
```

```
['ExerciseSheet8.pdf',
 '.DS_Store',
 'ExerciseSheet1.pdf',
 'ExerciseSheet2.pdf',
 'ExerciseSheet3.pdf',
 'ExerciseSheet7.pdf',
 'ExerciseSheet3-Hints.pdf',
 'ExerciseSheet6.pdf',
 'ExerciseSheet5.pdf']
```

To check whether a file or directory exists, we can use `os.path.exists`.

```
os.path.exists('Quiz')
```

```
False
```

```
os.path.exists('Theory')
```

```
True
```

```
os.path.exists('Theory/Ref')
```

```
False
```

```
os.path.exists('Theory/Ref/intro.txt')
```

```
False
```

```
os.path.exists('lec00-intro.ipynb')
```

```
True
```

```
os.path.exists('Theory/lec00-intro.ipynb')
```

```
False
```

To check whether a path refers to a file or directory, we can use `isdir`, which return `True` if a path refers to a directory.

```
os.path.isdir('Lab')
```

```
True
```

And `isfile` which returns `True` if a path refers to a file.

```
os.path.isfile('words.txt')
```

```
True
```

Reading Files

Let's read a book -- *The Adventures of Sherlock Holmes* by Arthur Conan Doyle -- the version that is available from Project Gutenberg (https://www.gutenberg.org/).

The book is in a file called `pg1661.txt`.

- To read it, we'll use the built-in function `open`, which takes the name of the file as a parameter and returns a **file object**.
- A **file object** is an object that represents an open file and keeps track of which parts of the file have been read or written.

```
book = 'pg1661.txt'
file_obj = open(book)
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_2130/2805317443.py in <cell line: 0>()
      1 book = 'pg1661.txt'
----> 2 file_obj = open(book)

FileNotFoundError: [Errno 2] No such file or directory: 'pg1661.txt'
```

The `readline` method reads characters from the file until it gets to a newline and returns the result as a string.

```
file_obj.readline()
```

```
'\ufeffThe Project Gutenberg eBook of The Adventures of Sherlock Holmes\n'
```

The file object keeps track of where it is in the file, so if you call `readline` again, you get the next line.

```
line = file_obj.readline()
line
```

```
'Title: The Adventures of Sherlock Holmes\n'
```

To remove the newline from the end of the word, we can use `strip` method. It removes whitespace characters -- including spaces, tabs, and newlines -- from the beginning and end of the string.

```
line1 = line.strip()
line1
```

```
'Title: The Adventures of Sherlock Holmes'
```

Once we start reading a file and decide to re-read at some point, we have to close it and open it again.

Files should be closed after use to free system resources and ensure data integrity. One way to close a file is to explicitly call the `close` method.

```
file_obj.close()
```

A file object is an iterator -- a for loop can directly request the next line from the file as needed.

- In the below snippet, the `readline` method is implicitly called.

Let us read and print all lines in the file.

```
total = 0
file_obj = open(book)
for line in file_obj:
    print(line.strip())
    total += 1
total
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_2130/419487991.py in <cell line: 0>()
      1 total = 0
----> 2 file_obj = open(book)
      3 for line in file_obj:
      4     print(line.strip())
      5     total += 1

FileNotFoundError: [Errno 2] No such file or directory: 'pg1661.txt'
```

```
file_obj.close()
```

```
file_obj.readline()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[41], line 1
----> 1 file_obj.readline()

ValueError: I/O operation on closed file.
```

Alternatively, we can use the `with` statement -- automatically handles file closing.

```
total = 0
with open(book) as file_obj:
    for line in file_obj:
        print(line.strip())
        total += 1
total
```

```
The Project Gutenberg eBook of The Adventures of Sherlock Holmes

This ebook is for the use of anyone anywhere in the United States and
most other parts of the world at no cost and with almost no restrictions
whatsoever. You may copy it, give it away or re-use it under the terms
of the Project Gutenberg License included with this ebook or online
at www.gutenberg.org. If you are not located in the United States,
you will have to check the laws of the country where you are located
before using this eBook.

Title: The Adventures of Sherlock Holmes
```

```
Author: Arthur Conan Doyle

Release date: March 1, 1999 [eBook #1661]
Most recently updated: October 10, 2023

Language: English

Credits: an anonymous Project Gutenberg volunteer and Jose Menendez


*** START OF THE PROJECT GUTENBERG EBOOK THE ADVENTURES OF SHERLOCK HOLMES ***




The Adventures of Sherlock Holmes

by Arthur Conan Doyle


Contents

I.     A Scandal in Bohemia
II.    The Red-Headed League
III.   A Case of Identity
IV.    The Boscombe Valley Mystery
V.     The Five Orange Pips
VI.    The Man with the Twisted Lip
VII.   The Adventure of the Blue Carbuncle
VIII.  The Adventure of the Speckled Band
IX.    The Adventure of the Engineer's Thumb
X.     The Adventure of the Noble Bachelor
XI.    The Adventure of the Beryl Coronet
XII.   The Adventure of the Copper Beeches




I. A SCANDAL IN BOHEMIA


I.

To Sherlock Holmes she is always _the_ woman. I have seldom heard him
mention her under any other name. In his eyes she eclipses and
```

```
file_obj.readline()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[43], line 1
----> 1 file_obj.readline()

ValueError: I/O operation on closed file.
```

We can also read the entire contents of the file in one shot.

```
total = 0
with open(book) as file_obj:
    content = file_obj.read()     #string
    lines = content.splitlines()
    for line in lines:
        print(line.strip())
        total += 1
print(total)
```

```
The Project Gutenberg eBook of The Adventures of Sherlock Holmes

This ebook is for the use of anyone anywhere in the United States and
most other parts of the world at no cost and with almost no restrictions
whatsoever. You may copy it, give it away or re-use it under the terms
of the Project Gutenberg License included with this ebook or online
at www.gutenberg.org. If you are not located in the United States,
you will have to check the laws of the country where you are located
before using this eBook.

Title: The Adventures of Sherlock Holmes

Author: Arthur Conan Doyle

Release date: March 1, 1999 [eBook #1661]
Most recently updated: October 10, 2023

Language: English

Credits: an anonymous Project Gutenberg volunteer and Jose Menendez


*** START OF THE PROJECT GUTENBERG EBOOK THE ADVENTURES OF SHERLOCK HOLMES ***




The Adventures of Sherlock Holmes

by Arthur Conan Doyle


Contents

I.     A Scandal in Bohemia
II.    The Red-Headed League
III.   A Case of Identity
IV.    The Boscombe Valley Mystery
V.     The Five Orange Pips
VI.    The Man with the Twisted Lip
VII.   The Adventure of the Blue Carbuncle
VIII.  The Adventure of the Speckled Band
IX.    The Adventure of the Engineer's Thumb
X.     The Adventure of the Noble Bachelor
XI.    The Adventure of the Beryl Coronet
```

```
XII.   The Adventure of the Copper Beeches



I. A SCANDAL IN BOHEMIA


I.

To Sherlock Holmes she is always _the_ woman. I have seldom heard him
mention her under any other name. In his eyes she eclipses and
```

We can strip off leading and trailing white space (blank, tab, newline) from a string using `lstrip`, `rstrip` and `strip` methods.

```
with open(book) as file_obj:
    lines = file_obj.readlines() #list of lines and each line is terimated b
for line in lines:
    print(line.strip())
print(len(lines))
```

```
with open(book) as file_obj:
    lines = list(file_obj)
for line in lines:
    print(line.strip())
print(len(lines))
```

```
with open(book) as file_obj:
    lines_stripped = [x.strip() for x in file_obj]
for line in lines_stripped:
    print(line)
print(len(lines_stripped))
```

Writing Files

Modes of opening a file - `open` takes an optional parameters that specifies the "mode".

```
read = open("newfile.txt","r") #same as read = open("newfile.txt")
```

```
read.close()
```

```
write = open("newfile.txt","w")
```

- Opening a non-existent file for reading generates an error

- Opening a non-existent file for writing creates a new file
  - If the file already exists, its contents will be overwritten
- `file_obj.write(s)` writes the string `s` to the associated file
  - Need to add `\n` ourselves to ensure the end of line
- `file_obj.writelines(ls)` writes a list of strings `ls`
  - Again, we need to ensure `\n` is present at the end of each string in the list

```
write.write("Python\n")
```

```
7
```

```
write.close()
```

```
write = open("newfile.txt","w")
write.writelines(["1\t","2\t","3"])
```

```
write.close()
```

A third mode of opening a file to append our writes at the end of a file

```
append = open("newfile.txt","a")
```

```
append.write("\nHello\n")
append.close()
```

There are other methods like

- `fh.seek(n)` moves to position `n` in the file
  - After reading a file, use `fh.seek(0)` to return to the start
- `fh.read(n)` reads `n` characters from the current position

Processing Files

In addition to the text of the book, `pg1661.txt` contains a section at the beginning with information about the book and a section at the end with information about the license. Before we process the text, we can remove this extra material by finding the special lines at the beginning and end that begin with `'***'`.

The following function takes a line and checks whether it is one of the special lines.

- It uses the `startswith` method, which checks whether a string starts with a given sequence of characters.

```
def is_special_line(line):
    return line.strip().startswith('***')
```

We can use this function to loop through the lines in the file and print only the special lines.

```
with open(book) as file_obj:
    for line in file_obj:
        if is_special_line(line):
            print(line.strip())
```

```
*** START OF THE PROJECT GUTENBERG EBOOK THE ADVENTURES OF SHERLOCK HOLMES **
*** END OF THE PROJECT GUTENBERG EBOOK THE ADVENTURES OF SHERLOCK HOLMES ***
```

Now let's create a new file, called `holmes.txt`, that contains only the text of the book.

```
def clean_file(input_file, output_file):
    reader = open(input_file)
    writer = open(output_file, 'w')
    for line in reader:
        if is_special_line(line):
            break
    # The file pointer now sits right after the first special line
    # The following loop reads the rest of the file, one line at a time.
    # When it finds the special line that indicates the end of the text, it
    # Otherwise, it writes the line to the output file.
    for line in reader:
        if is_special_line(line):
            break
        writer.write(line)
    reader.close()
    writer.close()
```

```
clean_file(book,'holmes.txt')
```

To check whether this process was successful, we can read the first few lines from the new file we just created.

```
with open('holmes.txt') as file_obj:
    for line in file_obj:
        line = line.strip()
        if len(line) > 0:
            print(line)
```

```
            if line.endswith('BOHEMIA'):
                break
```

```
The Adventures of Sherlock Holmes
by Arthur Conan Doyle
Contents
I.     A Scandal in Bohemia
II.    The Red-Headed League
III.   A Case of Identity
IV.    The Boscombe Valley Mystery
V.     The Five Orange Pips
VI.    The Man with the Twisted Lip
VII.   The Adventure of the Blue Carbuncle
VIII.  The Adventure of the Speckled Band
IX.    The Adventure of the Engineer’s Thumb
X.     The Adventure of the Noble Bachelor
XI.    The Adventure of the Beryl Coronet
XII.   The Adventure of the Copper Beeches
I. A SCANDAL IN BOHEMIA
```

The `endswith` method checks whether a string ends with a given sequence of characters.

Various types of exceptions can occur while working with files. It is a good programming practice to handle them instead of letting the program crash.

- **FileNotFoundError** - occurs if you attempt to open a non-existent file for reading
- **PermissionsError** - occurs if you attempt an operation for which you do not have permission
- **ValueError** - occurs if you attempt to write a file that has already been closed

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- NumPy (Numerical Python)

References:

- https://numpy.org/doc/stable/user/quickstart.html
- https://www.cmi.ac.in/~madhavan/courses/python2025/

NumPy is an open-source library for scientific computing.

Computational science is a branch of computer science that uses advanced computing techniques, mathematical modeling and simulation to analyze and solve complex problems in science and engineering.

```
import numpy as np
```

The central data structure in NumPy is called **ndarray** (n-dimensional array).

- It is a table of elements (usually numbers), all of the same type, indexed by a tuple of non-negative integers. In NumPy dimensions are called axes.
- `ndarray` is a class (also known by the alias `array`), possessing numerous methods and attributes.
- `ndarray` objects have a fixed size at creation. Changing the size of an ndarray will create a new array and delete the original.

Most array operations execute significantly faster than corresponding list operations

Vectors using Arrays

Let us begin with creating a vector (1D array).

`np.array` function constructs an array from an input sequence

- Sequence can be a list, tuple, output of a `range`, list of lists, etc.
- Size of the array is fixed by the sequence
- Underlying type is also fixed by the sequence

```
a = np.array([10,2,-3])
```

```
a
```

```
array([10,  2, -3])
```

NumPy auto-formats arrays aligning the columns within each row.

```
print(a)
```

```
[10  2 -3]
```

```
type(a)
```

```
numpy.ndarray
```

We can create arrays by passing ranges as arguments

```
b = np.array(range(10))
print(b)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

However, it is better to use `arange` function as it is optimized for arrays

```
c = np.arange(3)
d = np.arange(5,10)
e = np.arange(10,1,-2)
print(c,d,e)
```

```
[0 1 2] [5 6 7 8 9] [10  8  6  4  2]
```

Important attributes of an array object are as follows

- `ndarray.ndim` - the number of axes (dimensions) of the array.
- `ndarray.shape` - the dimensions of the array. This is a tuple of integers indicating the size of the array in each dimension. For a matrix with n rows and m columns, shape will be (n,m). The length of the shape tuple is therefore the number of axes, ndim.
- `ndarray.size` - the total number of elements of the array. This is equal to the product of the elements of shape.
- `ndarray.dtype` - an object describing the type of the elements in the array.

```
a
```

```
array([10,  2, -3])
```

```
a.ndim
```

```
1
```

```
a.shape
```

```
(3,)
```

```
a.size
```

```
3
```

```
a.dtype
```

```
dtype('int64')
```

In this case, each element is stored as a 64-bit (i.e., 8-byte) integer. To get the number of bytes needed to store each element,

```
a.itemsize
```

```
8
```

There are often cases when we want to initialize the values of the array while creation. Methods like `ones`, `zeros` and `random.random` are useful in such cases.

```
c = np.zeros(5)
print(c)
```

```
[0. 0. 0. 0. 0.]
```

```
c1 = np.zeros(5,dtype='int64')
print(c1)
```

```
[0 0 0 0 0]
```

`random` function of the `random` module of the `numpy` library is used to generate random floating-point numbers in $[0.0, 1.0)$

```
d = np.random.random(5)
print(d)
```

```
[0.62335083 0.78932893 0.0071171  0.16385561 0.56097097]
```

We can index and slice 1D arrays like lists

```
print(a, a[0], a[0:2], a[-1])
```

```
[10  2 -3] 10 [10  2] -3
```

We can use aggregation methods -- there are methods like min, max, sum, mean to get the average, prod to get the result of multiplying all the elements together, std to get standard deviation and many more.

```
print(a.min(), a.max(), a.sum())
```

Matrices using Arrays

Let us create a matrix (2D array) using a list of lists

```
c = np.array([[0,1,0],[2,3,2]])
print(c)
```

NumPy auto-formats arrays based on the dimension, aligning the columns within each row.

```
c1 = np.array([[0,1,0],[2,2]])
print(c1)
```

```
d = np.array([(0,1,0),range(3)])
print(d)
```

```
[[0 1 0]
 [0 1 2]]
```

We can also use the methods `ones`, `zeros` and `random.random` as long as we pass a tuple describing the dimensions of the matrix we are creating

```
e = np.zeros((5,3))  # np.zeros(shape) where shape = (dim1,dim2,...,dimk)
print(e)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

```
f = np.zeros(5,3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_1059/3537978275.py in <cell line: 0>()
----> 1 f = np.zeros(5,3)

TypeError: Cannot interpret '3' as a data type
```

```
g = np.random.random((4,6))
print(g)
```

```
[[0.86003312 0.14262628 0.46590897 0.04797853 0.7142981  0.14002904]
 [0.5314898  0.3662193  0.26022932 0.47450758 0.57214834 0.26399596]
 [0.4175918  0.91384647 0.06502573 0.8612592  0.75473237 0.44877912]
 [0.78000274 0.02988434 0.25757093 0.34080014 0.65003332 0.55716013]]
```

```
h = np.random.randint(1,100,(3,4))
print(h)
```

Arrays are iterable

```
nums = np.array([[1,2,3],[4,5,6]])
for row in nums:
        for col in row:
                print(col,end=' ')
        print()
```

```
1 2 3
4 5 6
```

We can also iterate through a higher dimensional array as if it were one-dimensional by using the `flat` attribute

```
for i in nums.flat:
    print(i,end = ' ')
```

```
1 2 3 4 5 6
```

Reshaping and Copying Arrays

```
a1 = np.arange(1,21)
a1.shape
```

```
(20,)
```

```
a2 = np.arange(1,21).reshape(4,5)
print(a2)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
```

```
a2.shape
```

```
(4, 5)
```

```
a2
```

```
array([[ 1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10],
       [11, 12, 13, 14, 15],
       [16, 17, 18, 19, 20]])
```

```
a2.shape = (10,2)
```

```
a2
```

```
array([[ 1,  2],
       [ 3,  4],
       [ 5,  6],
       [ 7,  8],
       [ 9, 10],
       [11, 12],
       [13, 14],
       [15, 16],
       [17, 18],
       [19, 20]])
```

The `reshape` method is used to change the shape of an array without modifying its underlying data, provided the new shape has the same total number of elements as the original array.

```
a2.reshape(5,4)
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16],
       [17, 18, 19, 20]])
```

```
a2
```

```
a = np.array([[0,1,0],[2,3,2],[4,5,6],[7,8,9]])
```

```
a.shape
```

```
(4, 3)
```

```
a.shape = 6, 2
print(a)
```

```
[[0 1]
 [0 2]
 [3 2]
 [4 5]
 [6 7]
 [8 9]]
```

```
a.shape = 1,12
print(a)
```

```
a.shape = 2,3,2
print(a)
```

Copying arrays

```
c = a.copy()  # Creates a copy of the array
print(c)
```

Updating `c` has no effect on the others since it is a different copy

```
print(a)
```

We can flatten an array into a sequence

```
for i in a.flat:
    print(i,end=" ")
```

Array Arithmetic

Once we've created arrays, we can start to manipulate them in interesting ways.

```
v1 = np.arange(1,11)
print(v1)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

```
v2 = np.arange(-1,-11,-1)
print(v2)
```

```
[ -1  -2  -3  -4  -5  -6  -7  -8  -9 -10]
```

```
print(v1+v2)
```

```
[0 0 0 0 0 0 0 0 0 0]
```

```
print(v1-v2)
```

```
[ 2  4  6  8 10 12 14 16 18 20]
```

```
print(v2-v1)
```

```
[ -2  -4  -6  -8 -10 -12 -14 -16 -18 -20]
```

```
print(v1 * v2)
```

```
[  -1   -4   -9  -16  -25  -36  -49  -64  -81 -100]
```

```
print(v1 // v2)
```

```
[-1 -1 -1 -1 -1 -1 -1 -1 -1 -1]
```

```
print(v1 / v2)
```

```
[-1. -1. -1. -1. -1. -1. -1. -1. -1. -1.]
```

Scalar operations applied pointwise

```
print(v1)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

```
print(v1*3)
```

```
[ 3  6  9 12 15 18 21 24 27 30]
```

```
print(v1)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

This is an example of what is called **broadcasting**.

- Broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations.

- Subject to certain constraints, the smaller array is “broadcast” across the larger array so that they have compatible shapes.

```
v1 + 2, v2-3, v1 ** 4
```

```
(array([ 3,  4,  5,  6,  7,  8,  9, 10, 11, 12]),
 array([ -4,  -5,  -6,  -7,  -8,  -9, -10, -11, -12, -13]),
 array([    1,    16,    81,   256,   625,  1296,  2401,  4096,  6561,
        10000]))
```

Matrix Operations

```
M = np.array([[1,2],[3,4]])
N = np.array([[5,6],[7,8]])
```

```
M
```

```
array([[1, 2],
       [3, 4]])
```

```
N
```

```
array([[5, 6],
       [7, 8]])
```

Broadcasting in matrices

```
3 * M, 3 + M, M - 5, M ** 2
```

```
(array([[ 3,  6],
        [ 9, 12]]),
 array([[4, 5],
        [6, 7]]),
 array([[-4, -3],
        [-2, -1]]),
 array([[ 1,  4],
        [ 9, 16]]))
```

Pointwise addition, multiplication, subtraction, division

```
M + N, M * N, M - N, M // N
```

```
(array([[ 6,  8],
        [10, 12]]),
 array([[ 5, 12],
        [21, 32]]),
 array([[-4, -4],
        [-4, -4]]),
 array([[0, 0],
        [0, 0]]))
```

Matrix multiplication

```
np.matmul(M,N)
```

```
array([[19, 22],
       [43, 50]])
```

```
np.dot(M,N)
```

```
array([[19, 22],
       [43, 50]])
```

For 2-D arrays, np.matmul, np.dot, and @ all perform the same matrix multiplication.

```
M @ N
```

```
np.matmul(N,M)
```

`dot` works differently from `matmul` and `@` on higher dimensional arrays

Transpose and inverse

```
M.T
```

```
array([[1, 3],
       [2, 4]])
```

```
np.linalg.inv(M)
```

```
array([[-2. ,  1. ],
       [ 1.5, -0.5]])
```

MM^{-1} should be the identity matrix, however, there could be small imprecision due to round off error

```
np.matmul(M,np.linalg.inv(M))
```

```
array([[1.0000000e+00, 0.0000000e+00],
       [8.8817842e-16, 1.0000000e+00]])
```

We can aggregate matrices the same way we aggregated vectors using `max`, `min`, `sum`, etc. We can also aggregate across the rows or columns by using the `axis` parameter.

```
print(M)
```

```
[[1 2]
 [3 4]]
```

```
print(M.max(axis=0)) #Get max in each col
```

```
[3 4]
```

```
print(M.max(axis=1)) #Get max in each row
```

```
[2 4]
```

Higher Dimensional Arrays

A multidimensional array is a data structure that can store data in multiple dimensions.

1D arrays and 2D arrays

```
v1 = np.array([-1,2,4,5,1])
v1
```

```
array([-1,  2,  4,  5,  1])
```

```
v1.ndim, v1.shape, v1.size
```

```
(1, (5,), 5)
```

```
v2 = np.ones(5)
v2
```

```
array([1., 1., 1., 1., 1.])
```

```
v3 = np.arange(5,15,2)
v3
```

```
array([ 5,  7,  9, 11, 13])
```

```
v3 = np.random.randint(1,5,3)
v3
```

```
array([4, 2, 2])
```

```
v1+v2, v1-v2, v1*v2, v1//v2, v1/v2
```

```
(array([0., 3., 5., 6., 2.]),
 array([-2.,  1.,  3.,  4.,  0.]),
 array([-1.,  2.,  4.,  5.,  1.]),
 array([-1.,  2.,  4.,  5.,  1.]),
 array([-1.,  2.,  4.,  5.,  1.]))
```

```
5*v1, v1**3, 3+v2, v1**2, v2-2, v1//3
```

```
(array([-5, 10, 20, 25,  5]),
 array([-5, 10, 20, 25,  5]),
 array([4., 4., 4., 4., 4.]),
 array([ 1,  4, 16, 25,  1]),
 array([-1., -1., -1., -1., -1.]),
 array([-1,  0,  1,  1,  0]))
```

```
M1 = np.array([[1,2],[3,4],[5,6]])
M1
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
M1.ndim, M1.shape, M1.size
```

```
(2, (3, 2), 6)
```

```
M2 = np.ones((2,5))
M2
```

```
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

```
M2.ndim, M2.shape, M2.size
```

```
(2, (2, 5), 10)
```

An exmaple of a 3D array.

```
M3 = np.random.randint(1,5,(2,5,4))
M3
```

```
array([[[2, 1, 1, 3],
        [1, 1, 1, 1],
        [3, 2, 1, 4],
        [3, 4, 1, 3],
        [3, 4, 4, 4]],

       [[2, 1, 4, 3],
        [3, 1, 2, 4],
        [4, 3, 3, 2],
        [2, 1, 3, 3],
        [4, 1, 3, 2]]])
```

```
M3.ndim, M3.shape, M3.size
```

```
(3, (2, 5, 4), 40)
```

```
M4 = np.random.randint(1,5,(3,3))
M4
```

```
array([[3, 1, 4],
       [4, 1, 4],
       [3, 3, 4]])
```

```
M5 = np.random.randint(1,5,(2,5))
M5
```

```
array([[2, 4, 3, 3, 1],
       [2, 3, 3, 2, 2]])
```

```
M5+M2, M5-M2, M5*M2, M1@M5, M1.T, np.linalg.inv(M4)
```

```
(array([[3., 5., 4., 4., 2.],
        [3., 4., 4., 3., 3.]]),
 array([[1., 3., 2., 2., 0.],
        [1., 2., 2., 1., 1.]]),
 array([[2., 4., 3., 3., 1.],
        [2., 3., 3., 2., 2.]]),
 array([[ 6, 10,  9,  7,  5],
        [14, 24, 21, 17, 11],
        [22, 38, 33, 27, 17]]),
 array([[1, 3, 5],
        [2, 4, 6]]),
 array([[-1.   ,  1.   ,  0.   ],
        [-0.5  ,  0.   ,  0.5  ],
        [ 1.125, -0.75 , -0.125]]))
```

```
M1*2, M2-3, M3+5, M2**2
```

```
(array([[ 2,  4],
        [ 6,  8],
        [10, 12]]),
 array([[-2., -2., -2., -2., -2.],
        [-2., -2., -2., -2., -2.]]),
 array([[[7, 6, 6, 8],
         [6, 6, 6, 6],
         [8, 7, 6, 9],
         [8, 9, 6, 8],
         [8, 9, 9, 9]],

        [[7, 6, 9, 8],
         [8, 6, 7, 9],
         [9, 8, 8, 7],
         [7, 6, 8, 8],
         [9, 6, 8, 7]]]),
 array([[1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1.]]))
```

```
M1
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
M1.reshape(2,3)
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
M1
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

We can use nested sequences to produce higher dimensional arrays. Think of a 2D array as an array of 1D arrays and a 3D array as an array of 2D arrays and so on.

```
arr1 = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(arr1)
```

```
[[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

```
arr1.ndim, arr1.shape, arr1.size
```

```
(3, (2, 2, 3), 12)
```

```
arr2 = np.random.randint(0,10,(2,3,12))
arr2
```

```
array([[[4, 4, 9, 0, 3, 1, 5, 2, 0, 0, 7, 7],
        [2, 1, 0, 1, 2, 8, 5, 8, 1, 1, 2, 2],
        [2, 3, 8, 5, 5, 5, 2, 4, 0, 3, 6, 9]],

       [[4, 6, 6, 3, 7, 7, 7, 6, 7, 8, 6, 4],
        [6, 6, 8, 9, 4, 2, 2, 9, 4, 7, 7, 8],
        [0, 5, 6, 2, 2, 7, 6, 0, 4, 5, 3, 9]]])
```

```
arr2.ndim, arr2.shape, arr2.size
```

```
(3, (2, 3, 12), 72)
```

```
arr3 = np.array([[(0,1,0),[2,3,2]],[[4,5,4],[6,7,6]]])
print(arr3)
```

```
[[[0 1 0]
  [2 3 2]]

 [[4 5 4]
  [6 7 6]]]
```

```
arr3.ndim, arr3.shape, arr3.size
```

```
(3, (2, 2, 3), 12)
```

```
arr4 = np.ones((4,2,3))
print(arr4)
```

```
[[[1. 1. 1.]
  [1. 1. 1.]]

 [[1. 1. 1.]
  [1. 1. 1.]]

 [[1. 1. 1.]
  [1. 1. 1.]]

 [[1. 1. 1.]
  [1. 1. 1.]]]
```

```
arr4.ndim, arr4.shape, arr4.size
```

```
(3, (4, 2, 3), 24)
```

```
arr5 = np.array([1, 2, 3, 4], ndmin=5)

print(arr5)
```

```
[[[[[1 2 3 4]]]]]
```

```
arr5.ndim, arr5.shape, arr5.size
```

```
(5, (1, 1, 1, 1, 4), 4)
```

In this array, the innermost dimension (5th dim) has 4 elements, the 4th dim has 1 element that is the vector, the 3rd dim has 1 element that is the matrix with one row, the 2nd dim has 1 element that is 3D array and 1st dim has 1 element that is a 4D array.

Indexing and Slicing

Single element indexing works exactly like that for other standard Python sequences. Indices start from 0 and negative indices are for indexing from the end of the array.

```
x = np.arange(36)
print(x[3], x[-3])
```

```
3 33
```

Multidimensional arrays can have one index per axis. These indices are given in a tuple separated by commas

```
x.shape = (2, 5)  # now x is 2-dimensional
print(x)
print(x[1, 3], x[1, -1])
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1082/3259290975.py in <cell line: 0>()
----> 1 x.shape = (2, 5)  # now x is 2-dimensional
      2 print(x)
      3 print(x[1, 3], x[1, -1])

ValueError: cannot reshape array of size 36 into shape (2,5)
```

```
x.shape = (6, 6)  # now x is 2-dimensional
print(x)
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]
 [18 19 20 21 22 23]
 [24 25 26 27 28 29]
 [30 31 32 33 34 35]]
```

```
print(x[[1,2,4]])
```

```
[[ 6  7  8  9 10 11]
 [12 13 14 15 16 17]
 [24 25 26 27 28 29]]
```

If one indexes a multidimensional array with fewer indices than dimensions, one gets a subdimensional array

```
print(x[0])
```

```
[0 1 2 3 4]
```

Matrix indexing and slicing

```
M = np.array(([1,2,3,4,5,6],[3,4,5,6,7,8],[5,6,7,8,9,10]))
M
```

```
array([[ 1,  2,  3,  4,  5,  6],
       [ 3,  4,  5,  6,  7,  8],
       [ 5,  6,  7,  8,  9, 10]])
```

To slice a 2D array, you have to specify row index and column index which are separated using a comma.

In general, the syntax for each dimension is `[start:end:step]`

- `start` is by default 0
- `end` is by default the length of array in that dimension
- `step` is by default 1

```
print(f"M={M}")
print(f"\nM[0,1]={M[0,1]}") #(0,1)th entry
print(f"\nM[1:3]={M[1:3]}") # Row 1 and Row 2
print(f"\nM[0:2,0]={M[0:2,0]}") # Row 0, Row 1, Col 0
print(f"\nM[1,1:4]={M[1,1:4]}") # Row 1, Cols 1 to 3
print(f"\nM[0:2,0:2]={M[0:2,0:2]}") # Row 0, Row 1, Col 0, Col 1
```

```
M=[[1 2]
 [3 4]]

M[0,1]=2

M[1:3]=[[3 4]]

M[0:2,0]=[1 3]

M[1,1:4]=[4]

M[0:2,0:2]=[[1 2]
 [3 4]]
```

Use the `-` operator to refer to an index from the end.

```
print(f"M={M}")
print(f"\nM[0:2,-1:-2:-1]={M[0:2,-1:-3:-1]}") # Row 0, Row 1, Col n-1, Col n-2 where n is the no. of cols
```

```
M=[[1 2]
 [3 4]]

M[0:2,-1:-2:-1]=[[2 1]
 [4 3]]
```

```
print(f"M={M}")
print(f"\nM[::,::-1]={M[::,::-1]}") #Flip horizontal
print(f"\nM[::-1,::]={M[::-1,::]}") #Flip vertical
print(f"\nM[::-1, ::-1]={M[::-1, ::-1]}") #Rotate 180 deg
```

```
M=[[ 1  2  3  4  5  6]
 [ 3  4  5  6  7  8]
 [ 5  6  7  8  9 10]]

M[::,::-1]=[[ 6  5  4  3  2  1]
 [ 8  7  6  5  4  3]
 [10  9  8  7  6  5]]

M[::-1,::]=[[ 5  6  7  8  9 10]
 [ 3  4  5  6  7  8]
 [ 1  2  3  4  5  6]]

M[::-1, ::-1]=[[10  9  8  7  6  5]
 [ 8  7  6  5  4  3]
 [ 6  5  4  3  2  1]]
```

NumPy in Action

Basic Image Processing

A **pixel** (picture element) is the smallest, fundamental unit of a digital display, appearing as a tiny dot or square.

Millions of pixels, each assigned specific color and brightness values, are arranged in a grid to form images, videos and text on digital screens.

A digital image is just a **matrix of numbers**. The dimension of this matrix is called the image's **resolution**.

An image file format is a file format for a digital image.

The **PGM (Portable Graymap)** format is a simple, portable and easily parsable file format for grayscale digital images.

- It represents an image as a grid of pixel values ranging from black (represented by 0) to white
- Pixel values stored in plaintext (P2) or binary (P5) formats

A typical P2 PGM file has the following format.

```
P2
# optional comment
width height
max_gray_value
pixel data
```

P2 identifies the file as a grayscale image stored as text and the image size is given by width and height in pixels.

max_gray_value indicates that the pixel values range from 0 to max_gray_value. Typically, max_gray_value is 255 (8-bit grayscale) or 65535(16-bit grayscale).

The pixel data are numbers representing pixel intensities (brightness) -- starting from the top-left corner, moving in **raster order** (similar to reading a book).

```
img = [ [255, 0, 255, 0],
        [0, 255, 0, 255],
        [100, 200, 150, 100],
        [50, 150, 200, 0]]
```

We can interpret this array as an image in gray scale by treating the entries as pixel data. Let us display the image corresponding to these pixel values using `matplotlib`. Matplotlib is an open-source plotting library for creating static, animated, and interactive visualizations.

```
!pip3 install matplotlib
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (
```

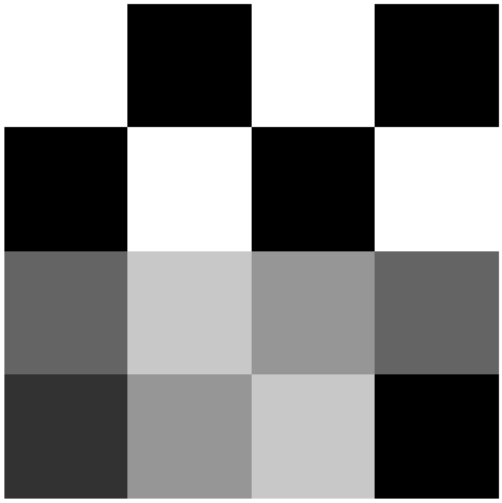
Let us import the plotting module from matplotlib

```
import matplotlib.pyplot as plt
```

The `imshow` function is used to display an array as an image using a color map. A color map is a rule that tells how numerical values in an array are converted into colors when displaying an image.

When an image is displayed, the pixel values in the image array are mapped onto the figure area with the pixels scaled appropriately (i.e., the size of a pixel depends on the figure size).

```
plt.imshow(img, cmap="gray")
plt.axis("off")
plt.show()
```

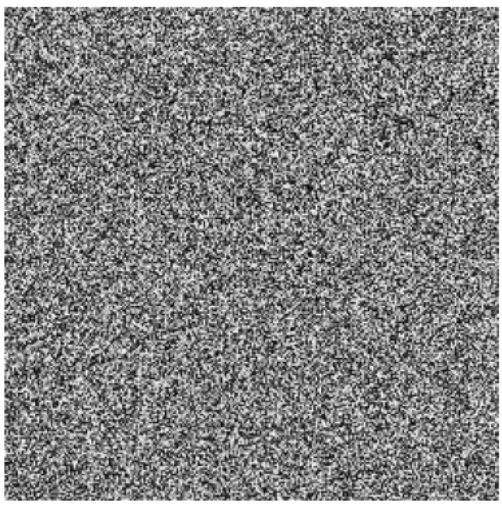


Here's a random image.

```
img = np.random.randint(0,256,(256,256))
img
```

```
array([[137, 146,  42, ..., 205,  72, 159],
       [173,  24, 183, ..., 214,  51,  82],
       [120, 118,  81, ..., 185, 156, 152],
       ...,
       [166, 242, 173, ...,  76, 157, 254],
       [205, 159,  29, ..., 229,  85, 121],
       [240, 128, 146, ...,  12,  51, 136]])
```

```
plt.imshow(img, cmap="gray")
plt.axis("off")
plt.show()
```



Let us now perform some basic image processing begining with reading a pgm file and storing its pixel data as an array.

```
def read_pgm(filename):
    with open(filename, 'r') as f:
        assert f.readline().strip() == 'P2'

        line = f.readline()
        while line.startswith('#'):
            line = f.readline()

        width, height = map(int, line.split())
        maxval = int(f.readline())

        pixels = []
        for line in f:
            pixels.extend(map(int, line.split()))

        img = np.array(pixels).reshape((height, width))
        return img, maxval
```

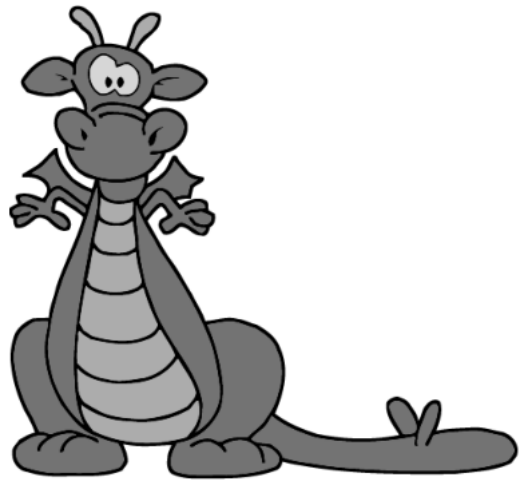
```
img,max = read_pgm("dragon.pgm")
#image src: https://people.sc.fsu.edu/~jburkardt/data/pgma/pgma.html
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_1059/53217997.py in <cell line: 0>()
----> 1 img,max = read_pgm("dragon.pgm")
      2 #image src: https://people.sc.fsu.edu/~jburkardt/data/pgma/pgma.html

/tmp/ipykernel_1059/2444782022.py in read_pgm(filename)
      1 def read_pgm(filename):
----> 2     with open(filename, 'r') as f:
      3         assert f.readline().strip() == 'P2'
      4
      5         line = f.readline()

FileNotFoundError: [Errno 2] No such file or directory: 'dragon.pgm'
```

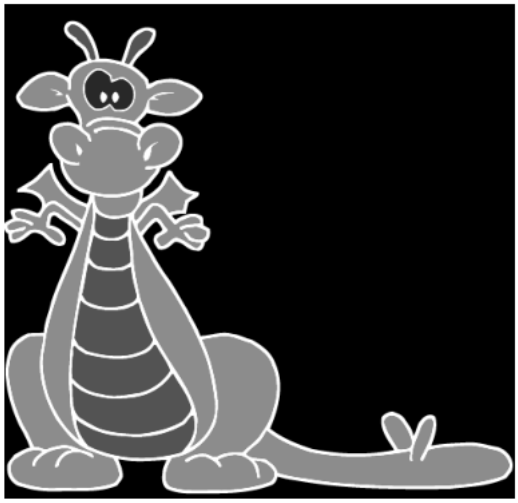
```
img.shape
```

```
(460, 475)
```

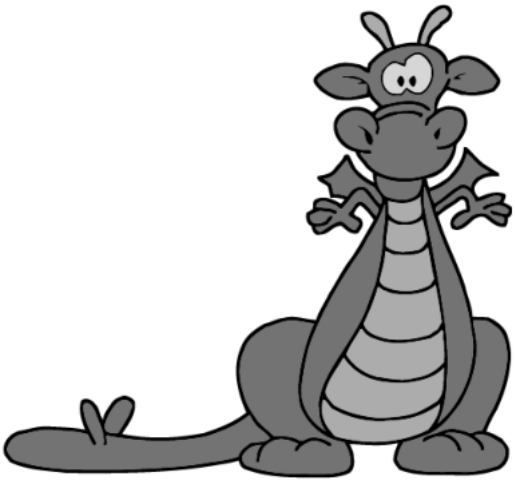
```
plt.imshow(img, cmap="gray")
plt.axis("off")
plt.show()
```



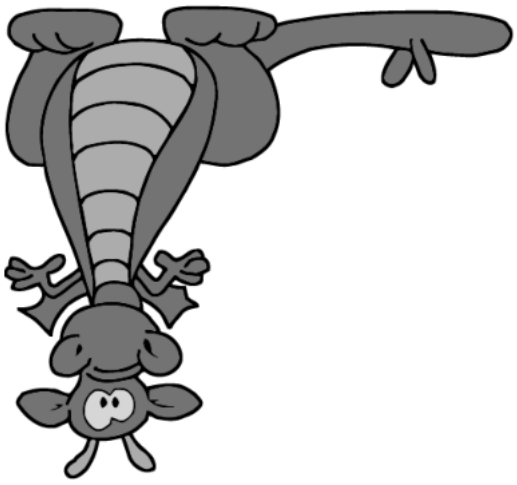
```
# negative image (invert brightness)
neg = max - img

plt.imshow(neg, cmap="gray")
plt.axis("off")
plt.show()
```



```
# horizontal flip (Mirror left-right)
flip_h = img[:, ::-1]

plt.imshow(flip_h, cmap="gray")
plt.axis("off")
plt.show()
```

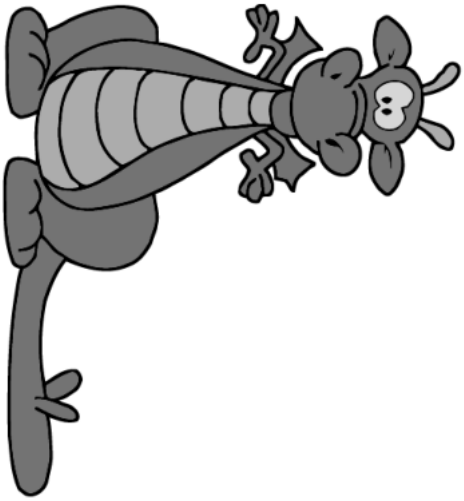


```
# vertical flip (Mirror up-down)
flip_v = img[::-1, :]

plt.imshow(flip_v, cmap="gray")
plt.axis("off")
plt.show()
```

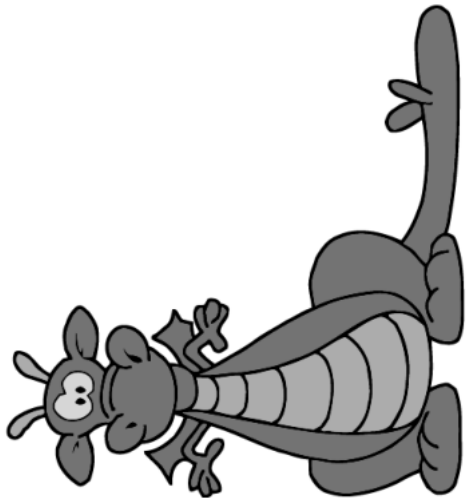


```
# Diagonal flip
plt.imshow(img.T, cmap="gray")
plt.axis("off")
plt.show()
```

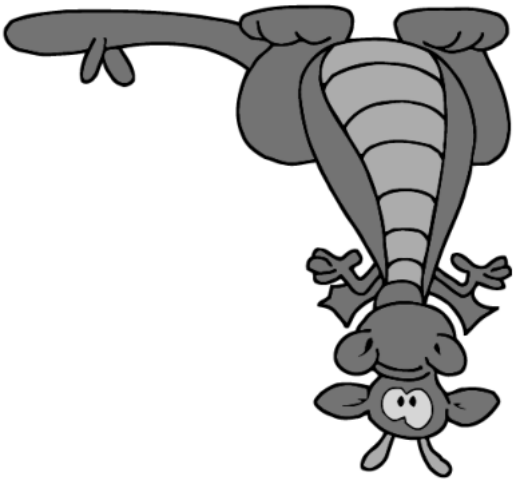
```
# Clockwise rotation 90 deg
plt.imshow(img.T[:, ::-1], cmap="gray")
plt.axis("off")
plt.show()
```



```
#Anticlockwise rotation 90 deg
plt.imshow(img.T[::-1, :], cmap="gray")
plt.axis("off")
plt.show()
```



```
# rotate 180 deg
plt.imshow(img[::-1, ::-1], cmap="gray")
plt.axis("off")
plt.show()
```



ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Classes, Objects, Attributes, Methods
    - Defining new types

Reference: Think Python (Edition 3) by Allen B. Downey

Programming Paradigms

A programming paradigm is a high-level way to conceptualize and structure a program. Two common programming paradigms are

- Procedural programming – a program is organized as procedures (functions) that call each other
- Object-oriented programming (OOP) – a program consists of objects that interact with one another
    - An object is an entity that has a state data (**attributes**), behavior (**methods**) and identity
    - Objects represent things in the real world and methods correspond to the ways these things interact
    - Most of the computation is expressed in terms of operations on objects

Creating a Point Type and Instantiating Point Objects

A location on the screen is often represented as a point on a 2-dimensional plane. A point in the plane is typically associated with two numbers representing its `x` and `y` coordinates. There are several ways to represent a point.

- Store the coordinates in two variables `x` and `y`.
- Store the coordinates as elements in a list or tuple.
- Create a new type to represent points as objects that contain the attributes `x` and `y`.

Let us take the third approach and start with creating a type called `Point`.

A programmer-defined type is also called a **class**. A new type can be defined by a template using the keyword `class`. Here's how a class definition for `Point` looks

like.

```
class Point:
    """Represents a point in 2D space."""
```

Once a class (new type) is defined, independent instances of this class (**objects**) can be created. Creating a new object is called **instantiation**, and the object is an **instance** of the class.

```
p1 = Point()
p2 = Point()
```

```
print(type(p1))
```

```
<class '__main__.Point'>
```

The new objects' type is `__main__.Point`, where `__main__` is the name of the module where `Point` is defined. When Python runs a program, it loads the code as a module. The module that is executed directly (and not via `import`) gets the special name: `__main__`.

We can use `isinstance` to check whether an object is an instance of a particular class.

```
isinstance(p2, Point)
```

```
True
```

A docstring is a string written using triple quotes (`' ' '` or `" " "`) that appears as the first statement inside a class.

A docstring explains what the class is meant to represent. They are stored as metadata, accessible at runtime and used by help and documentation tools.

```
help(Point)
```

```
Help on class Point in module __main__:

class Point(builtins.object)
 |  Represents a point in 2D space.
 |
 |  Data descriptors defined here:
 |
 |  __dict__
 |      dictionary for instance variables (if defined)
 |
```

```
 |  __weakref__
 |      list of weak references to the object (if defined)
```

We intended a `Point` object to represent a point on the plane.

```
p1.x = 7
p1.y = 6
```

Each object has its own internal state -- the values of its **attributes** (think of attributes as local variables)

```
p2.x = -10
p2.y = -5
```

Not all objects need to have the same set of attributes.

```
p3 = Point()
p3.z = -17
```

When you print an object, Python tells you what type it is and where it is stored in memory (the prefix `0x` means that the following number is in hexadecimal). We can customize how an object is shown when printed.

```
print(p1,p2,p3)
```

```
<__main__.Point object at 0x107777c40> <__main__.Point object at 0x107777190>
```

We defined the datatype `Point` to represent a point in the 2D plane. Since a `Point` object is meant to represent a point, it seems reasonable to expect that it should have attributes denoting its coordinates. This can be forced as part of the class definition.

- How should an instance of `Point` look like?
- A `Point` object should ideally contain variables (attributes) that can store the coordinates (state of the object).

```
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, a, b):
        self.x = a
        self.y = b
    # self refers to the object being created
```

```
        # The Point object p has attributes p.x and p.y that are initialized to

        def __str__(self):
            return f'Point({self.x}, {self.y})'
        # self refers to the object whose str repr we are trying to obtain
```

```
p4 = Point(2,3)
```

```
print(p4)
```

```
Point(2, 3)
```

```
p4.x = 10
p4.y = -100
```

```
print(p4)
```

```
Point(10, -100)
```

Special Functions Associated with a Class

- `__init__` is a special function (called **constructor**) that is called automatically when you create a new object of the class.
  - `self` refers to the object being created.
  - The name `self` for the current object (first parameter) is only a convention and any other name can be used.
- `__init__` initializes the attributes of a new object.
- `__init__` takes the coordinates as parameters and assigns them to attributes (instance variables) `x` and `y` of the object being created.
- `__str__` is a special method that specifies how an object is shown as a string.
  - This method is automatically invoked by the `print` function.
  - If you use the built-in function `str` to convert a `Point` object to a string, Python uses the `__str__` method in the `Point` class.

When we instantiate a `Point` object, Python invokes `__init__`, and passes along the arguments. So we can create an object and initialize the attributes at the same time.

```
p5 = Point()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 1
----> 1 p5 = Point()

TypeError: __init__() missing 2 required positional arguments: 'a' and 'b'
```

```
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b

    def __str__(self):
        return f'Point({self.x}, {self.y})'
```

```
p5 = Point()
```

```
print(p5)
```

```
Point(0, 0)
```

```
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'
```

In this example, the parameters are optional, so if you call `Point` with no arguments, you get the default values.

```
p = Point()
```

If you provide one argument, it overrides the corresponding default value.

```
q = Point(x=1)
```

If you provide two arguments, they override both default values.

```
r = Point(1,5)
```

```
r1 = Point(y=6,x=1)
```

It is a good practice to define `__init__` and `__str__` while defining a class.

```
p = Point()
q = Point(1,5)
r = Point(y=6,x=1)
print(p)
print(q)
print(r)
```

```
Point(0, 0)
Point(1, 5)
Point(1, 6)
```

We can add new attributes to objects.

```
r.z = 10
```

```
print(r)
```

```
Point(1, 6)
```

```
str(r)
```

```
'Point(1, 6)'
```

We can read the value of an attribute using the dot operator and use an attribute as part of any expression.

- Each object has its own internal state -- the values of its local variables (attributes).

Objects as Arguments and Return Values

We can pass an object as an argument in a function call.

Example 1: Here's a function to calculate distance between 2 points.

```
import math
def dist(p1, p2):
    dx = (p1.x - p2.x)
    dy = (p1.y - p2.y)
    dsq = dx**2 + dy**2
    return math.sqrt(dsq)
```

```
u = Point(1,5)
v = Point(-1,1)
dist(u,v)
```

```
4.47213595499958
```

Example 2: Here's a function that returns a `Point` object.

```
def scale(p,c):
    q = Point(c*p.x,c*p.y)
    return q
```

```
a = Point(45,66)
print(scale(a,-6))
```

```
Point(-270, -396)
```

Example 3: Here's another function that changes a `Point` object.

```
def reset(p):
    p.x = 0
    p.y = 0
```

```
w = Point(1,5)
print(w)
reset(w)
print(w)
```

```
Point(1, 5)
Point(0, 0)
```

By default, instances of programmer-defined classes are **mutable**.

- Can add or modify attributes after the object is created.

However, it is possible to define custom classes so that the objects are **immutable**.

Example 4: Suppose you want to calculate the distance of a point from origin.

```
import math
def dist_to_origin(p):
    dx = (p.x)
    dy = (p.y)
    dsq = dx**2 + dy**2
    return math.sqrt(dsq)
```

```
z = Point(1,5)
dist_to_origin(z)
```

```
5.0990195135927845
```

These examples describe functions that **act** on `Point` object(s). We may also think of these functions as describing the **behavior** of an object.

- A point can compute its distance from origin
- A point can compute its distance from another point
- A point can scale itself by a number or compute a point that is a scaled version of it
- A point can reset itself

We can rewrite such behaviour functions as **methods** defined inside the class definition.

Defining Methods

Methods are functions defined within a class and can be called on objects of that class. Think of the following methods as **acting** on `Point` object(s).

What are all the operations that we would want on `Point` objects?

A class definition typically specifies data and procedures associated with an object of this class.

- data attributes (or simply attributes) -- data that make up an object of this class
- procedural attributes (methods) -- specify how to interact with an object of this class

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0
    # self is a reference to the object of the class on which the method is

    def scale(self,c):
```

```
        self.x *= c
        self.y *= c

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)
```

Methods are associated with a particular class.

- Methods define the behavior of an object and actions that an object can perform.
- A method can take arguments, including a reference to the object on which it is called.
    - This allows the method to access and modify the object's attributes.
- In the definition of methods above, `self` is a reference to the object on which the method is being invoked.
- The name `self` for the current object (first parameter in a method) is only a convention - we can use any other name.

When we call a method on an object, reference to this object gets automatically passed to the method.

```
z = Point(1,7)
o = Point(0,0)
print(z.dist(o))
# When we call a method on an object, reference to this object gets automati
y = Point(-1,-7)
print(z.dist(y))
print(y.dist(z))
```

```
7.0710678118654755
14.142135623730951
14.142135623730951
```

All objects of a class share the same methods to query/update their state.

- Think of methods as functions being invoked on an object of this class.

Static Methods

A **static method** is a method that does not require an instance of the class to be invoked. It behaves like a normal function but is logically grouped inside the class.

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x *= c
        self.y *= c

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)
```

`collinear` describes a relationship between 3 point objects and is not tied to a single object.

```
p1 = Point(1,1)
p2 = Point(2,2)
p3 = Point(3,3)
print(Point.collinear(p1,p2,p3))
```

```
True
```

It is possible to define `collinear` as an instance method too.

```
import math
class Point:
    """Represents a point in 2-D space."""
```

```
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x *= c
        self.y *= c

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    def collinear(self, p2, p3):
        return (p2.y - self.y)*(p3.x - self.x) == (p3.y - self.y)*(p2.x - se
```

```
q1 = Point(1,1)
q2 = Point(2,2)
q3 = Point(3,3)
print(q1.collinear(q2,q3))
```

```
True
```

Operator Overloading

Consider the following code snippet.

```
p = Point(-1,8)
q = Point(6,-4)
print(p+q)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[44], line 3
      1 p = Point(-1,8)
      2 q = Point(6,-4)
----> 3 print(p+q)

TypeError: unsupported operand type(s) for +: 'Point' and 'Point'
```

For every operator, there is a corresponding special method. Examples: `+` is mapped to `__add__`, `<` is mapped to `__lt__`, etc.

`+` implicitly calls `__add__()` and `p + q` gets translated as `p.__add__(q)`

For built-in types (`int`, `str`, etc.), the method `__add__()` is defined inside the implementation of the type.

Let us define `__add__()` inside `Point` so that it returns a new `Point` without modifying its arguments.

After defining this method for this class, we can use the `+` operator on `Point` objects.

```
import math

class Point:

    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x = c * self.x
        self.y = c * self.y

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)
```

```
    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)

    def __add__(self,p):
        return(Point(self.x + p.x, self.y + p.y))

    def __sub__(self,p):
        return(Point(self.x - p.x, self.y - p.y))

    def __mul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __matmul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __lt__(self,p):
        return(self.x < p.x and self.y < p.y)

    def __gt__(self,p):
        return(self.x > p.x and self.y > p.y)

    def __le__(self,p):
        return(self.x <= p.x and self.y <= p.y)

    def __ge__(self,p):
        return(self.x >= p.x and self.y >= p.y)
```

`__add__()` is executed in the context of one point and the other point is passed to it as an argument.

By defining other special methods, we can specify the behavior of other operators on our new type.

```
a = Point(-1,8)
b = Point(6,-4)
print(a+b)
print(a-b)
print(a*b)
print(a@b)
```

```
Point(5, 4)
Point(-7, 12)
Point(-6, -32)
Point(-6, -32)
```

Changing the behavior of an operator so that it works with programmer-defined types is called **operator overloading**.

```
p = Point(3,4)
q = Point(7,10)
p < q, q < p, q > p
```

```
(True, False, True)
```

```
p <= q, q <= p, q >= p
```

```
(True, False, True)
```

```
p1 = Point(1,1)
p2 = Point(1,1)
p1 == p2
```

```
False
```

You might expect `==` to yield `True` because the objects contain the same data. But for programmer-defined classes, the default behavior of the `==` operator is the same as the `is` operator -- it checks identity, not equivalence.

If we want to change this behavior, we can provide a special method called `__eq__` that defines what it means for two `Point` objects to be equal.

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x = c * self.x
        self.y = c * self.y

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
```

```
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)

    def __add__(self,p):
        return(Point(self.x + p.x, self.y + p.y))

    def __sub__(self,p):
        return(Point(self.x - p.x, self.y - p.y))

    def __mul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __matmul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __lt__(self,p):
        return(self.x < p.x and self.y < p.y)

    def __gt__(self,p):
        return(self.x > p.x and self.y > p.y)

    def __le__(self,p):
        return(self.x <= p.x and self.y <= p.y)

    def __ge__(self,p):
        return(self.x >= p.x and self.y >= p.y)

    def __eq__(self, other):
        return (self.x == other.x) and (self.y == other.y)

    def __ne__(self,p):
        return(not(self == p))
```

This definition considers two `Points` to be equal if their attributes are equal. Now when we use the `==` operator, it invokes the `__eq__` method, which indicates that `p1` and `p2` are considered equal.

```
p1 = Point(1,1)
p2 = Point(1,1)
p1 == p2
```

```
True
```

```
p1 != p2
```

```
False
```

But the `is` operator still indicates that they are different objects.

```
p1 is p2
```

```
False
```

It's not possible to change the behaviour of the `is` operator -- it always checks whether the objects are identical.

Copying Objects

Assignment statements do not copy objects, they create bindings between a target and an object. For collections that are mutable or contain mutable items, a copy is sometimes needed so one can change one copy without changing the other.

The `copy` module provides a function called `copy` that can duplicate any object.

```
import copy
```

To see how it works, let's start with a new `Point` object.

```
start = Point(10, 20)
```

And make a copy.

```
end = copy.copy(start)
```

Now `start` and `end` contain the same data.

```
print(start)
print(end)
```

```
Point(10, 20)
Point(10, 20)
```

However, the `is` operator confirms that they are not the same object.

```
start is end
```

```
False
```

Changing attributes of one copy does not affect the other.

```
start.x = 5
print(start)
print(end)
```

```
Point(5, 20)
Point(10, 20)
```

Programmer-defined Objects as Attributes

Let's define a type `Line` that has `Point` objects as attributes.

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x = c * self.x
        self.y = c * self.y

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)

    def __add__(self,p):
        return(Point(self.x + p.x, self.y + p.y))

    def __sub__(self,p):
        return(Point(self.x - p.x, self.y - p.y))

    def __mul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __matmul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __lt__(self,p):
```

```
        return(self.x < p.x and self.y < p.y)

    def __gt__(self,p):
        return(self.x > p.x and self.y > p.y)

    def __le__(self,p):
        return(self.x <= p.x and self.y <= p.y)

    def __ge__(self,p):
        return(self.x >= p.x and self.y >= p.y)

    def __eq__(self, other):
        return (self.x == other.x) and (self.y == other.y)

    def __ne__(self,p):
        return(not(self == p))

    def translate(self,dx,dy):
        self.x += dx
        self.y += dy

    def translated(self, dx=0, dy=0):
        point = copy.copy(self)
        point.translate(dx, dy)
        return point
```

```
class Line:
    def __init__(self, q1, q2):
        self.p1 = q1
        self.p2 = q2

    def __str__(self):
        return f"Line: {self.p1}, {self.p2}"
```

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
print(L)
```

```
Line: Point(1, 2), Point(4, 6)
```

Can we uniquely define a line using only one point? How do we obtain the equation of a line?

```
class Line:
    def __init__(self, p1, p2):
        if p1==p2:
            raise ValueError("Two distinct points are required to uniquely d

        self.p1 = p1
        self.p2 = p2
```

```
        # m is a derived attribute
        if p1.x == p2.x:
            self.m = None
        else:
            self.m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.m is not None:
            return round(self.m,2)
        return None

    def isVertical(self):
        return self.m is None

    def __str__(self):
        if self.m is None:
            return f"Line: x = {self.p1.x}"
        return f"Line: y - {self.p1.y} = {self.m:.2f}(x - {self.p1.x})"
```

Now we can display the line in point-slope form.

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
print(L)
print(f"Slope: {L.getSlope()}")
```

```
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
```

```
L1= Line(Point(2,1), Point(2,5))
print(L1)
print(f"Slope: {L1.getSlope()}")
```

```
Line: x = 2
Slope: None
```

Consider the following snippet.

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
print(L)
L.p1 = Point(0,0)
print(L)
```

```
Line: y - 2 = 1.33(x - 1)
Line: y - 0 = 1.33(x - 0)
```

The equation of the line is incorrect. The issue seems to be that we are able to change an attribute outside the class definition. So let us restrict access to points associated with lines.

```
class Line:
    def __init__(self, p1, p2):
        if p1==p2:
            raise ValueError("Two distinct points are required to uniquely d

        self.__p1 = p1
        self.__p2 = p2

        if p1.x == p2.x:
            self.__m = None
        else:
            self.__m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.__m is not None:
            return round(self.__m,2)
        return None

    def isVertical(self):
        return self.__m is None

    def __str__(self):
        if self.__m is None:
            return f"Line: x = {self.__p1.x}"
        return f"Line: y - {self.__p1.y} = {self.__m:.2f}(x - {self.__p1.x})
```

A double underscore `__` before a variable or method triggers **name mangling** which renames the attribute making it harder to access from outside the class.

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
print(L)
print(L.__p1)
```

```
Line: y - 2 = 1.33(x - 1)
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[68], line 6
      4 L = Line(u, v)
      5 print(L)
----> 6 print(L.__p1)

AttributeError: 'Line' object has no attribute '__p1'
```

Name mangling essentially hides the attribute. Python interpreter changes the name of the attribute internally. The new name can be used to access the attribute -- however this is discouraged.

```
print(L._Line__p1)
```

```
Point(1, 2)
```

Consider the following snippet.

```
u = Point(1, 2)
v = Point(4, 6)
L = Line(u, v)
print(L)
print(f"Slope: {L.getSlope()}")
u.reset()
print(L)
print(f"Slope: {L.getSlope()}")
```

```
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
Line: y - 0 = 1.33(x - 0)
Slope: 1.33
```

The slope attribute no longer denotes the slope! Let us redefine `Line` that prevents such problems.

```
class Line:
    def __init__(self, p1, p2):
        if p1.x == p2.x and p1.y == p2.y:
            raise ValueError("Two distinct points are required to uniquely d

        self.__p1 = Point(p1.x, p1.y)
        self.__p2 = Point(p2.x, p2.y)

        if p1.x == p2.x:
            self.__m = None
        else:
            self.__m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.__m is not None:
            return round(self.__m,2)
        return None

    def isVertical(self):
        return self.__m is None

    def __str__(self):
        if self.__m is None:
```

```
            return f"Line: x = {self.__p1.x}"
        return f"Line: y - {self.__p1.y} = {self.__m:.2f}(x - {self.__p1.x})
```

```
u = Point(1, 2)
v = Point(4, 6)
L = Line(u, v)
print(L)
print(f"Slope: {L.getSlope()}")
u.reset()
print(u)
print(L)
print(f"Slope: {L.getSlope()}")
```

```
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
Point(0, 0)
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
```

This is the beginning of building a mini geometry library. A geometry library is a collection of classes and functions that model geometric objects and operations. What geometric objects would you add? When you define these objects, make sure

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Programmer-defined Types
  - Defining new type (class) and instantiation (creating objects of the class)
  - Attributes (state) of an object
  - Objects as arguments and return values
  - Methods specify behaviour of objects
  - Operator overloading
  - Objects as attributes and creating complex types
  - Copying objects (shallow and deep)
  - Static methods, class methods and class variables
- Key principles of OOP (to improve code reusability, modularity and maintainability)
  - Encapsulation - bundling data and methods together and restricting direct access to internal data
  - Abstraction - hiding the internal implementation and exposing only the necessary interface
  - Polymorphism - ability of a method or operator to exhibit different behavior depending on the type of the object on which it is called
    - Operator overloading is a special example of polymorphism
  - Inheritance - ability to define a new class that acquires the attributes and methods of another class
    - Single and multiple inheritance

References:

- https://docs.python.org/3/tutorial/classes.html
- https://docs.python.org/3/howto/mro.html
- Think Python (Edition 3) by Allen B. Downey

Programming Paradigms

A programming paradigm is a high-level way to conceptualize and structure a program. Two common programming paradigms are

- Procedural programming – a program is organized as procedures (functions) that call each other
- Object-oriented programming (OOP) – a program consists of objects that interact with one another
    - An object is an entity that has a state data (**attributes**), behavior (**methods**) and identity
    - Objects represent things in the real world and methods correspond to the ways these things interact
    - Most of the computation is expressed in terms of operations on objects

Point Type and Instantiating Point Objects

A location on the screen is often represented as a point on a 2-dimensional plane. A point in the plane is typically associated with two numbers representing its `x` and `y` coordinates. There are several ways to represent a point.

- Store the coordinates in two variables `x` and `y`.
- Store the coordinates as elements in a list or tuple.
- Create a new type to represent points as objects that contain the attributes `x` and `y`.

Let us take the third approach and start with creating a type called `Point`.

A programmer-defined type is also called a **class**. A new type can be defined by a template using the keyword `class`. Here's how a class definition for `Point` looks like.

```
class Point:
    """Represents a point in 2D space."""
```

Once a class (new type) is defined, independent instances of this class (**objects**) can be created. Creating a new object is called **instantiation**, and the object is an **instance** of the class.

```
p1 = Point()
p2 = Point()
```

```
print(type(p1))
```

```
<class '__main__.Point'>
```

The new objects' type is `__main__.Point`, where `__main__` is the name of the module where `Point` is defined. When Python runs a program, it loads the code as a module. The module that is executed directly (and not via `import`) gets the special name: `__main__`.

We can use `isinstance` to check whether an object is an instance of a particular class.

```
isinstance(p2, Point)
```

```
True
```

A docstring is a string written using triple quotes (`' ' '` or `" " "`) that appears as the first statement inside a class.

A docstring explains what the class is meant to represent. They are stored as metadata, accessible at runtime and used by help and documentation tools.

```
help(Point)
```

```
Help on class Point in module __main__:

class Point(builtins.object)
 |  Represents a point in 2D space.
 |
 |  Data descriptors defined here:
 |
 |  __dict__
 |      dictionary for instance variables (if defined)
 |
 |  __weakref__
 |      list of weak references to the object (if defined)
```

We intended a `Point` object to represent a point on the plane.

```
p1.x = 7
p1.y = 6
```

Each object has its own internal state -- the values of its **attributes** (think of attributes as local variables)

```
p2.x = -10
p2.y = -5
```

Not all objects need to have the same set of attributes.

```
p3 = Point()
p3.z = -17
```

When you print an object, Python tells you what type it is and where it is stored in memory (the prefix `0x` means that the following number is in hexadecimal). We can customize how an object is shown when printed.

```
print(p1,p2,p3)
```

```
<__main__.Point object at 0x10d730070> <__main__.Point object at 0x10d706d30>
```

We defined the datatype `Point` to represent a point in the 2D plane. Since a `Point` object is meant to represent a point, it seems reasonable to expect that it should have attributes denoting its coordinates. This can be forced as part of the class definition.

- How should an instance of `Point` look like?
- A `Point` object should ideally contain variables (attributes) that can store the coordinates (state of the object).

```
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, a, b):
        self.x = a
        self.y = b
    # self refers to the object being created
    # The Point object p has attributes p.x and p.y that are initialized to

    def __str__(self):
        return f'Point({self.x}, {self.y})'
    # self refers to the object whose str repr we are trying to obtain
```

```
p4 = Point(2,3)
```

```
print(p4)
```

```
Point(2, 3)
```

```
p4.x = 10
p4.y = -100
```

```
print(p4)
```

```
Point(10, -100)
```

Special Functions Associated with a Class

- `__init__` is a special function (called **constructor**) that is called automatically when you create a new object of the class.
  - `self` refers to the object being created.
  - The name `self` for the current object (first parameter) is only a convention and any other name can be used.
- `__init__` initializes the attributes of a new object.
- `__init__` takes the coordinates as parameters and assigns them to attributes (instance variables) `x` and `y` of the object being created.
- `__str__` is a special method that specifies how an object is shown as a string.
  - This method is automatically invoked by the `print` function.
  - If you use the built-in function `str` to convert a `Point` object to a string, Python uses the `__str__` method in the `Point` class.

When we instantiate a `Point` object, Python invokes `__init__`, and passes along the arguments. So we can create an object and initialize the attributes at the same time.

```
p5 = Point()
```

```

```

```
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b

    def __str__(self):
        return f'Point({self.x}, {self.y})'
```

```
p5 = Point()
```

```
print(p5)
```

```
Point(0, 0)
```

```
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'
```

In this example, the parameters are optional, so if you call `Point` with no arguments, you get the default values.

```
p = Point()
```

If you provide one argument, it overrides the corresponding default value.

```
q = Point(x=1)
```

If you provide two arguments, they override both default values.

```
r = Point(1,5)
```

```
r1 = Point(y=6,x=1)
```

It is a good practice to define `__init__` and `__str__` while defining a class.

```
p = Point()
q = Point(1,5)
r = Point(y=6,x=1)
print(p)
print(q)
print(r)
```

```
Point(0, 0)
Point(1, 5)
Point(1, 6)
```

We can add new attributes to objects.

```
r.z = 10
```

```
print(r)
```

```
Point(1, 6)
```

```
str(r)
```

```
'Point(1, 6)'
```

We can read the value of an attribute using the dot operator and use an attribute as part of any expression.

- Each object has its own internal state -- the values of its local variables (attributes).

Objects as Arguments and Return Values

We can pass an object as an argument in a function call.

Example 1: Here's a function to calculate distance between 2 points.

```
import math
def dist(p1, p2):
    dx = (p1.x - p2.x)
    dy = (p1.y - p2.y)
    dsq = dx**2 + dy**2
    return math.sqrt(dsq)
```

```
u = Point(1,5)
v = Point(-1,1)
dist(u,v)
```

```
4.47213595499958
```

Example 2: Here's a function that returns a `Point` object.

```
def scale(p,c):
    q = Point(c*p.x,c*p.y)
    return q
```

```
a = Point(45,66)
print(scale(a,-6))
```

```
Point(-270, -396)
```

Example 3: Here's another function that changes a `Point` object.

```
def reset(p):
    p.x = 0
    p.y = 0
```

```
w = Point(1,5)
print(w)
reset(w)
print(w)
```

```
Point(1, 5)
Point(0, 0)
```

By default, instances of programmer-defined classes are **mutable**.

- Can add or modify attributes after the object is created.

However, it is possible to define custom classes so that the objects are **immutable**.

Example 4: Suppose you want to calculate the distance of a point from origin.

```
import math
def dist_to_origin(p):
    dx = (p.x)
    dy = (p.y)
    dsq = dx**2 + dy**2
    return math.sqrt(dsq)
```

```
z = Point(1,5)
dist_to_origin(z)
```

```
5.0990195135927845
```

These examples describe functions that **act** on `Point` object(s). We may also think of these functions as describing the **behavior** of an object.

- A point can compute its distance from origin
- A point can compute its distance from another point
- A point can scale itself by a number or compute a point that is a scaled version of it
- A point can reset itself

We can rewrite such behaviour functions as **methods** defined inside the class definition.

Defining Methods

Methods are functions defined within a class and can be called on objects of that class. Think of the following methods as **acting** on `Point` object(s).

What are all the operations that we would want on `Point` objects?

A class definition typically specifies data and procedures associated with an object of this class.

- data attributes (or simply attributes) -- data that make up an object of this class
- procedural attributes (methods) -- specify how to interact with an object of this class

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0
    # self is a reference to the object of the class on which the method is

    def scale(self,c):
        self.x *= c
        self.y *= c

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)
```

Methods are associated with a particular class.

- Methods define the behavior of an object and actions that an object can perform.
- A method can take arguments, including a reference to the object on which it is called.
    - This allows the method to access and modify the object's attributes.
- In the definition of methods above, `self` is a reference to the object on which the method is being invoked.

- The name `self` for the current object (first parameter in a method) is only a convention - we can use any other name.

When we call a method on an object, reference to this object gets automatically passed to the method.

```
z = Point(1,7)
o = Point(0,0)
print(z.dist(o))
# When we call a method on an object, reference to this object gets automati
y = Point(-1,-7)
print(z.dist(y))
print(y.dist(z))
```

```
7.0710678118654755
14.142135623730951
14.142135623730951
```

All objects of a class share the same methods to query/update their state.

- Think of methods as functions being invoked on an object of this class.

Static Methods

A **static method** is a method that does not require an instance of the class to be invoked. It behaves like a normal function but is logically grouped inside the class.

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x *= c
        self.y *= c

    def scaled(self,c):
        return Point(c*self.x,c*self.y)
```

```
    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)
```

`collinear` describes a relationship between 3 point objects and is not tied to a single object.

```
p1 = Point(1,1)
p2 = Point(2,2)
p3 = Point(3,3)
print(Point.collinear(p1,p2,p3))
```

```
True
```

It is possible to define `collinear` as an instance method too.

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x *= c
        self.y *= c

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)
```

```
    def collinear(self, p2, p3):
        return (p2.y - self.y)*(p3.x - self.x) == (p3.y - self.y)*(p2.x - se
```

```
q1 = Point(1,1)
q2 = Point(2,2)
q3 = Point(3,3)
print(q1.collinear(q2,q3))
```

```
True
```

Operator Overloading

Consider the following code snippet.

```
p = Point(-1,8)
q = Point(6,-4)
print(p+q)
```

For every operator, there is a corresponding special method. Examples: `+` is mapped to `__add__`, `<` is mapped to `__lt__`, etc.

`+` implicitly calls `__add__()` and `p + q` gets translated as `p.__add__(q)`

For built-in types (`int`, `str`, etc.), the method `__add__()` is defined inside the implementation of the type.

Let us define `__add__()` inside `Point` so that it returns a new `Point` without modifying its arguments.

After defining this method for this class, we can use the `+` operator on `Point` objects.

```
import math

class Point:

    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
```

```
        self.y = 0

    def scale(self,c):
        self.x = c * self.x
        self.y = c * self.y

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)

    def __add__(self,p):
        return(Point(self.x + p.x, self.y + p.y))

    def __sub__(self,p):
        return(Point(self.x - p.x, self.y - p.y))

    def __mul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __matmul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __lt__(self,p):
        return(self.x < p.x and self.y < p.y)

    def __gt__(self,p):
        return(self.x > p.x and self.y > p.y)

    def __le__(self,p):
        return(self.x <= p.x and self.y <= p.y)

    def __ge__(self,p):
        return(self.x >= p.x and self.y >= p.y)
```

`__add__()` is executed in the context of one point and the other point is passed to it as an argument.

By defining other special methods, we can specify the behavior of other operators on our new type.

```
a = Point(-1,8)
b = Point(6,-4)
print(a+b)
print(a-b)
```

```
print(a*b)
print(a@b)
```

```
Point(5, 4)
Point(-7, 12)
Point(-6, -32)
Point(-6, -32)
```

Changing the behavior of an operator so that it works with programmer-defined types is called **operator overloading**.

```
p = Point(3,4)
q = Point(7,10)
p < q, q < p, q > p
```

```
(True, False, True)
```

```
p <= q, q <= p, q >= p
```

```
(True, False, True)
```

```
p1 = Point(1,1)
p2 = Point(1,1)
p1 == p2
```

```
False
```

You might expect `==` to yield `True` because the objects contain the same data. But for programmer-defined classes, the default behavior of the `==` operator is the same as the `is` operator -- it checks identity, not equivalence.

If we want to change this behavior, we can provide a special method called `__eq__` that defines what it means for two `Point` objects to be equal.

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
```

```
            self.x = c * self.x
            self.y = c * self.y

        def scaled(self,c):
            return Point(c*self.x,c*self.y)

        def dist(self, p2):
            dx = (self.x - p2.x)
            dy = (self.y - p2.y)
            dsq = dx**2 + dy**2
            return math.sqrt(dsq)

        @staticmethod
        def collinear(p1, p2, p3):
            return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)

        def __add__(self,p):
            return(Point(self.x + p.x, self.y + p.y))

        def __sub__(self,p):
            return(Point(self.x - p.x, self.y - p.y))

        def __mul__(self,p):
            return(Point(self.x * p.x, self.y * p.y))

        def __matmul__(self,p):
            return(Point(self.x * p.x, self.y * p.y))

        def __lt__(self,p):
            return(self.x < p.x and self.y < p.y)

        def __gt__(self,p):
            return(self.x > p.x and self.y > p.y)

        def __le__(self,p):
            return(self.x <= p.x and self.y <= p.y)

        def __ge__(self,p):
            return(self.x >= p.x and self.y >= p.y)

        def __eq__(self, other):
            return (self.x == other.x) and (self.y == other.y)

        def __ne__(self,p):
            return(not(self == p))
```

This definition considers two `Points` to be equal if their attributes are equal. Now when we use the `==` operator, it invokes the `__eq__` method, which indicates that `p1` and `p2` are considered equal.

```
p1 = Point(1,1)
p2 = Point(1,1)
```

```
p1 == p2
```

```
True
```

```
p1 != p2
```

```
False
```

But the `is` operator still indicates that they are different objects.

```
p1 is p2
```

```
False
```

It's not possible to change the behaviour of the `is` operator -- it always checks whether the objects are identical.

Copying Objects

Assignment statements do not copy objects, they create bindings between a target and an object. For collections that are mutable or contain mutable items, a copy is sometimes needed so one can change one copy without changing the other.

The `copy` module provides a function called `copy` that can duplicate any object.

```
import copy
```

To see how it works, let's start with a new `Point` object.

```
start = Point(10, 20)
```

And make a copy.

```
end = copy.copy(start)
```

Now `start` and `end` contain the same data.

```
print(start)
print(end)
```

```
Point(10, 20)
Point(10, 20)
```

However, the `is` operator confirms that they are not the same object.

```
start is end
```

```
False
```

Changing attributes of one copy does not affect the other.

```
start.x = 5
print(start)
print(end)
```

```
Point(5, 20)
Point(10, 20)
```

Objects as Attributes

Let's define a type `Line` that has `Point` objects as attributes.

```
import math
class Point:
    """Represents a point in 2-D space."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x = c * self.x
        self.y = c * self.y

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2):
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)
```

```
    def __add__(self,p):
        return(Point(self.x + p.x, self.y + p.y))

    def __sub__(self,p):
        return(Point(self.x - p.x, self.y - p.y))

    def __mul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __matmul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __lt__(self,p):
        return(self.x < p.x and self.y < p.y)

    def __gt__(self,p):
        return(self.x > p.x and self.y > p.y)

    def __le__(self,p):
        return(self.x <= p.x and self.y <= p.y)

    def __ge__(self,p):
        return(self.x >= p.x and self.y >= p.y)

    def __eq__(self, other):
        return (self.x == other.x) and (self.y == other.y)

    def __ne__(self,p):
        return(not(self == p))

    def translate(self,dx,dy):
        self.x += dx
        self.y += dy

    def translated(self, dx=0, dy=0):
        point = copy.copy(self)
        point.translate(dx, dy)
        return point
```

```
class Line:
    def __init__(self, q1, q2):
        self.p1 = q1
        self.p2 = q2

    def __str__(self):
        return f"Line: {self.p1}, {self.p2}"
```

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
print(L)
```

```
Line: Point(1, 2), Point(4, 6)
```

Can we uniquely define a line using only one point? How do we obtain the equation of a line?

```
class Line:
    def __init__(self, p1, p2):
        if p1==p2:
            raise ValueError("Two distinct points are required to uniquely d

        self.p1 = p1
        self.p2 = p2

        # m is a derived attribute
        if p1.x == p2.x:
            self.m = None
        else:
            self.m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.m is not None:
            return round(self.m,2)
        return None

    def isVertical(self):
        return self.m is None

    def __str__(self):
        if self.m is None:
            return f"Line: x = {self.p1.x}"
        return f"Line: y - {self.p1.y} = {self.m:.2f}(x - {self.p1.x})"
```

Now we can display the line in point-slope form.

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
print(L)
print(f"Slope: {L.getSlope()}")
```

```
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
```

```
L1= Line(Point(2,1), Point(2,5))
print(L1)
print(f"Slope: {L1.getSlope()}")
```

```
Line: x = 2
Slope: None
```

Consider the following snippet.

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
print(L)
L.p1 = Point(0,0)
print(L)
```

```
Line: y - 2 = 1.33(x - 1)
Line: y - 0 = 1.33(x - 0)
```

The equation of the line is incorrect. The issue seems to be that we are able to change an attribute outside the class definition. So let us restrict access to points associated with lines.

```
class Line:
    def __init__(self, p1, p2):
        if p1==p2:
            raise ValueError("Two distinct points are required to uniquely d

        self.__p1 = p1
        self.__p2 = p2

        if p1.x == p2.x:
            self.__m = None
        else:
            self.__m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.__m is not None:
            return round(self.__m,2)
        return None

    def isVertical(self):
        return self.__m is None

    def __str__(self):
        if self.__m is None:
            return f"Line: x = {self.__p1.x}"
        return f"Line: y - {self.__p1.y} = {self.__m:.2f}(x - {self.__p1.x})
```

A double underscore `__` before a variable or method triggers **name mangling** which renames the attribute making it harder to access from outside the class.

```
u = Point(1, 2)
v = Point(4, 6)

L = Line(u, v)
```

```
print(L)
print(L.__p1)
```

Name mangling essentially hides the attribute. Python interpreter changes the name of the attribute internally. The new name can be used to access the attribute -- however this is discouraged.

```
print(L._Line__p1)
```

```
Point(1, 2)
```

Consider the following snippet.

```
u = Point(1, 2)
v = Point(4, 6)
L = Line(u, v)
print(L)
print(f"Slope: {L.getSlope()}")
u.reset()
print(L)
print(f"Slope: {L.getSlope()}")
```

```
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
Line: y - 0 = 1.33(x - 0)
Slope: 1.33
```

The slope attribute no longer denotes the slope! Let us redefine `Line` that prevents such problems.

```
class Line:
    def __init__(self, p1, p2):
        if p1.x == p2.x and p1.y == p2.y:
            raise ValueError("Two distinct points are required to uniquely d

        self.__p1 = Point(p1.x, p1.y)
        self.__p2 = Point(p2.x, p2.y)

        if p1.x == p2.x:
            self.__m = None
        else:
            self.__m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.__m is not None:
            return round(self.__m,2)
        return None
```

```
    def isVertical(self):
        return self.__m is None

    def __str__(self):
        if self.__m is None:
            return f"Line: x = {self.__p1.x}"
        return f"Line: y - {self.__p1.y} = {self.__m:.2f}(x - {self.__p1.x})
```

```
u = Point(1, 2)
v = Point(4, 6)
L = Line(u, v)
print(L)
print(f"Slope: {L.getSlope()}")
u.reset()
print(u)
print(L)
print(f"Slope: {L.getSlope()}")
```

```
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
Point(0, 0)
Line: y - 2 = 1.33(x - 1)
Slope: 1.33
```

This is the beginning of building a mini geometry library. A geometry library is a collection of classes and functions that model geometric objects and operations. What geometric objects would you add? When you define these objects, make sure you think about the interaction between them.

Encapsulation and Abstraction

Recall the following definitions.

```
import math
import copy

class Point:

    """Represents a point in 2-D space.
        attributes: x and y for the coordinates
        methods: reset, scale, scaled, dist, collinear (static), translate,
    """

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'
```

```
    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x = c * self.x
        self.y = c * self.y

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2=None):
        if p2 is None:
            p2=Point(0,0)
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)

    def __add__(self,p):
        return(Point(self.x + p.x, self.y + p.y))

    def __sub__(self,p):
        return(Point(self.x - p.x, self.y - p.y))

    def __mul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __matmul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __lt__(self,p):
        return(self.x < p.x and self.y < p.y)

    def __gt__(self,p):
        return(self.x > p.x and self.y > p.y)

    def __le__(self,p):
        return(self.x <= p.x and self.y <= p.y)

    def __ge__(self,p):
        return(self.x >= p.x and self.y >= p.y)

    def __eq__(self, other):
        return (self.x == other.x) and (self.y == other.y)

    def __ne__(self,p):
        return(not(self == p))

    def translate(self,dx,dy):
```

```
        self.x += dx
        self.y += dy

    def translated(self, dx=0, dy=0):
        point = copy.copy(self)
        point.translate(dx, dy)
        return point
```

Note the default argument in the definition of `dist`. Specifying `p2=Point(0,0)` instead of `p2=None` will result in an error.

```
z = Point(1,7)
print(z.dist())
z1 = Point(-1,-7)
print(z.dist(z1))
print(z1.dist(z))
```

```
7.0710678118654755
14.142135623730951
14.142135623730951
```

```
import matplotlib.pyplot as plt
class Line:

    """Represents a line passing through 2 points in 2-D space.
    """

    def __init__(self, p1, p2):
        if p1.x == p2.x and p1.y == p2.y:
            raise ValueError("Two distinct points are required to uniquely d

        self.__p1 = Point(p1.x, p1.y)
        self.__p2 = Point(p2.x, p2.y)

        if p1.x == p2.x:
            self.__m = None
        else:
            self.__m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.__m is not None:
            return round(self.__m,2)
        return None

    def isVertical(self):
        return self.__m is None

    def __str__(self):
        if self.__m is None:
            return f"Line: x = {self.__p1.x}"
        return f"Line: y - {self.__p1.y} = {self.__m:.2f}(x - {self.__p1.x})
```

```
    def draw(self):
        x = [self.__p1.x, self.__p2.x]
        y = [self.__p1.y, self.__p2.y]
        plt.plot(x, y, marker='o',color='green')
```

Encapsulation -- bundling data and methods together and restricting direct access to internal data. The idea is to hide the internal representation of the objects and allow access only through methods.

For example, `Line`'s attributes `p1`, `p2`, `m` are not meant for access outside the class.

Abstraction -- hiding the internal implementation and exposing only the necessary interface.

For example, `Line` represents a line on the 2D plane but the users do not need to know how it is implemented/maintained internally -- like how is slope calculated, etc.

Notice the method to draw the line segment between the two given points of a line. In `matplotlib`, `plot()` draws points, connects them with straight lines on a graph and `marker` tells `plot()` what symbol to draw at each data point.

```
u = Point(1, 2)
v = Point(1, 6)
L = Line(u, v)
L.draw()
a = Point(-1, 2)
b = Point(10, 6)
L1 = Line(a, b)
L1.draw()
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Rectangles

Now let's define a class that represents and draws axis-parallel rectangles. What attributes do you think we should use to specify the location and size of such a rectangle?

There are at least two possibilities

- You could specify the width and height of the rectangle and the location of one corner.

- You could specify two opposing corners.

At this point it's hard to say whether either is better than the other, so let's implement the first one. Here is the class definition.

```
class Rectangle:

    """Represents an axis-parallel rectangle.
    attributes: width, height, lower-left corner.
    """

    def __init__(self, width, height, corner):
        self.width = width
        self.height = height
        self.corner = corner

    def __str__(self):
        return f'Rectangle({self.width}, {self.height}, {self.corner})'
```

As usual, the `__init__` method assigns the parameters to attributes and the `__str__` returns a string representation of the object. Now we can instantiate a `Rectangle` object, using a `Point` as the location of the lower-left corner.

```
corner = Point(30, 20)
box1 = Rectangle(100, 50, corner)
print(box1)
```

```
Rectangle(100, 50, Point(30, 20))
```

```
box1.draw()
```

To draw a rectangle,

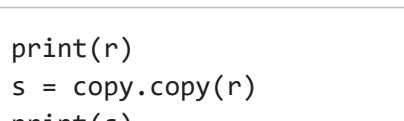
- make four `Point` objects to represent the corners (vertices)
- make four `Line` objects to represent the sides (edges)
- draw the vertices and the edges

```
import matplotlib.pyplot as plt

class Rectangle:

    """Represents an axis-parallel rectangle.
    attributes: width, height, lower-left corner.
    """

    def __init__(self, width, height, corner):
```

```
        self.width = width
        self.height = height
        self.corner = corner

    def __str__(self):
        return f'Rectangle({self.width}, {self.height}, {self.corner})'

    def make_points(self):
        p1 = self.corner
        p2 = p1.translated(self.width, 0)
        p3 = p2.translated(0, self.height)
        p4 = p3.translated(-self.width, 0)
        return p1, p2, p3, p4

    def make_lines(self):
        p1, p2, p3, p4 = self.make_points()
        return Line(p1, p2), Line(p2, p3), Line(p3, p4), Line(p4, p1)

    def draw(self):
        for line in self.make_lines():
            line.draw()

    def translate(self, dx, dy):
        self.corner.translate(dx, dy)
```

Here's an example.

```
r = Rectangle(4, 2, Point(1,1))
r.draw()
plt.gca().set_aspect('equal') #x and y scales are same
plt.show()
```



Copying a rectangle

```
print(r)
s = copy.copy(r)
print(s)
```

```
Rectangle(4, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

```
s is r
```

```
False
```

```
s.corner is r.corner
```

```
True
```

```
r.translate(30, 20)
r.draw()
s.draw()
plt.gca().set_aspect('equal')
plt.show()
```

```
s.translate(-30, -20)
print(s)
print(r)
```

```
Rectangle(4, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

```
s.width += 3
print(s)
print(r)
```

```
Rectangle(7, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

What `copy` function does is called a **shallow copy** -- it copies the object but not the objects it contains. That is, it does not copy nested objects. By performing a shallow copy, the outer object is copied but the inner objects are shared.

The `copy` module provides another function, called `deepcopy`, that copies not only the object but also the objects it refers to, and the objects *they* refer to, and so on. That is, it copies nested objects and this operation is called a **deep copy**.

```
r = Rectangle(4, 2, Point(1,1))
s = copy.deepcopy(r)
print(s)
print(r)
```

```
Rectangle(4, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

```
s is r
```

```
False
```

```
s.corner is r.corner
```

```
False
```

```
r.translate(30, 20)
r.draw()
s.draw()
```

```
    plt.gca().set_aspect('equal')
    plt.show()
```

The difference between shallow and deep copying is only relevant for compound objects (objects that contain other objects, like lists or class instances):

- A shallow copy constructs a new compound object and then (to the extent possible) inserts references into it to the objects found in the original.
    - Allows for data to be shared between copies
- A deep copy constructs a new compound object and then, recursively, inserts copies into it of the objects found in the original.
    - Allows for an independent copy

Polymorphism

```
start = Point(0, 0)
end1 = copy.copy(start)
end1.translate(30, 0)
end2 = start.translated(0, 15)
line1 = Line(start, end1)
line2 = Line(start, end2)
corner = Point(20, 20)
box3 = Rectangle(10, 50, corner)
box4 = copy.deepcopy(box3)
box3.translate(50, 30)
```

```
shapes = [line1, line2, box3, box4]
```

The elements of this list are different types, but they all provide a `draw` method, so we can loop through the list and invoke `draw` on each one.

```
for shape in shapes:
    shape.draw()
```

The first and second time through the loop, `shape` refers to a `Line` object, so when `draw` is invoked, the method that runs is the one defined in the `Line` class.

The third and fourth time through the loop, `shape` refers to a `Rectangle` object, so when `draw` is invoked, the method that runs is the one defined in the `Rectangle` class.

In a sense, each object knows how to draw itself. This feature is called **polymorphism**.

Polymorphism is the ability of a method or operator to exhibit different behavior depending on the type of the object on which it is called.

Recall operator overloading -- changing the behavior of an operator so that it works with programmer-defined types

Inheritance

Suppose you want to create a type `Square`. Do you observe any relationship between `Rectangle` and `Square`?

Instead of rewriting all the methods that `Rectangle` has, we can make `Square` "inherit" from `Rectangle`.

To define a new class that is based on an existing class, we mention the name of the existing class in parentheses.

```
class Square(Rectangle):

    """A rectangle with equal sides"""

    def __init__(self, length, corner):
        super().__init__(length, length, corner)

    def __str__(self):
        return f'Square({self.width}, {self.corner})'
```

Inheritance is the ability to define a new class that is a modified version of an existing class. It allows one class to acquire the attributes and methods of another class.

`Square` (**subclass**) automatically inherits attributes and methods from `Rectangle` (**superclass**).

However, if we redefine any of the methods that are already defined in the parent class, the ones in the parent class are overridden by the ones in the child class.

When a new class inherits from an existing one, the existing one is called the **parent** and the new class is called the **child**. In general,

- Instances of the child class should have all of the attributes of the parent class, but they can have additional attributes.
- The child class should have all of the methods of the parent class, but it can have additional methods.

- If a child class overrides a method from the parent class, the new method should take the same parameters and return a compatible result.

`super()` returns a proxy object that delegates method calls to a parent or sibling class. This is useful for accessing inherited methods that have been overridden in a class.

```
s = Square(5, Point(1,1))
s.draw()
s.translate(3,3)
s.draw()
plt.gca().set_aspect('equal') #x and y scale are same
plt.show()
```

```
s = Square(5, Point(1,1))
print(s.corner)
print(s)
```

```
Point(1, 1)
Square(5, Point(1, 1))
```

Class Methods

Suppose we want to create a rectangle using 2 points -- the lower left corner and the upper right corner.

- On which object do we call this method?

A **class method** works with the class itself instead of an instance. It is commonly used for alternative constructors.

```
import matplotlib.pyplot as plt

class Rectangle:

    """Represents a rectangle.

    attributes: width, height, corner.
    """

    def __init__(self, width, height, corner):
        self._width = width
        self._height = height
        self._corner = corner

    def __str__(self):
        return f'Rectangle({self._width}, {self._height}, {self._corner})'
```

```
    def make_points(self):
        p1 = self._corner
        p2 = p1.translated(self._width, 0)
        p3 = p2.translated(0, self._height)
        p4 = p3.translated(-self._width, 0)
        return p1, p2, p3, p4

    def make_lines(self):
        p1, p2, p3, p4 = self.make_points()
        return Line(p1, p2), Line(p2, p3), Line(p3, p4), Line(p4, p1)

    def draw(self):
        for line in self.make_lines():
            line.draw()

    def translate(self, dx, dy):
        self._corner.translate(dx, dy)

    def area(self):
        return self._width * self._height

    @classmethod
    def build(cls, p1, p2):
        width = abs(p2.x - p1.x)
        height = abs(p2.y - p1.y)
        corner = Point(min(p1.x, p2.x), min(p1.y, p2.y))
        return cls(width, height, corner)
```

A single underscore `_` before a variable or method is a naming convention that signals to other programmers that this attribute is intended for use inside the class or its subclasses, not from outside.

```
p1 = Point(1, 1)
p2 = Point(5, 4)

r = Rectangle.build(p1, p2)
r.draw()
```

Note: We can also use **static method** as alternative constructors, however, class methods are a better option in the context of inheritance.

Class Variables

Suppose we want to count how many `Square` objects are created. We can use a **class variable** for this purpose. A class variable is shared by all objects of the class.

```
class Square(Rectangle):

    """A rectangle with equal sides"""

    sqCount = 0

    def __init__(self, length, corner):
        super().__init__(length, length, corner)
        Square.sqCount += 1


    def __str__(self):
        return f'Square({self._width}, {self._corner})'

    @classmethod
    def build(cls, p1, p2):
        width = abs(p2.x - p1.x)
        height = abs(p2.y - p1.y)
        if width != height:
            raise ValueError("Points do not form a square")
        corner = Point(min(p1.x, p2.x), min(p1.y, p2.y))
        return cls(width, corner)
```

Accessing `s._corner` from outside the class is discouraged. If a double underscore `__` is used before a variable or method, then this attribute would be accessible from a subclass only using the mangled name.

```
s = Square(5, Point(1,1))
s1 = Square.build(Point(1,1), Point(5,5))
print(Square.sqCount)
```

```
2
```

```
s2= Square.build(Point(1,1), Point(6,5))
```

Creating squares using (deep) copy mechanism does not invoke `__init__` and therefore does not affect `count`. You will have to redefine `__deepcopy__` for that.

```
Square.sqCount = 0
```

```
print(Square.sqCount)
```

```
0
```

Multiple Inheritance

```
import matplotlib.pyplot as plt

class Colored:
    def __init__(self, color):
        self.color = color

    def get_color(self):
        return self.color

    def fill(self, square):
        x = square._corner.x
        y = square._corner.y
        w = square._width
        h = square._height
        rect = plt.Rectangle((x, y), w, h, fill=True, color=self.color)
        plt.gca().add_patch(rect)
```

```
class ColoredSquare(Square, Colored):
    def __init__(self, size, corner, color):
        Square.__init__(self, size, corner)
        Colored.__init__(self, color)

    #def fill(self):
    #    Colored.fill(self, self)
```

```
cs = ColoredSquare(5, Point(0,0), "gray")
print(cs.area())        # from Rectangle
print(cs.get_color())   # from Colored
print(cs._width)        # from Square
cs.draw()               # from Rectangle
cs.fill(cs)             # from Colored
#cs.fill() works if fill is redefined in ColoredSquare as mentioned above
```

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture: Key principles of OOP (to improve code reusability, modularity and maintainability)

- Encapsulation, Abstraction, Polymorphism, Inheritance

Reference: Think Python (Edition 3) by Allen B. Downey

Creating Point and Line

```
import math
import copy

class Point:

    """Represents a point in 2-D space.
        attributes: x and y for the coordinates
        methods: reset, scale, scaled, dist, collinear (static), translate, translated
    """

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return f'Point({self.x}, {self.y})'

    def reset(self):
        self.x = 0
        self.y = 0

    def scale(self,c):
        self.x = c * self.x
        self.y = c * self.y

    def scaled(self,c):
        return Point(c*self.x,c*self.y)

    def dist(self, p2=None):
        if p2 is None:
            p2=Point(0,0)
        dx = (self.x - p2.x)
        dy = (self.y - p2.y)
        dsq = dx**2 + dy**2
        return math.sqrt(dsq)

    @staticmethod
    def collinear(p1, p2, p3):
        return (p2.y - p1.y)*(p3.x - p1.x) == (p3.y - p1.y)*(p2.x - p1.x)

    def __add__(self,p):
        return(Point(self.x + p.x, self.y + p.y))

    def __sub__(self,p):
        return(Point(self.x - p.x, self.y - p.y))

    def __mul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __matmul__(self,p):
        return(Point(self.x * p.x, self.y * p.y))

    def __lt__(self,p):
        return(self.x < p.x and self.y < p.y)

    def __gt__(self,p):
        return(self.x > p.x and self.y > p.y)

    def __le__(self,p):
        return(self.x <= p.x and self.y <= p.y)

    def __ge__(self,p):
        return(self.x >= p.x and self.y >= p.y)

    def __eq__(self, other):
        return (self.x == other.x) and (self.y == other.y)
```

```
    def __ne__(self,p):
        return(not(self == p))

    def translate(self,dx,dy):
        self.x += dx
        self.y += dy

    def translated(self, dx=0, dy=0):
        point = copy.copy(self)
        point.translate(dx, dy)
        return point
```

Note the default argument in the definition of `dist`. Specifying `p2=Point(0,0)` instead of `p2=None` will result in an error.

```
z = Point(1,7)
print(z.dist())
z1 = Point(-1,-7)
print(z.dist(z1))
print(z1.dist(z))
```

```
7.0710678118654755
14.142135623730951
14.142135623730951
```

```
import matplotlib.pyplot as plt
class Line:

    """Represents a line passing through 2 points in 2-D space.
    """

    def __init__(self, p1, p2):
        if p1.x == p2.x and p1.y == p2.y:
            raise ValueError("Two distinct points are required to uniquely define a line")

        self.__p1 = Point(p1.x, p1.y)
        self.__p2 = Point(p2.x, p2.y)

        if p1.x == p2.x:
            self.__m = None
        else:
            self.__m = (p2.y - p1.y) / (p2.x - p1.x)

    def getSlope(self):
        if self.__m is not None:
            return round(self.__m,2)
        return None

    def isVertical(self):
        return self.__m is None

    def __str__(self):
        if self.__m is None:
            return f"Line: x = {self.__p1.x}"
        return f"Line: y - {self.__p1.y} = {self.__m:.2f}(x - {self.__p1.x})"

    def draw(self):
        x = [self.__p1.x, self.__p2.x]
        y = [self.__p1.y, self.__p2.y]
        plt.plot(x, y, marker='o',color='green')
```

Encapsulation -- bundling data and methods together and restricting direct access to internal data. The idea is to hide the internal representation of the objects and allow access only through methods.

For example, `Line`'s attributes `p1`, `p2`, `m` are not meant for access outside the class.

Abstraction -- hiding the internal implementation and exposing only the necessary interface.

For example, `Line` represents a line on the 2D plane but the users do not need to know how it is implemented/maintained internally -- like how is slope calculated, etc.

Notice the method to draw the line segment between the two given points of a line. In `matplotlib`, `plot()` draws points, connects them with straight lines on a graph and `marker` tells `plot()` what symbol to draw at each data point.

```
u = Point(1, 2)
v = Point(1, 6)
L = Line(u, v)
```

```
L.draw()
a = Point(-1, 2)
b = Point(10, 6)
L1 = Line(a, b)
L1.draw()
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Creating Rectangles

Now let's define a class that represents and draws axis-parallel rectangles. What attributes do you think we should use to specify the location and size of such a rectangle?

There are at least two possibilities

- You could specify the width and height of the rectangle and the location of one corner.
- You could specify two opposing corners.

At this point it's hard to say whether either is better than the other, so let's implement the first one. Here is the class definition.

```
class Rectangle:

    """Represents an axis-parallel rectangle.
    attributes: width, height, lower-left corner.
    """

    def __init__(self, width, height, corner):
        self.width = width
        self.height = height
        self.corner = corner

    def __str__(self):
        return f'Rectangle({self.width}, {self.height}, {self.corner})'
```

As usual, the `__init__` method assigns the parameters to attributes and the `__str__` returns a string representation of the object. Now we can instantiate a `Rectangle` object, using a `Point` as the location of the lower-left corner.

```
corner = Point(30, 20)
box1 = Rectangle(100, 50, corner)
print(box1)
```

```
Rectangle(100, 50, Point(30, 20))
```

```
box1.draw()
```

To draw a rectangle,

- make four `Point` objects to represent the corners (vertices)
- make four `Line` objects to represent the sides (edges)
- draw the vertices and the edges

```
import matplotlib.pyplot as plt

class Rectangle:

    """Represents an axis-parallel rectangle.
    attributes: width, height, lower-left corner.
    """

    def __init__(self, width, height, corner):
        self.width = width
        self.height = height
        self.corner = corner

    def __str__(self):
        return f'Rectangle({self.width}, {self.height}, {self.corner})'

    def make_points(self):
        p1 = self.corner
```

```
        p2 = p1.translated(self.width, 0)
        p3 = p2.translated(0, self.height)
        p4 = p3.translated(-self.width, 0)
        return p1, p2, p3, p4

    def make_lines(self):
        p1, p2, p3, p4 = self.make_points()
        return Line(p1, p2), Line(p2, p3), Line(p3, p4), Line(p4, p1)

    def draw(self):
        for line in self.make_lines():
            line.draw()

    def translate(self, dx, dy):
        self.corner.translate(dx, dy)
```

Here's an example.

```
r = Rectangle(4, 2, Point(1,1))
r.draw()
plt.gca().set_aspect('equal') #x and y scales are same
plt.show()
```


Copying a rectangle

```
print(r)
s = copy.copy(r)
print(s)
```

```
Rectangle(4, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

```
s is r
```

```
False
```

```
s.corner is r.corner
```

```
True
```

```
r.translate(30, 20)
r.draw()
s.draw()
plt.gca().set_aspect('equal')
plt.show()
```

```
s.translate(-30, -20)
print(s)
print(r)
```

```
Rectangle(4, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

```
s.width += 3
print(s)
print(r)
```

```
Rectangle(7, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

What `copy` function does is called a **shallow copy** -- it copies the object but not the objects it contains. That is, it does not copy nested objects. By performing a shallow copy, the outer object is copied but the inner objects are shared.

The `copy` module provides another function, called `deepcopy`, that copies not only the object but also the objects it refers to, and the objects *they* refer to, and so on. That is, it copies nested objects and this operation is called a **deep copy**.

```
r = Rectangle(4, 2, Point(1,1))
s = copy.deepcopy(r)
print(s)
print(r)
```

```
Rectangle(4, 2, Point(1, 1))
Rectangle(4, 2, Point(1, 1))
```

```
s is r
```

```
False
```

```
s.corner is r.corner
```

```
False
```

```
r.translate(30, 20)
r.draw()
s.draw()
plt.gca().set_aspect('equal')
plt.show()
```

The difference between shallow and deep copying is only relevant for compound objects (objects that contain other objects, like lists or class instances):

- A shallow copy constructs a new compound object and then (to the extent possible) inserts references into it to the objects found in the original.
  - Allows for data to be shared between copies
- A deep copy constructs a new compound object and then, recursively, inserts copies into it of the objects found in the original.
  - Allows for an independent copy

Polymorphism

```
start = Point(0, 0)
end1 = copy.copy(start)
end1.translate(30, 0)
end2 = start.translated(0, 15)
line1 = Line(start, end1)
line2 = Line(start, end2)
corner = Point(20, 20)
box3 = Rectangle(10, 50, corner)
box4 = copy.deepcopy(box3)
box3.translate(50, 30)
```

```
shapes = [line1, line2, box3, box4]
```

The elements of this list are different types, but they all provide a `draw` method, so we can loop through the list and invoke `draw` on each one.

```
for shape in shapes:
    shape.draw()
```

The first and second time through the loop, `shape` refers to a `Line` object, so when `draw` is invoked, the method that runs is the one defined in the `Line` class.

The third and fourth time through the loop, `shape` refers to a `Rectangle` object, so when `draw` is invoked, the method that runs is the one defined in the `Rectangle` class.

In a sense, each object knows how to draw itself. This feature is called **polymorphism**.

Polymorphism is the ability of different types to provide the same methods, which makes it possible to perform many computations -- like drawing shapes -- by invoking the same method on different types of objects.

It refers to the ability of a method or operator to exhibit different behavior depending on the type of the object on which it is called.

Recall operator overloading -- changing the behavior of an operator so that it works with programmer-defined types

Inheritance

Suppose you want to create a type `Square`. Do you observe any relationship between `Rectangle` and `Square`?

Instead of rewriting all the methods that `Rectangle` has, we can make `Square` "inherit" from `Rectangle`.

To define a new class that is based on an existing class, we mention the name of the existing class in parentheses.

```
class Square(Rectangle):

    """A rectangle with equal sides"""

    def __init__(self, length, corner):
        super().__init__(length, length, corner)
```

Inheritance is the ability to define a new class that is a modified version of an existing class. It allows one class to acquire the properties and methods of another class.

`Square` (**subclass**) automatically inherits attributes and methods from `Rectangle` (**superclass**).

However, if we redefine any of the methods that are already defined in the parent class, the ones in the parent class are overridden by the ones in the child class.

When a new class inherits from an existing one, the existing one is called the **parent** and the new class is called the **child**. In general,

- Instances of the child class should have all of the attributes of the parent class, but they can have additional attributes.
- The child class should have all of the methods of the parent class, but it can have additional methods.
- If a child class overrides a method from the parent class, the new method should take the same parameters and return a compatible result.

`super()` returns a proxy object that delegates method calls to a parent or sibling class. This is useful for accessing inherited methods that have been overridden in a class.

```
s = Square(5, Point(1,1))
s.draw()
s.translate(3,3)
s.draw()
plt.gca().set_aspect('equal') #x and y scale are same
plt.show()
```

```
print(s)
```

```
Rectangle(5, 5, Point(4, 4))
```

```
class Square(Rectangle):

    """A rectangle with equal sides"""

    def __init__(self, length, corner):
        super().__init__(length, length, corner)

    def __str__(self):
        return f'Square({self.width}, {self.corner})'
```

```
s = Square(5, Point(1,1))
print(s.corner)
print(s)
```

```
Point(1, 1)
Square(5, Point(1, 1))
```

This is another example of polymorphism -- the ability of a method/operator to exhibit different behavior depending on the type of the object on which it is called.

A single underscore `_` before a variable or method is a naming convention that signals to other programmers that this attribute is intended for use inside the class or its subclasses, not from outside.

```
import matplotlib.pyplot as plt

class Rectangle:

    """Represents a rectangle.

    attributes: width, height, corner.
```

```
        """

        def __init__(self, width, height, corner):
            self._width = width
            self._height = height
            self._corner = corner

        def __str__(self):
            return f'Rectangle({self._width}, {self._height}, {self._corner})'

        def make_points(self):
            p1 = self._corner
            p2 = p1.translated(self._width, 0)
            p3 = p2.translated(0, self._height)
            p4 = p3.translated(-self._width, 0)
            return p1, p2, p3, p4

        def make_lines(self):
            p1, p2, p3, p4 = self.make_points()
            return Line(p1, p2), Line(p2, p3), Line(p3, p4), Line(p4, p1)

        def draw(self):
            for line in self.make_lines():
                line.draw()

        def translate(self, dx, dy):
            self._corner.translate(dx, dy)
```

```
class Square(Rectangle):

    """A rectangle with equal sides"""

    def __init__(self, length, corner):
        super().__init__(length, length, corner)

    def __str__(self):
        return f'Square({self._width}, {self._corner})'
```

```
s = Square(5, Point(1,1))
print(s._corner)
print(s)
```

```
Point(1, 1)
Square(5, Point(1, 1))
```

Accessing `s._corner` from outside the class is discouraged.

If a double underscore `__` is used before a variable or method, then this attribute would be accessible from a subclass only using the mangled name.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture:

- Multifile Programming
- Command-line Arguments

Reference: https://docs.python.org/3/tutorial/modules.html

Modules

As programs gets longer, you may want to split it into several files for easier maintenance. For example, you may want to use a function that you've written in several programs without copying its definition into each program.

To support this, Python has a way to put (class/function) definitions in a file and use them in a script or in an interactive instance of the interpreter.

- Such a file is called a **module**; definitions from a module can be imported into other modules or into the main module.
- A module is a file containing Python definitions and statements.

Multifile programming involves organizing code into separate `.py` files (modules) to improve readability, management and reusability.

Here is a simple example demonstrating how to create a module and use it in another file.

- Create files named `point.py`, `line.py`, `rectangle.py`, `square.py`, `colored.py`, `colsquare.py` containing the corresponding class definitions.
  - Note that you will have to import appropriate modules
- Write a main program `shapes.py` that uses the functionality of the imported modules.

For larger projects, you can organize related modules into a **package**, which is a directory containing multiple module files. This allows for a hierarchical structure (e.g., from package.module import class/function). You can further organize related packages into a **library**.

Executing Modules as Scripts

Within a module, the module's name (as a string) is available as the value of the global variable `__name__`.

On executing a module (script), the code in this module will be executed with `__name__` set to `__main__`.

By adding `if __name__ == "__main__":` block at the end of a module, you can make the file usable as a script as well as an importable module.

Command-Line Arguments

Python's standard library's `sys` module enables a script to receive command-line arguments that are passed into the program. These include the script's name and any values that appear to the right of it when you execute the script. The `sys` module's `argv` list contains the arguments.

For example, consider the following program that simulates 500 dice throws using the `random` module's `randrange` function. This function generates a random integer from a specified range.

```
import random

rolls = [random.randrange(1,7) for i in range(500)]
counts = [rolls.count(i) for i in range(1,7)]
print(counts)
```

```
[93, 70, 67, 99, 89, 82]
```

Store this code in a file `RollDie.py` with the line

`rolls = [random.randrange(1,7) for i in range(500)]`

replaced with

`rolls = [random.randrange(1,7) for i in range(int(sys.argv[1]))]`.

Also, add `import sys` to the beginning of the file.

Now, execute this script using the command `python3 RollDie.py 500`. Here, `argv[0]` is the string 'RollDie.py' and `argv[1]` is the string '500'.

ID1110 Introduction to Programming (in Python)

Topics for Today's Lecture

- Random module

Reference: https://docs.python.org/3/library/random.html

Random Phenomena and Random Experiments

Many physical processes are **intrinsically random**, examples include the behavior of subatomic particles, such as the polarization of a photon and the spin of an electron.

Some processes that are not necessarily random are **treated as random** because they are too complex to be predicted exactly. An example is the flipping of a coin. Will the coin land with heads side up or tails side up?

- The answer depends on several factors such as wind, humidity, the turbulence in the air, the force with which the coin was flipped, the initial orientation of the coin, the impact point between the finger and the coin, the surface smoothness of the table the coin lands on, the material characteristics of the coin and the table, and so on.
- If we know all these relevant measurements precisely we would be able to definitively say whether the coin would come up heads or tails. However, in the absence of these information, we cannot predict the outcome of the coin flip with absolute certainty.

A **random phenomenon** is an uncertain situation whose outcome we can't predict with certainty

- Rainfall is a random phenomenon in hydrology
  - Though the physical processes behind rainfall are understood, the exact timing, location and intensity of rainfall cannot be predicted with absolute certainty
  - Estimating rainfall helps estimate flood probability for a river or drainage system
- Road roughness is a random phenomenon in transportation engineering
  - Irregular vertical deviations of a road surface from a perfectly flat plane
  - As a car drives, the road profile acts as a random excitation resulting in vibration responses in the car body, suspension and driver's seat
  - Estimating the vibration helps in vehicle design
- A bit error is the random corruption of a single digital bit during transmission or storage caused by unpredictable environmental noise, signal degradation, etc.
  - Bit error rate is ratio of incorrectly received bits to total bits transmitted (represents probability of error per bit)
  - Estimating bit error rate helps decide whether a communication system is reliable enough
- Machine learning - study of algorithms that can learn from data, generalize to unseen data and perform tasks without explicit instructions
  - Randomness is used to make models learn better and handle uncertainty
- Cryptography - study of techniques for secure communication in the presence of adversarial behavior
  - Encryption keys are generated randomly and some signature schemes use randomness
  - Estimate how unlikely it is for an attacker to guess keys and break the system

We can model a random phenomenon as a **random experiment** -- a well-defined procedure that produces an observable outcome that cannot be predicted with absolute certainty in advance.

The computer programs that we have seen so far are **deterministic**

- Given the same input, they generate the same output every time
- Determinism is desirable since we expect the same computation to yield the same result

To **simulate** (imitate) a random phenomenon by performing a random experiment, a program must be able to make unpredictable choices. Key to this ability is generating random numbers (like those obtained from random physical processes).

- Generating purely random number is difficult but they are ways to generate numbers that appear random -- **pseudorandom** numbers.
- Pseudorandom numbers are not truly random because they are generated by a deterministic procedures (**pseudorandom number generators**) but just by looking at the numbers it is almost impossible to distinguish them from purely random numbers.

Random Numbers, Shuffling and Sampling

The `random` module is a built-in library that provides functions that generate (pseudo)random numbers and perform random selections or shuffling of sequences. This module implements pseudo-random number generators for various distributions.

```
import random
```

The `randint` function returns a random integer between two given numbers (both inclusive).

```
print(random.randint(1,10))
```

```
5
```

The `randrange` function returns a random element from `range(start, stop, step)`.

```
print(random.randrange(1,11,2))
```

```
3
```

The `random` function returns a random floating-point number between 0 and 1 (including 0 but excluding 1)

```
print(random.random())
```

```
0.17461325436919373
```

The `uniform(a,b)` function returns a random floating-point number n with $a \leq n \leq b$ for $a \leq b$ and $b \leq n \leq a$ for $b < a$.

```
print(random.uniform(5, 7))
```

```
5.111601171819974
```

The `choice` function can be used to choose an element from a list at random.

```
t = [1, 2, 3]
random.choice(t)
```

```
2
```

The `choices` function selects multiple random elements from a sequence, and allows for **sampling with replacement** (meaning the same item can be selected multiple times). It supports optional weights parameters to change the probability of each item being selected.

```
random.choices([1,2,3,4,5,6,7], weights = [1,2,2,1,3,3,1], k = 3)
```

```
[5, 6, 6]
```

The `sample` function can be used to return a new list containing k random elements (in selection order) from a given sequence without modifying the original sequence. It is typically used for random **sampling without replacement**. If the original sequence contains repeats, then each occurrence is a possible selection in the sample.

```
random.sample([10, 20, 30, 40, 50], k = 3)
```

```
[10, 50, 40]
```

The `shuffle` function is used to randomly reorder the elements of a mutable sequence (like a list) in place.

```
deck = 'ace two three four'.split()
random.shuffle(deck)                    # Shuffle a list in-place
deck
```

```
['three', 'two', 'ace', 'four']
```

The `numpy.random` module also implements pseudo-random number generators with the ability to draw samples from a variety of probability distributions. It is optimized for high-performance scientific computing.

1. Simulating Dice Rolls

```
import random
```

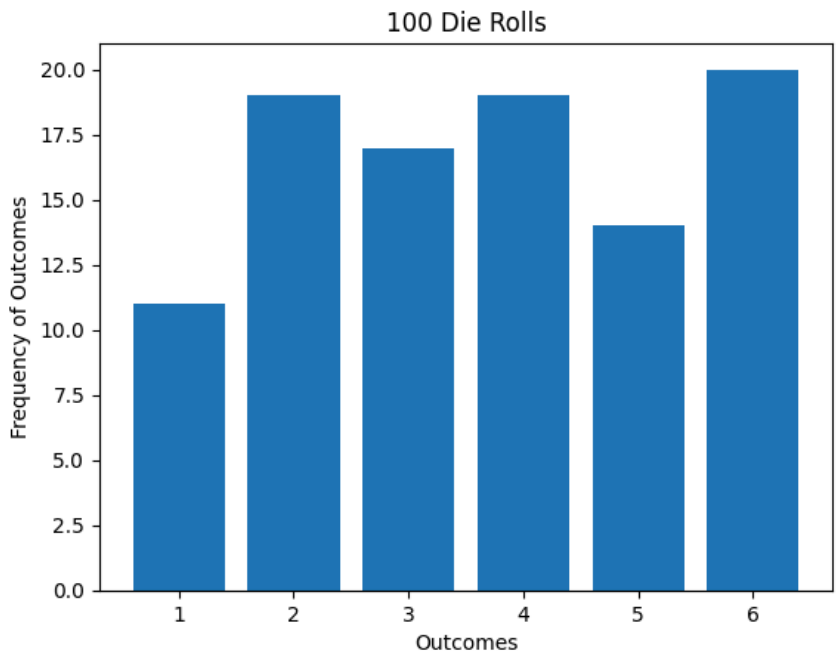
(a) Write a function `simulateDice(n)` that rolls an unbiased six-sided die n times and returns the frequencies of the results, i.e., the number of times 1 appears, the no. of times 2 appears, etc.

```
def throwDice():
    return random.randint(1,6)

def simulateDice(n):
    freq = [0,0,0,0,0,0]
    for i in range(n):
        r = throwDice()
        freq[r-1] = freq[r-1] + 1
    return freq
```

(b) Run `simulateDice(100)` and plot a bar chart with outcomes of die rolls against the frequencies.

```
import matplotlib.pyplot as p
# import the pyplot module from matplotlib library with alias p
```

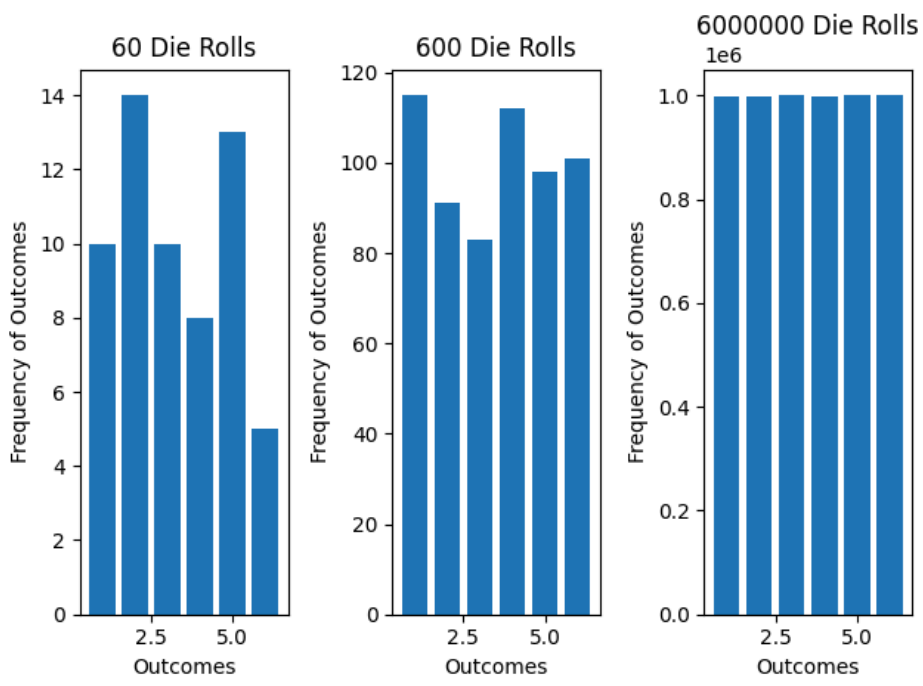
```
freq = simulateDice(100)

# create a bar chart from the given data
p.bar(range(1,7), freq)
# bar(x, y)

# add a title to the bar chart
p.title(f"100 Die Rolls")

# add labels to x-axis and y-axis
p.xlabel("Outcomes")
p.ylabel("Frequency of Outcomes")

# display the created plot
p.show()
```



(c) For each $n \in \{60, 600, 6 \times 10^6\}$, run `simulateDice(n)` and plot a bar chart with outcomes of die rolls against the frequencies.

In matplotlib, subplots are used to arrange multiple plots within a single figure. The primary method for this is `pyplot.subplots`, which creates the figure and a grid of subplots in a single call.

```
iterations = [60, 600, 6*(10**6)]
f, axes = p.subplots(1, len(iterations)) # 1 x 3 grid of plots
# subplots() returns the figure object (the entire canvas) &  a list of the individual subplots
for i, itr in enumerate(iterations):
    freq = simulateDice(itr)
    axes[i].bar(range(1,7), freq)
    axes[i].set_title(f"{itr} Die Rolls")
    axes[i].set_xlabel("Outcomes")
    axes[i].set_ylabel("Frequency of Outcomes")
p.tight_layout() # automatically adjusts the spacing between subplots
p.show()
```

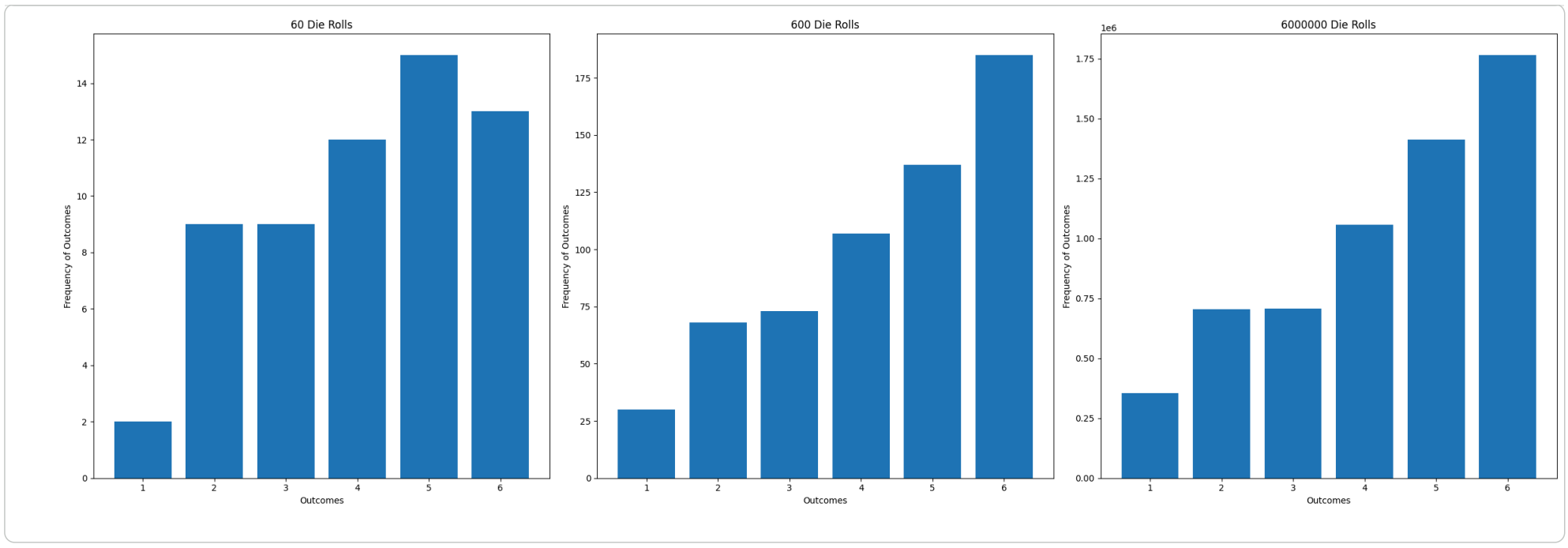


(d) Write a function `simulateBiasedDice(n,bias)` that rolls a biased six-sided die n times and returns the frequencies of the results, i.e., the number of times 1 appears, the no. of times 2 appears, etc.

```
def throwBiasedDice(probs=[1/6,1/6,1/6,1/6,1/6,1/6]):
    return random.choices([1,2,3,4,5,6], weights=probs)[0]

def simulateBiasedDice(n, probs=[1/6,1/6,1/6,1/6,1/6,1/6]):
    freq = [0,0,0,0,0,0]
    for i in range(n):
        r = throwBiasedDice(probs)
        freq[r-1] = freq[r-1] + 1
    return freq
```

(e) For each $n \in \{60, 600, 6 \times 10^6\}$, run `simulateBiasedDice(n)` and plot a bar chart with outcomes of die rolls against the frequencies.

```
probs = [1, 2, 2, 3, 4, 5]
iterations = [60, 600, 6*(10**6)]
f, axes = p.subplots(1, len(iterations), figsize=(24,8))
for i, itr in enumerate(iterations):
    freq = simulateBiasedDice(itr,probs)
    axes[i].bar(range(1,7), freq)
    axes[i].set_title(f"{itr} Die Rolls")
    axes[i].set_xlabel("Outcomes")
    axes[i].set_ylabel("Frequency of Outcomes")
p.tight_layout()
p.show()
```



2. Shuffling Cards

Deal 13 cards picked at random from a standard pack of cards.

```
class Card:
    suits = ['♠', '♥', '♦', '♣']
    ranks = ['A','2','3','4','5','6','7','8','9','10','J','Q','K']

    def __init__(self, suit, rank):
        if suit not in Card.suits:
            raise ValueError("Invalid suit")
        if rank not in Card.ranks:
            raise ValueError("Invalid rank")
        self.suit = suit
        self.rank = rank

    def __str__(self):
        return f"{self.rank}{self.suit}"
```

```
import random
class Deck:

    def __init__(self):
        self.cards = [Card(s, r) for s in Card.suits for r in Card.ranks]

    def shuffle(self):
        random.shuffle(self.cards)

    def deal(self, n):
        if n > len(self.cards):
            raise ValueError("Not enough cards in deck")
        hand = self.cards[:n] #take first n cards
        self.cards = self.cards[n:] #leave rest on deck
        return hand

    def __str__(self):
        return "Deck: "+', '.join(str(card) for card in self.cards)

    def __repr__(self):
        return str(self)
```

The information a programmer needs about an object typically differs from how the program should display the same object for the user. The special methods `__repr__` and `__str__` both return string representations of the object. The reason there are two methods to display an object is that they have different purposes:

- `__repr__` provides the official string representation of an object, aimed at the programmer.
- `__str__` provides the informal string representation of an object, aimed at the user.

Python's REPL (interactive mode) returns the official string representation defined by `__repr__` when you evaluate a line that includes only the object or its variable name.

If a class defines `__repr__` but not `__str__`, then `__repr__` is used when an informal string representation of instances of that class is required.

```
deck = Deck()
deck
```

```
Deck: A♠, 2♠, 3♠, 4♠, 5♠, 6♠, 7♠, 8♠, 9♠, 10♠, J♠, Q♠, K♠, A♥, 2♥, 3♥, 4♥, 5♥, 6♥, 7♥, 8♥, 9♥, 10♥, J♥, Q♥, K♥, A♦, 2♦, 3♦, 4♦, 5♦, 6♦, 7♦, 8♦, 9♦, 10♦, J♦, Q♦, K♦,
A♣, 2♣, 3♣, 4♣, 5♣, 6♣, 7♣, 8♣, 9♣, 10♣, J♣, Q♣, K♣
```

```
deck.shuffle()
deck
```

```
Deck: 5♠, 10♣, K♦, 5♥, 6♠, 6♦, K♠, Q♥, J♣, 2♦, 4♠, A♥, 3♠, 10♠, Q♦, 7♥, 4♥, 4♣, J♥, 2♥, 3♥, 7♠, 10♥, A♠, K♥, 8♥, 5♣, 8♣, 4♦, 3♣, 6♥, K♣, 9♥, 9♦, A♣, 10♦, 6♣, 2♣, Q♣,
3♦, 5♦, A♦, 7♦, 8♦, 9♠, J♦, Q♠, 2♠, 7♣, 9♣, 8♠, J♠
```

```
hand = deck.deal(13)
print("13 random cards:", ', '.join(str(card) for card in hand))
print(deck)
```

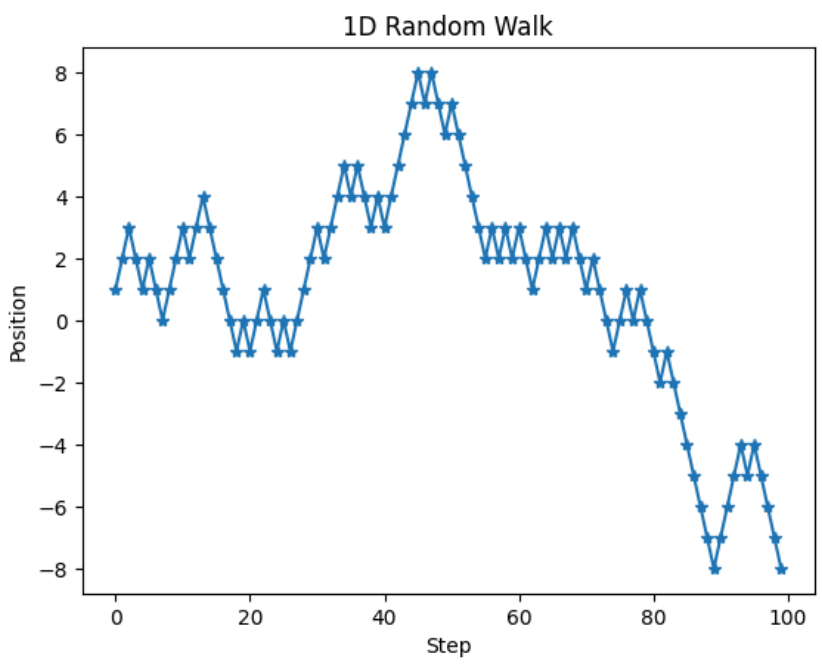
```
13 random cards: 5♠, 10♣, K♦, 5♥, 6♠, 6♦, K♠, Q♥, J♣, 2♦, 4♠, A♥, 3♠
Deck: 10♠, Q♦, 7♥, 4♥, 4♣, J♥, 2♥, 3♥, 7♠, 10♥, A♠, K♥, 8♥, 5♣, 8♣, 4♦, 3♣, 6♥, K♣, 9♥, 9♦, A♣, 10♦, 6♣, 2♣, Q♣, 3♦, 5♦, A♦, 7♦, 8♦, 9♠, J♦, Q♠, 2♠, 7♣, 9♣, 8♠, J♠
```

3. Random Walk

A **random walk** on a mathematical space is a sequence of discrete steps in which each step is randomly taken subject to a set of restrictions.

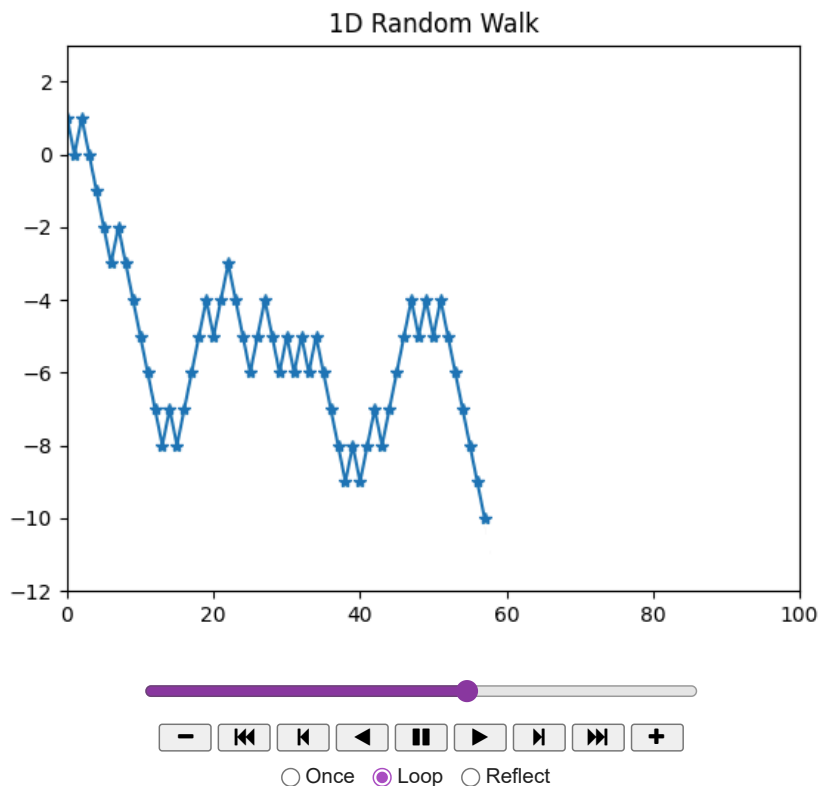
An elementary example of a random walk is the random walk on the integer number line $\mathbb{Z}$ which starts at 0 and at each step moves +1 or −1 with equal probability.

```
import numpy as np
import matplotlib.pyplot as plt


steps = 100

# random steps: -1 or +1
dx = np.random.choice([-1, 1], steps) #1st argument: possible values to choose from, 2nd argument: no. of samples
# positions: cumulative sums of steps
x = np.cumsum(dx)

# Plot
plt.plot(x, marker='*')
#y-values given by x
#x-values given by index positions of x

plt.title("1D Random Walk")
plt.xlabel("Step")
plt.ylabel("Position")
plt.show()
```



Let's animate the random walk.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

def update(frame):
    line.set_data(range(frame), x[:frame])

steps = 100

dx = np.random.choice([-1, 1], steps)
x = np.cumsum(dx)

plt.title("1D Random Walk")

#set the visible range of the plot
plt.xlim(0, len(x))
plt.ylim(min(x)-1, max(x)+1)

line = plt.plot([], [], marker='*')[0]

ani = FuncAnimation(plt.gcf(), update, frames=len(x), interval=100, blit=False)

HTML(ani.to_jshtml())
```

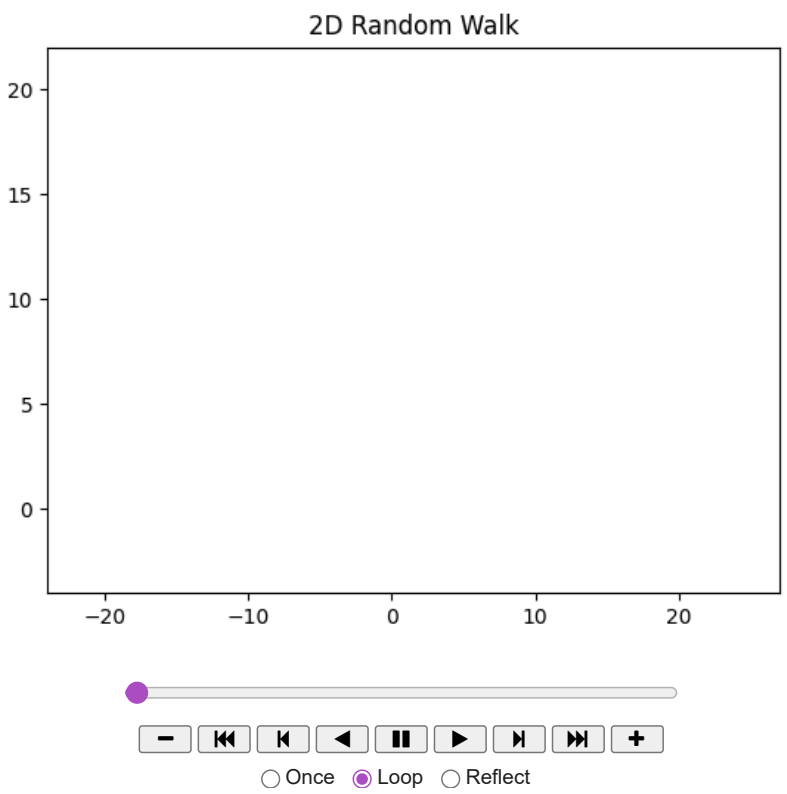


Here's a simulation of a random walk on $\mathbb{Z} \times \mathbb{Z}$.

```
import numpy as np
import matplotlib.pyplot as plt


steps = 1000

# random steps -1 or +1 in each axis
dx = np.random.choice([-1, 1], steps)
dy = np.random.choice([-1, 1], steps)

# cumulative sums denote positions
x = np.cumsum(dx)
y = np.cumsum(dy)

plt.plot(x, y, color="#CBAACB", linewidth=2)
plt.title("2D Random Walk")
plt.show()
```

Let's see the animated version.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

def update(frame):
    line.set_data(x[:frame], y[:frame])

steps = 500

dx = np.random.choice([-1, 1], steps)
dy = np.random.choice([-1, 1], steps)

x = np.cumsum(dx)
y = np.cumsum(dy)


plt.title("2D Random Walk")

#set the visible range of the plot
plt.xlim(min(x)-1, max(x)+1)
plt.ylim(min(y)-1, max(y)+1)

line = plt.plot([], [], color="#CBAACB", linewidth=2)[0]

ani = FuncAnimation(plt.gcf(), update, frames=len(x), interval=20, blit=False)

HTML(ani.to_jshtml())
```



Art using the random walk.

```
import numpy as np
import matplotlib.pyplot as plt


steps = 1000

dx = np.random.choice([-1, 1], steps)
dy = np.random.choice([-1, 1], steps)

x = np.cumsum(dx)
y = np.cumsum(dy)

# color palette
colors = ["#FFB7B2", "#B5EAD7", "#AEC6CF", "#FFDAC1", "#CBAACB"]

# Plot segment by segment with random colors
for i in range(steps - 1):
    col = np.random.choice(colors)
    plt.plot(x[i:i+2], y[i:i+2], color=col, linewidth=100, alpha=0.85)
    # draws a line segment between (x_i,y_i) and (x_{i+1},y_{i+1})
    # transparency is 0.85, width of line segments is 100

plt.axis("off")
plt.show()
```

Random walks are used in algorithmic art to create visual patterns by simulating random movement on a grid or in a continuous space. They also play an important role in many areas like physics, chemistry, biology, psychology, financial economics, semiconductor manufacturing and computer science. Random walks have been used to estimate the size of the Web and it is possible to estimate π with a random walk. PageRank, an algorithm used by Google Search to rank web pages in their search engine results, may be viewed as a way to organize random walk of various lengths.

4. Generating Random Text

Language recognition systems work by constantly predicting what's coming next. Having heard the first i words they try to generate a prediction of the $i+1$ th word. An important underlying idea is to model language generation as a random walk on the words of a dictionary.

Markov generation is a way to generate new text with words and phrases similar to the original text. These algorithms are similar to parts of a Large Language Model (LLM), which is the key component of a chatbot.

The key idea here is that we can use any existing text to construct a statistical model of the language as used in that text, and from that generate random text that has similar statistics to the original.

We'll start by counting the number of times each word appears in a book. Then we'll look at pairs of words, and make a list of the words that can follow each word. We'll make a simple version of a Markov generator.

(a) Read the text file `holmes.txt` and find the list of unique words with their frequency of occurrence.

Python's built-in regular expression module allows you to search text patterns, extract matches and replace patterns.

```
import re
```

We will also use counters to store words along with their frequencies.

```
from collections import Counter
```

```
text = open("holmes.txt",'r').read()
cleanedText = re.sub('\W+',' ', text).lower().split()
uniqueCount = Counter(cleanedText)
print(uniqueCount)
print(len(uniqueCount))
#re.sub() is called on text, using a regular expression pattern
#to replace any sequence of non-alphanumeric characters with a single space
#The resulting string is converted to lowercase and split into a list of words
```

```
Counter({'the': 5623, 'i': 3038, 'and': 3018, 'to': 2744, 'of': 2655, 'a': 2642, 'in': 1766, 'that': 1752, 'it': 1734, 'you': 1502, 'he': 1483, 'was': 1410, 'his': 1159
7917
```

Let us display the 5 longest words

```
sorted(uniqueCount.keys(), key=len)[-5:]
#returns a new sorted list
```

```
['improbabilities',
 'accomplishments',
 'conventionalities',
 'indistinguishable',
 'disproportionately']
```

A sequence of two words is called a **bigram** and a sequence of three words is called a **trigram**. In general, a sequence of n words is called an n**-gram**.

(b) Construct a dictionary `bigrams` with keys as the words in `holmes.txt`. The value corresponding to a key `k` is another dictionary whose keys correspond to words `w` that appear immediately after `k` with the corresponding value being the frequency of occurrence of `w` after `k`.

```
bigrams = {}

for i in range(len(cleanedText)-1):
    w1 = cleanedText[i]
    w2 = cleanedText[i+1]

    if w1 not in bigrams:
```

```
            bigrams[w1] = {}

        if w2 not in bigrams[w1]:
            bigrams[w1][w2] = 1
        else:
            bigrams[w1][w2] += 1
```

```
bigrams
```


```
{'the': {'adventures': 1,
  'red': 14,
  'boscombe': 11,
  'five': 4,
  'man': 50,
  'twisted': 6,
  'adventure': 14,
  'blue': 9,
  'speckled': 4,
  'engineer': 3,
  'noble': 5,
  'beryl': 3,
  'copper': 11,
  'whole': 35,
  'most': 40,
  'world': 13,
  'softer': 1,
  'observer': 2,
  'veil': 1,
  'trained': 1,
  'late': 3,
  'home': 1,
  'drowsiness': 1,
  'drug': 4,
  'fierce': 2,
  'study': 6,
  'official': 8,
  'case': 43,
  'trepoff': 1,
  'singular': 7,
  'atkinson': 1,
  'mission': 1,
  'reigning': 3,
  'readers': 1,
  'daily': 1,
  'twentieth': 1,
  'well': 4,
  'dark': 8,
  'blind': 2,
  'room': 61,
  'scent': 1,
  'bell': 16,
  'chamber': 4,
  'corner': 22,
  'fire': 24,
  'inside': 7,
  'firelight': 1,
  'leather': 2,
  'edges': 2,
  'sole': 3,
  'london': 3,
  'right': 19,
  'medical': 1,
  'ease': 1,
  'thing': 9,
  'distinction': 1,
  'steps': 9,
  'hall': 16,
```


(c) Use `bigrams` to generate a random text T of 20 words where the first word in T is a random word from the input file and the subsequent words are chosen according to `bigrams`. That is, the new text has the same relationships between consecutive words as in the original.

- Starting with any word that appears in the text, we look up its possible successors and choose one at random.
- Then, using the chosen word, we look up its possible successors and choose one at random.
- Repeat this process to generate as many words as desired.

```
word = random.choice(list(bigrams.keys()))
generated = [word]
for i in range(19):
    next_words = list(bigrams[word].keys())
    counts = list(bigrams[word].values())
    word = random.choices(next_words, weights=counts)[0]
    generated.append(word)

gentext = " ".join(generated)

print(gentext)
```

```
framework of dingy two steps until one night or on the stage ha you have a curtain in at first
```

(d) Use trigrams to generate random text.

```
trigrams = {}

for i in range(len(cleanedText)-2):
    w1 = cleanedText[i]
    w2 = cleanedText[i+1]
    w3 = cleanedText[i+2]

    key = (w1, w2)

    if key not in trigrams:
        trigrams[key] = {}

    if w3 not in trigrams[key]:
        trigrams[key][w3] = 1
```

```
        else:
            trigrams[key][w3] += 1
```

```
w1, w2 = random.choice(list(trigrams.keys()))
generated = [w1, w2]

for i in range(18):
    key = (w1, w2)

    if key not in trigrams:
        break

    next_words = list(trigrams[key].keys())
    counts = list(trigrams[key].values())

    w3 = random.choices(next_words, weights=counts)[0]

    generated.append(w3)

    w1, w2 = w2, w3
```

```
gentext = " ".join(generated)
print(gentext)
```

```
uncongenial atmosphere three gilt balls and a strongly marked german accent you have certainly been favoured with extraordinary luck during
```

5. Monty Hall Game

You are on a game show and given the choice of picking one of the three boxes (X, Y, Z). Two of the boxes are empty and one contains a gift. You pick a box, say X, and the host (who knows what is inside each of the boxes) opens another box, say Y, which is empty. You now have the option of retaining your choice (X) or switching it (to Z).

Simulate 2 strategies: first one is to always switch, second one is to never switch.

What do you observe from this simulation? Do you think that it to your advantage to switch your choice in order to get the gift? This advantage is defined as the fraction of times switching the choice leads to a win when the game is played 1000 times.

```
def gameShow(decide_to_switch):

    # Create three boxes with one of them having a gift
    box = [False, False, False]
    gift = random.randint(0,2)
    box[gift] = True

    # Player picks one of the boxes "X"
    loc_x = random.randint(0,2)

    # Host picks an empty box "Y" other and reveals it
    for i in range(3):
        if i != gift and i != loc_x:
            loc_y = i
            break

    # Locate the box "Z"
    loc_z = 3 - (loc_x + loc_y)

    # Player should switch or not
    if decide_to_switch:
        return box[loc_z]
    else:
        return box[loc_x]
```

```
trials = 1000
success_with_switch = 0
for i in range(trials):
    success = gameShow(decide_to_switch=True)
    if success:
        success_with_switch = success_with_switch + 1

success_with_no_switch = 0
for i in range(trials):
    success = gameShow(decide_to_switch=False)
    if success:
        success_with_no_switch = success_with_no_switch + 1

print(f"Out of {trials} experiments {success_with_switch} succeeded upon switching")
print(f"Out of {trials} many experiments {success_with_no_switch} succeeded upon not switching")
```

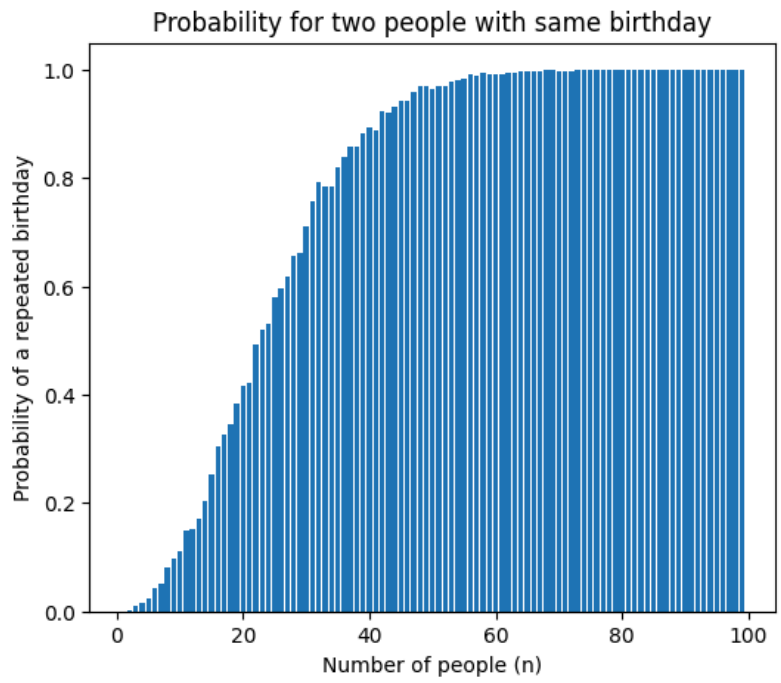
```
Out of 1000 experiments 674 succeeded upon switching
Out of 1000 many experiments 344 succeeded upon not switching
```

6. Birthday Paradox

For each $n \in [100]$, generate n random birthdays (i.e., n random integers between 1 and 366) and determine the number x of times that the n birthdays have an equal pair. Treat $x/100$ as the probability of two same birthdays among n random birthdays. Here, $[k]$ denotes the set $\{1, 2, \ldots, k\}$. Plot the probability values obtained as a function of n. What is the minimum n that guarantees two same birthdays with high probability? Identify a lower bound for n that makes the probability of two same birthdays greater than 0.5.

```
import random
import matplotlib.pyplot as p


def probSameBirthday(people_count, repetitions=1000):
    ''' Function to calculate the probability that two have the same birthday among people_count many people '''
    count = 0

    for r in range(repetitions):
        bday_list = []
```

```
            # Generate n random birthdays
            for i in range(people_count):
                # Generate another random day in a year as a birthday
                bday_list.append(random.randrange(1,366))

            # Checking if there are two with the same birthday
            if len(bday_list) != len(set(bday_list)):
                count = count + 1

    return count/repetitions

probability_values = [probSameBirthday(i) for i in range(1,100)]

p.figure(figsize=(6,5))
p.bar(range(1,100), probability_values)

p.title("Probability for two people with same birthday")
p.ylabel("Probability of a repeated birthday")
p.xlabel("Number of people (n)")
p.show()

for i, prob in enumerate(probability_values):
    if prob > 0.5:
        print("Probability exceeds 0.5 for n =", i+1)
        break
```




```
Probability exceeds 0.5 for n = 23
```

7. Law of Large Numbers

Consider the experiment of rolling a (unbiased six-sided) die. Let X denote the outcome.

- What is X?
- What is the average value of X if we repeat the experiment n times for sufficiently large n?

(a) Write functions `avgDice(n)` and `avgBiasedDice(n)` that call `simulateDice(n)` and `simulateBiasedDice(n)` resp. and computes the average of the `n` outcomes.

```
def avgDice(n):
    freq = simulateDice(n)
    avg = sum((i+1) * freq[i] for i in range(6))/n
    return avg
```

```
def avgBiasedDice(n,probs=[1/6,1/6,1/6,1/6,1/6,1/6]):
    freq = simulateBiasedDice(n,probs)
    avg = sum((i+1) * freq[i] for i in range(6))/n
    return avg
```

(b) Illustrate the **Law of Large Numbers** that states that if you repeat a random experiment a large number of times and average the result, what you obtain is close to the "expected value".

Consider the experiment of rolling a die and let X denote the outcome

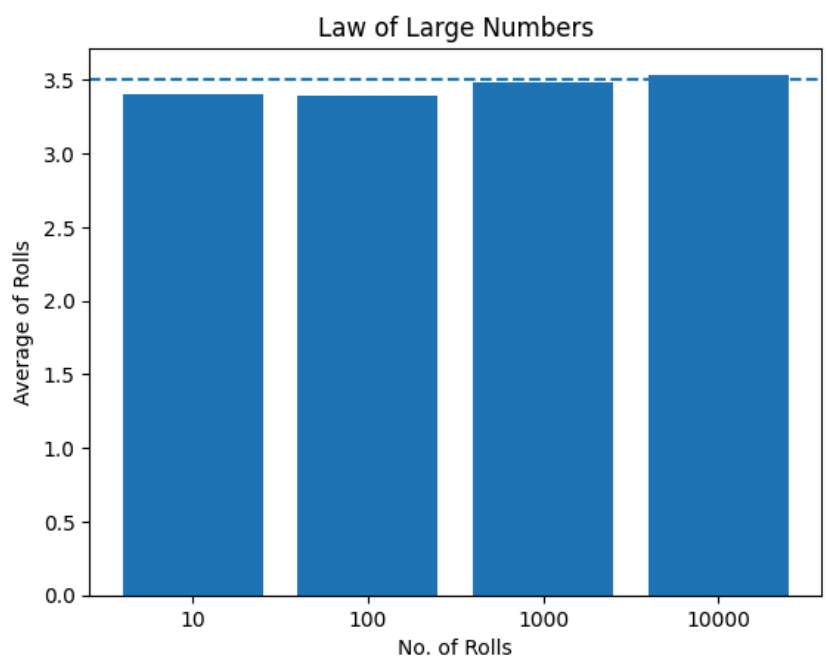
- If the die is unbiased, then $E[X]$ is $\sum_{i=1}^{6} \frac{1}{6} \cdot i = 3.5$
- If the die is biased with $[p_1, p_2, p_3, p_4, p_5, p_6]$, then $E[X]$ is $\sum_{i=1}^{6} p_i \cdot i$

Perform the experiment n times and let X_i denote the outcome of the i th trial

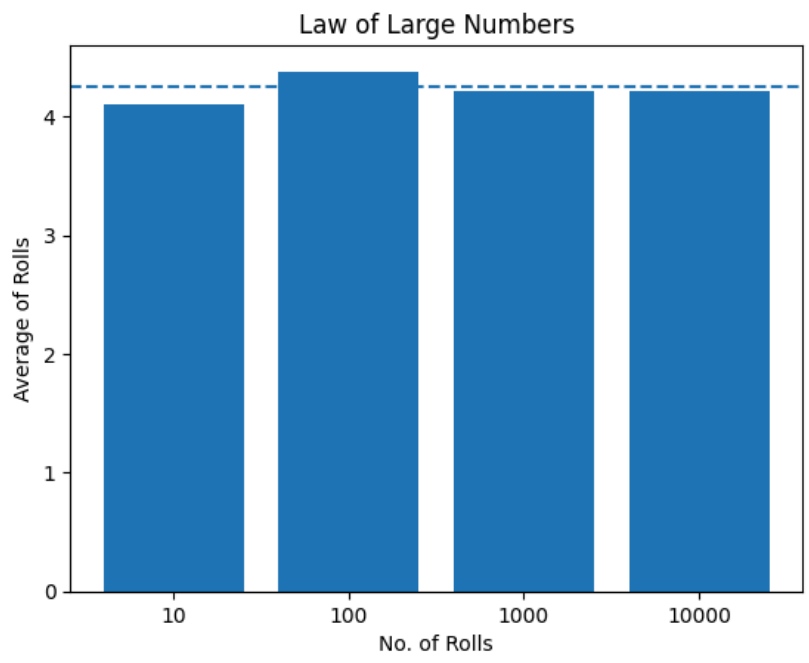
- By law of large numbers, for sufficiently large n, $\frac{\sum_{i=1}^{n} X_i}{n} \approx E[X]$
- That is average value of X is close to $E[X]$

```
n_values = [10, 10**2, 10**3, 10**4]
averages = []
for n in n_values:
    averages.append(avgDice(n))

x_positions = range(len(n_values))

p.bar(x_positions, averages)
# place the bars at 0, 1, 2, 3

p.xticks(x_positions, n_values)
```

```
# relabel 0, 1, 2, 3 as 10, 10**2, 10**3, 10**4

p.axhline(3.5, linestyle='--')
# draws a horizontal line at y = 3.5

p.title("Law of Large Numbers")
p.ylabel("Average of Rolls")
p.xlabel("No. of Rolls")
p.show()
```




```
probs = [0.05, 0.10, 0.15, 0.20, 0.25, 0.25]
n_values = [10, 10**2, 10**3, 10**4]
averages = []
for n in n_values:
    averages.append(avgBiasedDice(n,probs))

x_positions = range(len(n_values))

p.bar(x_positions, averages)

p.xticks(x_positions, n_values)

p.axhline((0.05*1 + 0.10 * 2 + 0.15 * 3 + 0.20 * 4 + 0.25 *5 + 0.25 *6), linestyle='--')

p.title("Law of Large Numbers")
p.ylabel("Average of Rolls")
p.xlabel("No. of Rolls")
p.show()
```



8. Central Limit Theorem

From a set $S = \{1, 1, 2, 3, 5, 5, 5, 7, 8, 10, 12\}$ of numbers, pick $n = 1$ numbers at random and compute the average. Repeat this 10^5 times and plot the frequency of the average values (i.e., how many times each average value occurs) as a histogram. Repeat this experiment with $n \in \{5, 10, 30, 100, 1000\}$.

```
#Illustrate Central limit theorem (TODO: reword a bit)
import random
import matplotlib.pyplot as p

values = [1,1,2,3,5,5,5,7,8,10,12]

# list of sample mean

# number of sampling
nsamples = 100000

# sample size
samplesize = [1,5,10,30,100,1000]
```

```
meansample = []

# for each sample size (1 to 1000)
for i in samplesize:
    eachaverage = []
    for j in range(0, nsamples):

        # sampling i sample from values
        rc = random.choices(values, k=i)

        # find the average
        eachaverage.append(sum(rc)/len(rc))

    # append the average obtained to a list
    meansample.append(eachaverage)

f, plots = p.subplots(len(samplesize),figsize=(21,18))
i = 0
for avgs in meansample:
    plots[i].set_title(f"Histogram of the outcome of tossing dice {samplesize[i]} times")
    plots[i].hist(avgs, 200, density=True)
    plots[i].plot()
    i = i + 1
```


9. Randomness Extraction

Given a coin with $P(H) = p$ and $P(T) = 1 - p$, let us generate a unbiased coin toss.

ID1110 Introduction to Programming (in Python)

Topic for Today's Lecture: Testing programs

References:

- https://docs.python.org/3/library/doctest.html
- https://docs.python.org/3/library/unittest.html
- Think Python (Edition 3) by Allen B. Downey

Testing Code

An important aspect of software development is testing your code to ensure that it works correctly. However, even with extensive testing, the code may still contain subtle bugs. **Testing cannot guarantee correctness**. It can only show the presence of bugs, not ascertain their absence.

Program analysis is a systematic, often non-programming process of examining code or execution behavior to understand, verify, and optimize its properties.

Python's standard library provides the `doctest` module that helps to **test code**. We have seen docstrings in the context of documenting a class/function -- that is, to explain what it does. It is also possible to use a docstring for testing purposes.

Consider the following version of the functions `throwDice` and `simulateDice` with docstrings that include test cases (tests).

```
import random
def throwDice():
    """
    Returns a random integer between 1 and 6

    Output is between 1 and 6:
    >>> 1 <= throwDice() <= 6
    True

    Each of the 100 calls' output is between 1 and 6:
    >>> all(1 <= throwDice() <= 6 for i in range(100))
    True
    """
    return random.randint(1,6)
```

Each test typically tests a specific unit of code such as a function, method or class. Such tests are called **unit tests**.

Each test begins with `>>>` and is followed by an expression, usually involving a function call. The following line indicates the value the expression should have if the function works correctly.

Note: The `all` built-in function returns `True` if all elements of the iterable are true or if the iterable is empty.

```
def simulateDice(n):
    """
    Simulates rolling a fair die n times and returns the outcome frequencies.

    Output has exactly 6 entries:
    >>> len(simulateDice(1))
    6
    >>> len(simulateDice(1000))
    6

    Sum of frequencies equals number of die rolls:
    >>> sum(simulateDice(0))
    0
    >>> sum(simulateDice(25))
    25
    >>> sum(simulateDice(10000))
    10000

    Corner case output:
    >>> simulateDice(0)
    [0, 0, 0, 0, 0, 0]
    """
    freq = [0,0,0,0,0,0]
    for i in range(n):
        r = throwDice()
        freq[r-1] = freq[r-1] + 1
    return freq
```

To run these tests, import the `doctest` module and run a function called `testmod`.

```
import doctest
doctest.testmod()
```

```
TestResults(failed=0, attempted=8)
```

`testmod` inspects your functions', methods' and classes' docstrings looking for statements preceeded by `>>>`, each followed on the next line by the given statement's expected output (if any).

- It then executes those statements and confirms that they produce the expected output.
- If the result of a test case is the expected value, the test **passes**, otherwise it **fails**.
- If there are failed test cases, `testmod` reports errors indicating which tests failed so you can locate and fix the problems in your code.

To see what happens when a test fails, here's an incorrect version of `simulateDice`.

```
def simulateDiceIncorrect(n):
    """
    Simulate rolling a fair die n times and return frequencies.
```

```
Output has exactly 6 entries:
>>> len(simulateDiceIncorrect(1))
6
>>> len(simulateDiceIncorrect(1000))
6

Sum of frequencies equals number of rolls:
>>> sum(simulateDiceIncorrect(0))
0
>>> sum(simulateDiceIncorrect(25))
25

>>> sum(simulateDiceIncorrect(10000))
10000

Corner case:
>>> simulateDiceIncorrect(0)
[0, 0, 0, 0, 0, 0]
"""
freq = [0,0,0,0,0,0]
for i in range(n):
    r = throwDice()
    freq[r-1] = freq[r-1] + 2
return freq
```

And here's what happens when we test it.

```
doctest.testmod()
```

```
**********************************************************************
File "__main__", line 14, in __main__.simulateDiceIncorrect
Failed example:
    sum(simulateDiceIncorrect(25))
Expected:
    25
Got:
    50
**********************************************************************
File "__main__", line 17, in __main__.simulateDiceIncorrect
Failed example:
    sum(simulateDiceIncorrect(10000))
Expected:
    10000
Got:
    20000
**********************************************************************
1 items had failures:
   2 of   6 in __main__.simulateDiceIncorrect
***Test Failed*** 2 failures.
TestResults(failed=2, attempted=14)
```

```
doctest.testmod(verbose=True)
```

```
    6
ok
Trying:
    sum(simulateDiceIncorrect(0))
Expecting:
    0
ok
Trying:
    sum(simulateDiceIncorrect(25))
Expecting:
    25
**********************************************************************
File "__main__", line 14, in __main__.simulateDiceIncorrect
Failed example:
    sum(simulateDiceIncorrect(25))
Expected:
    25
Got:
    50
Trying:
    sum(simulateDiceIncorrect(10000))
Expecting:
    10000
**********************************************************************
File "__main__", line 17, in __main__.simulateDiceIncorrect
Failed example:
    sum(simulateDiceIncorrect(10000))
Expected:
    10000
Got:
    20000
Trying:
    simulateDiceIncorrect(0)
Expecting:
    [0, 0, 0, 0, 0, 0]
ok
Trying:
    1 <= throwDice() <= 6
Expecting:
    True
ok
Trying:
    all(1 <= throwDice() <= 6 for i in range(100))
Expecting:
    True
ok
1 items had no tests:
    __main__
2 items passed all tests:
   6 tests in __main__.simulateDice
   2 tests in __main__.throwDice
**********************************************************************
1 items had failures:
   2 of   6 in __main__.simulateDiceIncorrect
14 tests in 4 items.
12 passed and 2 failed.
***Test Failed*** 2 failures.
TestResults(failed=2, attempted=14)
```

The output includes the example that failed, the value the function was expected to produce, and the value the function actually produced.

Execute `python -m doctest -v script_name.py` to run the `doctest` module as a script and test the doctests in `script_name.py` with verbose output.

Alternatively, you can also run doctests from within the module using the following snippet.

```
if __name__ == "__main__":
    import doctest
    doctest.testmod(verbose=True)
```

To test functions one by one, you can use the `doctest` module's `run_docstring_examples` function. To make this function easier to use, consider the following function which takes a function object as an argument.

```
from doctest import run_docstring_examples

def run_doctests(func):
    run_docstring_examples(func, globals(), name=func.__name__,verbose=True)
```

This function takes a function object and runs its doctests. Passing `globals()` as the second argument ensures that the doctest has access to the variables, functions, classes and imports defined in your script.

```
run_doctests(simulateDiceIncorrect)
```

```
run_doctests(simulateDice)
```

Python also provides another tool for running automated tests, called `unittest`. It is more powerful but more sophisticated to use.
Here's an example.

```
def add(a, b):
    '''Add two numbers.

    >>> add(2, 2)
    4
    '''
    return a + b
```

```
from unittest import TestCase

class TestExample(TestCase):

    def test_add(self):
        result = add(2, 2)
        self.assertEqual(result, 4)
```

First we import `TestCase`, which is a class in the `unittest` module. To use it, we have to define a new class that inherits from `TestCase` and provides at least one test method. The name of the test method must begin with `test` and should indicate which function it tests.

In this example, `test_add` tests the `add` function by calling it, saving the result, and invoking `assertEqual`, which is inherited from `TestCase`.

`assertEqual` takes two arguments and checks whether they are equal.

In order to run this test method, we have to run a function in `unittest` called `main` and provide several keyword arguments. The following function shows the details -- if you are curious, check out the official documentation.

```
import unittest

def run_unittest():
    unittest.main(argv=[''], verbosity=0, exit=False)
```

`run_unittest` does not take `TestExample` as an argument -- instead, it searches for classes that inherit from `TestCase`. Then it searches for methods that begin with `test` and runs them. This process is called **test discovery**.

Here's what happens when we call `run_unittest`.

```
run_unittest()
```

`unittest.main` reports the number of tests it ran and the results. In this case `OK` indicates that the tests passed.

To see what happens when a test fails, we'll add an incorrect test method to `TestExample`.

```
from unittest import TestCase

class TestExample(TestCase):

    def test_add(self):
        result = add(2, 2)
        self.assertEqual(result, 4)

    def test_add_broken(self):
        result = add(2, 2)
        self.assertEqual(result, 100)
```

Here's what happens when we run the tests.

```
run_unittest()
```

The report includes the test method that failed and an error message showing where. The summary indicates that two tests ran and one failed.